OBRA: REAL-TIME DRAWING RECOGNITION USING CNN AND CONCEPTNET IN A 2D ADVENTURE GAME

A Thesis Proposal

Presented to the Faculty of the

Department of Computer, Information Sciences and Mathematics

University of San Carlos

In Partial Fulfillment

of the Requirements for the Degree

BACHELOR OF SCIENCE IN COMPUTER SCIENCE

By:

ALARDE, RUSK RYE IOAN C.

API, ERIKA BIANCA E.

GATCHALIAN, KENT JESTER P.

GUTIERREZ, LADY KAINA V.

CHRISTINE F. PEÑA, MMath
Faculty Adviser

AUGUST 2026

ABSTRACT

Convolutional neural networks are established in game agents and content generation, yet rarely used to interpret what a player creates; drawing games treat recognition as an end in itself, guessing and scoring a drawing before discarding it. This study presents OBRA, a two-dimensional adventure game set in the Philippines in which a freehand drawing is recognized, granted a real-world-grounded ability, and animated from the player's own strokes as a playable entity. A convolutional neural network trained on Google Quick, Draw! classifies each drawing into a curated fifty-class vocabulary, served locally through ONNX Runtime; a prediction is acted upon only when confidence and top-two margin clear defined thresholds, so an uncertain prediction prompts a redraw. Each class is mapped offline to an ability derived from ConceptNet through six affordance-bearing relations, together with a guarded IsA traversal where a class carries no usable edge of its own, selected by game requirement first and assertion weight second; forty-eight of fifty classes are grounded this way, and the remaining two are hand-authored and marked as such. Recognition is reported as per-class precision and recall with a confusion matrix, on a held-out Quick, Draw! split and on the independent TU-Berlin corpus, which covers forty of the fifty classes. The game is evaluated with at least thirty participants using standardized usability, immersion, and player-experience instruments alongside in-game telemetry, and its design and cultural content are separately reviewed by game development practitioners and by students of anthropology, sociology, and history. OBRA offers a reusable pattern for grounding interactive behavior in documented knowledge.

TABLE OF CONTENTS

ABSTRACT ii
TABLE OF CONTENTS iii
LIST OF TABLES iv
LIST OF FIGURES v
LIST OF EQUATIONS vi
CHAPTER 1 INTRODUCTION 1
CHAPTER 2 REVIEW OF RELATED LITERATURE 10
CHAPTER 3 TECHNICAL BACKGROUND 19
CHAPTER 4 DESIGN AND METHODOLOGY 60
4.1 Research Environment and Respondents 60
4.2 Research Instrument and Sources of Data 71
4.3 Research Procedure 100
4.4 Concept 106
4.5 Analysis and Design 109
4.6 Development Model 135
4.7 Development Approaches 140
4.8 Software Development Tools 149
4.9 Project Management 151
4.10 Verification, Validation, and Testing 168
BIBLIOGRAPHY 182
APPENDICES
Appendix A: Transmittal Letter, Call for Participants, and Screening Questionnaire
Appendix B: Gameplay Evaluation Instruments
Appendix C: ConceptNet Ability Attribution Table
Appendix D: Software Requirements Specification
Appendix E: Wireframes and Initial Design
CURRICULUM VITAE

LIST OF TABLES

Table 1. Class Vocabulary, Spawn Behavior,
and the Relation Backing Each Group..... 27

Table 2. System Technologies and Constraints..... 52

Table 3. Data Sources at a Glance..... 85

Table 4. Dataset Profile: Google Quick, Draw! (Primary Training)..... 85

Table 5. Class Distribution Data..... 88

Table 6. Dataset Profile: TU - Berlin Sketch Dataset..... 94

Table 7. Data Source Profile: ConceptNet..... 96

Table 8. Data Coverage..... 97

Table 9. Attribution Modes..... 99

Table 10. Software Development Tools..... 149

Table 11.Table of Roles and Responsibilities..... 164

Table 12. Table of Expenses..... 166

Table 13. The Five Evaluation Aspects,
Their Measures, and Their Reporting..... 173

Table 14. Gameplay Performance Measures
and Their Instrumentation..... 175

LIST OF FIGURES

Figure 1. General Convolutional Neural Network (CNN) Classification Pipeline..... 36

Figure 2. Component Architecture of the Proposed System.....57

Figure 3. The Four Stages of One Interaction..... 58

Figure 4.The Request and Response Cycle..... 59

Figure 5. CNN Accuracy Degradation Curve..... 90

Figure 6. OBRA Curated 50-Class Selection: Confusion Matrix Validator..... 91

Figure 7. Conceptual Framework of OBRA..... 107

Figure 8. System Architecture (component and deployment view)..... 114

Figure 9. Request and Response Cycle (sequence view)..... 115

Figure 10. Client and Backend Development..... 116

Figure 11. Structural / Class Diagram (key types)..... 117

Figure 12. Use Case Diagram..... 118

Figure 13. Data Flow Diagram (Level 1)..... 119

Figure 14. Activity Diagram (draw to outcome)..... 120

Figure 15. Game Design Flow..... 134

Figure 16. Iterative and Incremental Development Model for OBRA..... 139

Figure 17. Design direction approach of OBRA..... 141

Figure 18. Gantt Chart of Activities for Summer 2026, A.Y. 2025-2026..... 154

Figure 19. Gantt Chart of Activities for First Semester, A.Y. 2026-2027..... 158

LIST OF EQUATIONS

Equation 1. Softmax Function..... 37
Equation 2. Confidence..... 39
Equation 3. Margin..... 40
Equation 4. Abstention (Dual-Gate Decision Rule)..... 40
Equation 5. Accuracy..... 42
Equation 6. Precision (Per Class)..... 42
Equation 7. Recall (Per Class)..... 43
Equation 8. F1-Score (Per Class)..... 43
Equation 9. Confusion Matrix..... 44
Equation 10. Class Diversity..... 44
Equation 11. Redraw Rate..... 45
Equation 12. End-to-End Response Time..... 45
Equation 13. Proportional-Derivative (PD) Joint Control..... 46

CHAPTER 1
INTRODUCTION

This chapter establishes the grounds on which the study is undertaken. It presents the rationale, situating drawing-based interaction within the history of games as a domain of computing research and identifying the two gaps the study addresses: that no established approach converts a recognized player drawing into a functional, ability-bearing game entity, and that where games do attach effects to recognized input, those effects are assigned by designers rather than grounded in documented knowledge. It then articulates the statement of the problem and the general and specific objectives that answer it, specifying what the system must classify, attribute, animate, and be evaluated against. Furthermore, it identifies the parties the study serves, from researchers in sketch recognition to independent game studios and teachers of Philippine culture. Finally, it delimits the study's scope and acknowledges the constraints on its findings, distinguishing the portable draw-to-ability mechanic from the Philippine setting in which it is realized.

1.1 Rationale of the Study

Video games have been tied to computing research since the field’s beginnings, and they remain one of its most demanding application domains, serving as testbeds for artificial-intelligence methods while constituting a major application area in their own right (Yannakakis & Togelius, 2018). Real-time rendering, physics simulation, and procedural generation all matured under the pressure of a frame budget, where a method is useful only if it is both correct and imperceptibly fast.

Within that domain, one family of games is distinguished less by what it renders than by what it asks of the player. Creative games hand the authoring tools to the player and treat what the player makes as the content of the game. User-generated content has become a cornerstone of the industry: Counter-Strike and DOTA began as player modifications of existing games, while

platforms such as Roblox and Dreams supply only rules, materials, and an editor, and let players build the rest (Duan et al., 2022). Such tools amplify what people can make (Shneiderman, 2007), but they carry a technical consequence: when the player authors the content, the software must interpret something it has never seen and decide, in real time, what to do with it. The problem shifts from retrieval to recognition.

The convolutional neural network is the technology most responsible for making that shift practical. Since a CNN trained directly on raw pixel input first learned to play classic arcade games at a human level (Mnih et al., 2015), the architecture has been adapted throughout game development for tasks with no obvious hand-coded solution, from agents that perceive their environment through pixels to procedural systems that generate levels and assets (Summerville et al., 2018). Across nearly all of these applications, though, the network's output feeds a process the player never touches directly. The interpretation side closest to the player — reading what the player made and acting on it within the same session — remains comparatively unexplored.

Drawing is the most demanding version of this input problem, and among the most accessible, since conventional interfaces are cumbersome for novices while freehand strokes are not (Olsen et al., 2009). Yet a drawing is harder to read than a photograph: a photograph is dense with color, texture, lighting, and background, whereas a drawing, specifically a sketch, is a handful of thin strokes on an empty field, made differently by every person attempting the same object, so a model must infer identity from sparse and highly variable lines (Eitz et al., 2012). Progress on this narrower problem has tracked the availability of data. The TU-Berlin sketch dataset first established sketch recognition as a formal benchmark (Eitz et al., 2012), and Google's Quick, Draw! later supplied tens of millions of labeled drawings at a scale that made robust, real-time recognition practical (Google Creative Lab, 2016; Ha & Eck, 2017) — and fast enough that the interaction still feels direct, a threshold long established at roughly a tenth of a second (Miller, 1968; Nielsen, 1993).

Where drawing has been used in games, however, the recognized label has been treated as a terminal result. In drawing-guessing games, the guess earns a point and the drawing is discarded. Ōkami mapped drawn marks onto abilities, but from a small fixed set of authored gestures rather than open recognition (Clover Studio, 2006). Scribblenauts showed that a summoned entity can carry usable properties, so that a ladder affords climbing and a bird affords flight, but the player types a word rather than drawing one, and the entity that appears is the developer's artwork, not the player's (5th Cell, 2009). No existing game closes the loop: recognize a genuine freehand drawing, attach to the recognized class a capability grounded in documented knowledge rather than designer preference, and return the player's own strokes as the thing they control. Two gaps follow. First, no established approach converts a recognized player drawing into a functional, ability-bearing game entity that the player then uses to progress, rather than a mere labeled guess. Second, where games do attach effects to recognized input, those effects are assigned arbitrarily by designers rather than grounded in documented knowledge about what the drawn thing can actually do.

This study addresses both gaps. Its central artifact is a convolutional neural network — the architecture that has driven the last decade of visual recognition by learning features directly from pixels (Krizhevsky et al., 2012; LeCun et al., 2015) — trained on the Quick, Draw! dataset to classify player drawings into a curated vocabulary of fifty classes and served through the ONNX Runtime at interactive speed. Because a classifier forced to choose will return a label for a scribble and an empty canvas alike, the recognizer is built to decline: predictions below a confidence or margin threshold are refused rather than acted upon. Each accepted class is resolved against ConceptNet, a commonsense knowledge graph, into a documented capability, so that a drawn bird is granted flight and a drawn key the ability to open (Speer et al., 2017). The recognized drawing is not replaced by generated artwork; the player's own vector strokes are animated procedurally into the entity they control. Because the pipeline is

assembled from open-source components — PyTorch for training, ONNX for portable inference, and the Godot engine for the game — it runs on ordinary consumer hardware without proprietary tools. With these components, the researchers will produce a game called OBRA.

The title OBRA is rooted in the linguistic and artistic heritage of the Philippines. Etymologically, the Filipino term is a direct loanword from the Spanish *obra*, which traces back to the Latin *opera* and *opus*, meaning "labor," “care,” "effort," or "work" (Real Academia Española, 2014). While it historically referred to physical labor or construction, its integration into Tagalog has elevated its primary meaning to designate a creative composition or "work of art," most prominently seen in the phrase *obra maestra* to honor an artist’s masterpiece (Almario, 2010). By choosing OBRA as the game's title, the project reframes the player's core mechanic of drawing and puzzle-solving as a form of deliberate artistic labor, directly turning their real-time drawings into interactive masterpieces. .

OBRA is deliberately a Filipino game. Local game development has long been constrained by an industry in which studios do outsourced work for foreign titles and Filipino players mostly play non-Filipino games (Cerda, 2021). OBRA separates two things that such games usually fuse: its draw-to-ability mechanic is culturally neutral and portable, while its Filipino character lives in its setting, story, characters, and art direction. The player is an apo searching for their Lola, a painter who has gone missing; guided by the spirit of their Lolo, and taking up her brush, the player travels through the worlds inside her paintings, from a bahay kubo near the Banaue Rice Terraces to a village pista and out to the open sea. That separation lets the study present a reusable drawing-to-ability system while grounding the game in a specific and meaningful cultural context.

1.2 Statement of the Problem

1.2.1 General Objective

This research aims to design, develop, and evaluate OBRA, a 2D adventure game in which a Convolutional Neural Network classifies freehand player drawings into a fixed fifty-class vocabulary, a ConceptNet-derived attribution table assigns each recognized class a documented real-world ability, and the player's own strokes are procedurally animated as a functional in-game entity, with recognition reported as per-class precision and recall with a confusion matrix, ability attribution reported with the provenance of every entry, and the resulting gameplay evaluated by around 45 participants using standardized usability, immersion, and player-experience instruments together with in-game telemetry, and a specialized questionnaire for game developer practitioners and cultural evaluators.

1.2.2 Specific Objectives

Specifically, the study aims to:

1. Design the game’s core framework, including a Philippine-based narrative (theme and setting), its visual assets, and its core mechanics — the level structures, controls, and physics systems.
2. Develop the drawing-recognition pipeline: a convolutional neural network trained to classify player drawings into a curated set of classes, together with the procedural animation of the player’s own strokes that brings a recognized drawing to life.
3. Establish a knowledge-based ability-attribution system in which each recognized class is attributed a predefined, real-world-grounded ability derived from a commonsense knowledge base where a usable assertion exists, with documented hand-authored exceptions where it does not.
4. Develop and integrate the game system, implementing the environment, levels, and physics-based movement within the Godot

engine as a single application, and incorporating a telemetry system to record player behavior and performance.

5. Evaluate the system and its design through user testing and review with at least forty-five participants, divided among play testers, game developer practitioners, and cultural evaluators, using post-play questionnaires and telemetry data to assess five aspects: CNN performance, recognition speed, gameplay performance, user experience, and gameplay effectiveness.

1.3 Significance of the Study

This study contributes both a method and a working artifact, and its value is best understood through the parties it serves.

Researchers in free-hand sketch recognition and human–computer interaction. The study's principal contribution is methodological. Whereas most applications of drawing recognition treat the recognized label as the end of the process, this study treats it as the beginning of a further stage, in which the recognized class is mapped to a meaningful consequence through documented relations drawn from a commonsense knowledge base rather than through arbitrary choice. In doing so it demonstrates a general pattern for grounding the behavior of an interactive system in structured knowledge, and it provides a concrete account of how a trained machine learning model can be integrated into real-time interactive software. These concerns recur wherever a learned model must operate reliably inside a responsive application, from sketch-based retrieval systems to on-device inference in consumer software.

Independent and small-scale game studios, particularly in the Philippines, and to developers of interactive media. For developers, the study offers a reusable pattern for turning player-generated drawings into functional, property-bearing game content. Because the entire system is assembled from freely available components and runs on ordinary consumer hardware, the

pattern is reproducible without specialized equipment or licensing, lowering the barrier for the solo developers and small teams that constitute the prevailing scale of game development in the country and that wish to incorporate on-device recognition into their work. More broadly, the technique of attaching meaningful, knowledge-grounded behavior to recognized user input is applicable beyond games to any interactive system that must respond to what its users create, including drawing-based learning software, creative tools, and assistive interfaces.

Teachers of Philippine culture and history, and to the cultural institutions and advocacy groups that commission interactive media. By situating a machine learning mechanic within a narrative rooted in Philippine setting and art direction, the study offers an example of local culture represented in an interactive and technically substantive form, which may serve as a point of engagement for learners with both the cultural material and the underlying technology.

Future researchers in sketch-based interaction and knowledge-grounded game design. The study is intended to serve as a building block. Its design, its vocabulary of recognizable classes, and its method for attributing abilities constitute a baseline against which subsequent work may measure improvements. Future studies might extend the knowledge-grounding step from a predefined mapping toward richer runtime reasoning, broaden the range of recognizable input, or explore more capable techniques for animating player-created content; in each case, the present study provides a documented point of departure.

1.4 Scope and Limitations

This study encompasses the design, development, and evaluation of a real-time drawing recognition pipeline integrated within a two-dimensional interactive environment. The research focuses on deploying a Convolutional

Neural Network (CNN) trained using PyTorch on the public Google *Quick, Draw!* dataset, tested against the *TU-Berlin Hand Sketch Image Dataset*, and served locally for high-speed inference via the ONNX runtime within the Godot engine.

To ensure reliable classification and logical defensibility, the scope of the recognition model is strictly bounded to a curated vocabulary of fifty predefined classes (twenty creatures, twenty-seven objects, and three primitives). The study does not attempt open-ended recognition of arbitrary drawings. Furthermore, the integration of commonsense knowledge is scoped to offline resolution; abilities are derived from ConceptNet in advance to form a fixed, inspectable mapping, explicitly excluding live semantic reasoning during runtime to prioritize system responsiveness.

In terms of visual output, the research investigates the procedural animation of the user's original vector strokes, explicitly excluding the procedural generation of new artwork or the use of pre-authored character sprites. The artifact is deployed purely as a single-player desktop application for macOS and Windows, excluding networked, multiplayer, or mobile architectures. Finally, the evaluation phase is scoped to assessing model recognition performance, system usability, and user experience. This is conducted through supervised testing with a minimum of thirty participants, the basis for which is given in Section 4.1.2, utilizing in-game telemetry and validated instruments (e.g., SUS, GEQ, and specialized subscales).

The findings and overall outcomes of this study are subject to several constraints that lie beyond the researchers' control, which are acknowledged in the interest of academic rigor:

- **Input Ambiguity and User Variance:** System accuracy and the resulting redraw rates are inextricably tied to the participant's drawing proficiency. Because freehand sketches are inherently ambiguous, misclassifications caused by messy, incomplete, or highly stylized inputs cannot be wholly isolated from the model's baseline performance.

- **Dataset Dependencies:** The reliance on crowd-sourced public training data affords the researchers limited control over data distribution and stylistic variance. This unevenness within the source data may influence the model's generalization capabilities and yield varying recognition performance across different classes.

- **Unpredictable Output Fidelity:** Because the procedural animation system relies entirely on the player's raw vector strokes, the visual fidelity and naturalness of the resulting on-screen motion cannot be guaranteed in advance. Consequently, there is no standardized, pre-authored visual baseline against which the animation quality can be objectively measured.

- **Generalizability of Findings:** The player-experience evaluation relies on a sample size of roughly thirty participants. This meets the conventional minimum for the descriptive use the study makes of its instruments — reporting SUS, UEQ-S, IEQ-SF, GEQ component, Godspeed Animacy, and QUESI Perceived Familiarity scores with confidence intervals, and comparing them against published benchmarks — but no a priori power analysis was conducted, and the study makes no claim to be powered for inferential hypothesis testing between subgroups. The sample is obtained by stratified random selection from a volunteer pool and is not universally representative, which constrains the generalizability of the usability and immersion findings to broader populations. Findings are accordingly reported as descriptive indicators of this sample's experience rather than as population estimates.

- **Undergraduate Resource Constraints:** The system is fundamentally a functional prototype developed within the strict time and resource limitations of an undergraduate thesis. Therefore, the research prioritizes the end-to-end validation of the core recognition-to-animation pipeline over commercial-scale content breadth or exhaustive software optimization.

CHAPTER 2
REVIEW OF RELATED LITERATURE

This chapter reviews the foundational technologies and existing paradigms that make the OBRA framework possible. It evaluates deep learning architectures, specifically CNNs versus attention-based models, for real-time drawing classification, and explores how commonsense knowledge bases like ConceptNet can ground those classified objects with real-world abilities. Furthermore, it analyzes current drawing-based game mechanics to highlight the gap between "recognition-as-scoring" and functional gameplay integration. Finally, it discusses the feasibility of procedurally animating a player's original drawing, and contextualizes the game's setting within the broader application of serious games and Philippine cultural representation.

2.1 Deep Learning and Image Classification

Deep learning has become the dominant approach to image classification, and convolutional neural networks (CNNs) in particular owe their success to learning visual features directly from data instead of relying on hand-engineered rules (LeCun, Bengio, & Hinton, 2015). Much of the recent literature affirms that CNNs remain both effective and computationally efficient for visual recognition, an advantage that is most pronounced when data or compute is limited (Maurício, Domingues, & Bernardino, 2023; Lee, Lee, & Song, 2021). Running alongside this consensus is a newer body of work showing that attention-based architectures now rival or exceed CNNs on large-scale benchmarks. The Vision Transformer (ViT) introduced by Dosovitskiy et al. (2021) applies self-attention to image patches and, after large-scale pretraining, matched state-of-the-art CNNs; Han et al. (2023) reach a similar conclusion in their survey, noting that ViTs achieve leading accuracy when given sufficient data. Interestingly, the influence has flowed in both directions, as Liu et al. (2022) demonstrated with ConvNeXt that a modernized pure-CNN design can match transformer performance, suggesting the two families are steadily converging. Direct comparisons reviewed

by Maurício et al. (2023) even find ViTs to be more robust to image perturbations and, on an F1 basis, occasionally superior to CNNs with fewer images.

This advantage, however, appears to be conditional rather than universal. Because ViTs lack the built-in inductive biases that let CNNs learn from limited examples, they tend to generalize poorly on small datasets (Lee, Lee, & Song, 2021), and CNNs correspondingly remain more efficient and often more accurate when training data is scarce (Maurício et al., 2023). These complementary strengths have also motivated hybrid CNN-transformer architectures. The strength of the convolutional approach is further underscored by Ali et al. (2023), who observe that CNN-based methods have improved recognition performance across image databases such as ImageNet, NORB, MNIST, and CIFAR-10. What emerges from this literature is broad agreement that CNNs excel on small, well-defined tasks while ViTs depend on large-scale data to reach their potential. Yet the comparisons behind this conclusion overwhelmingly target large natural-image benchmarks such as ImageNet or domain-specific medical imaging, and few examine performance on the small, curated, sparse-input class sets that must be recognized in real time. These are precisely the conditions a drawing-recognition game must operate under, which leaves a gap in guidance for selecting an architecture suited to bounded, responsive drawing classification.

2.2 Drawing Recognition

Freehand drawing (sketches) recognition is a distinct and harder problem than ordinary image classification because sketches lack color, texture, and shading, are highly abstract and vary widely from one person to the next (Xu et al., 2022). Despite this, researchers agree that recognizing freehand drawing is feasible and that large public datasets make training practical: the TU-Berlin set established the benchmark (Eitz et al., 2012), and Google's QuickDraw supplied roughly 50 million labeled drawings across 345 categories (Ha & Eck, 2017), enabling a wave of sketch-specific models. Building on these datasets, multiple studies report strong recognition performance. Xu et al.(2022) proposed a

Multi-Graph Transformer that represents drawings as connected graphs of strokes and reached 72.8% top-1 accuracy on QuickDraw, closing much of the gap to the CNN performance ceiling while outperforming recurrent models by up to 5%; the survey by Xu et al. (2023) systematically compares CNN, RNN, and graph architecture on drawing data; and more recent CNN-based systems report 89.02%, 87.22%, and 77.46% accuracy on 3, 10, and 50 classes of QuickDraw respectively (Ali et al., 2023). Broader reviews of visual recognition now position CNNs alongside vision transformers as complementary tools for such tasks (Amjoud & Amrouch, 2023).

The field remains divided on the best way to represent a drawing, and each representation carries a cost. Xu et al. (2022) found that deeper CNNs offer no advantage on drawings and can even lose accuracy, because redundant convolution and pooling discard information in sparse pixel space; sequence- and graph-based models exploit stroke order and dynamics but add complexity and still trail the CNN upper bound (Xu et al., 2022). A further limitation across approaches is that accuracy is typically measured on fixed benchmarks under clean conditions, and recognition degrades on messy, incomplete, or stylistically unusual drawings. The agreement that drawing recognition is achievable sits alongside unresolved trade-offs between raster and stroke-based methods and a reliance on benchmark accuracy; few studies address recognition for a small curated class set under real-time gameplay constraints, where inference speed and robustness to arbitrary player styles matter as much as top-1 accuracy, which this study aims to explore.

2.3 Commonsense Knowledge Bases and Semantic Grounding

Commonsense knowledge bases encode everyday facts about the world, including what objects are and what they can do, in machine-readable form, offering a principled source for grounding decisions in documented knowledge rather than arbitrary rules. Researchers broadly agree that ConceptNet is most widely used general-purpose commonsense knowledge graph, connecting

concepts through labeled relations such as *CapableOf* and *UsedFor* (Speer et al., 2017), and that such resources supply valuable object-centric knowledge for downstream systems (Hwang et al., 2021; Ilievski et al., 2021). Several efforts illustrate the breadth of this space: Sap et al. (2019) built ATOMIC to capture social, if-then commonsense; Zhang et al. (2020) automatically constructed the large-scale TransOMCS knowledge base from web text; Nguyen et al. (2021) developed methods to extract commonsense assertions with richer semantics, and Ilievski, Szekely and Zhang (2021) consolidated ConceptNet, WordNet, Wikidata, ATOMIC, and Visual Genome into a single graph to improve coverage.

In contrast, these same works document real limitations that temper the usefulness of any single resource. Hwang et al. (2021) found that roughly 90% of ConceptNet's tuples are taxonomic or lexical (such as *IsA* and *Synonym*),which leaves its genuinely commonsense content, including physical-capability knowledge, a relatively small portion of the graph. Crowdsourced knowledge bases also suffer coverage gaps and noise (Nguyen et al., 2021), and it is precisely this incompleteness that has made consolidation across sources has become a research goal (Ilievski et al., 2021). What stands out across this work is a striking asymmetry of application: commonsense knowledge bases are used extensively for language tasks such as question answering, yet very few studies enlist them to drive interactive object behavior like mapping a recognized object to a documented capability that becomes a usable in-game ability, and then validating that grounding against players' expectations. This use of commonsense relations as a source of gameplay affordances remains largely unexplored.

2.4 Drawing-Based & Property-Bearing Game Mechanics

Drawing-based interactions enable players to express their ideas visually rather than through buttons, menus, or typed commands. This makes drawing a great input method in games, wherein the player’s drawing can be interpreted by the system and used to affect gameplay. An example of such a research-based

game system is *Pixelor* , an academic prototype in which a player competes with an AI agent, *Pixelor,* to draw an object that an AI judge can recognize more quickly. The player who is recognized by the judge first wins. The system utilizes Sketch-a-Net 2.0, a deep convolutional neural network (CNN), as its recognition engine. To evaluate performance, Pixelor uses Early Recognition Efficacy (ERE), a metric that calculates the area under the curve of correct class probability as strokes are added. As a result, this enables the system to quantitatively measure the informativeness of a partial sketch by determining how quickly a concept becomes recognizable even when the judge has no access to future strokes (Bhunia et al., 2020). In addition to Pixelor, Quick, Draw by Google follows a similar concept in which the player is given a prompt and draws the object within 20 seconds, the trained recognition model attempts to guess what the drawing represents.The system records the player's drawing as a sequence of strokes and compares it against patterns learned from many previous drawings. When the neural network correctly identifies the drawing, the player receives feedback indicating that the drawing was recognized (Fernandez-Fernandez et al., 2019).

While Pixelor and Quick, Draw! demonstrates how drawings can be recognized and assessed based on how quickly they become identifiable, other games have explored player inputs through language rather than drawing. Scribblenauts, for instance, uses a text centered object-summoning system where the player writes or types the name of an object into an in-game notebook, and the system manifests that object into the game world. These summoned objects are then linked to predefined properties and behaviors, enabling them to interact with the environment and other game entities (John, 2009). This interaction can be grasped in relation to Natural Language Processing (NLP), a subfield of artificial intelligence dealing with enabling computers to process and interpret human language. NLP combines computational linguistics with machine learning and deep learning techniques to analyze written or spoken input and convert it into meaningful computational representations. Although Scribblenauts is not directly analyzed in most NLP studies, its mechanics align with the broader

idea of natural language interfaces in games because the player's typed input functions as a command that is mapped to a game-world object with functional properties (Picca et al., 2015).

The aforementioned systems such as Pixelor and Quick, Draw! demonstrates recognition-as-scoring wherein the system's recognition of a drawing is mainly used to evaluate the player's drawing, provide feedback, or determine success. However, the recognized drawing does not become a functional object that can actually affect the game world. Scribblenauts, on the other hand, demonstrates how player-generated input can produce property-bearing objects, but its interaction is based on typed language rather than drawing. OBRA bridges these approaches by using recognition-as-consequence: once the player's drawing is recognized, the result is mapped to corresponding abilities or affordances that directly influence gameplay. In this way, recognition is not only a scoring mechanism but a trigger for movement, interaction, and puzzle-solving based on the recognized object's real-world capabilities.

2.5 Vector Stroke Capture and Procedural Animation

In conventional 2D games, characters are commonly animated using pre-made individual images or sprite sheets containing different frames of an action. These frames are prepared before actual gameplay and are displayed in sequence to represent movements such as walking, jumping, or attacking. Godot, for an instance, supports character animation through collection of individual images or sprite sheets including required animation frames (Godot Engine contributors, n.d.). This approach is only suitable when the appearance and structure of a game character is already defined during development. However, it is not suitable for OBRA since the entities that the player morphs into does not exist until the player creates a drawing. Instead, OBRA employs Drawing-to-vector conversion and class-conditioned vector animation focused on how a player's drawing can become a usable and animated game vector.

Rather than replacing the player's drawing with a generated asset, OBRA preserves the original drawing as an editable vector representation — the ordered stroke points captured while the player draws — and animates that representation directly. Representing a freehand drawing as a sequence of vector strokes rather than as a fixed pixel image follows Ha and Eck (2017). The raster form of the drawing is used only by the recognition model to identify its class, while the vector form is retained for display and, crucially, for animation.

OBRA animates that vector form through simulated physics rather than through pre-scripted motion. When a drawing is accepted, its strokes are segmented and reconstructed into a physical body — a chain of rigid segments connected by joints — which is then actuated by lightweight controllers so that walking, flapping, hopping, or swimming emerges from the simulation itself. Object and primitive classes preserve the drawn strokes and receive class-specific collision and interaction behaviour instead. This places OBRA in the tradition of physics-based character animation, in which motion is produced by simulating and controlling a physical body rather than by replaying authored frames; Geijtenbeek and Pronost (2012), in their review of the field, note that rag-doll and actively controlled physical characters can yield responsive, situation-appropriate motion that kinematic playback cannot. The joint controllers follow the same principle as simple parameterized locomotion controllers such as SIMBICON, which generate a range of gaits and styles in real time from a small set of tunable parameters and remain robust to disturbances (Yin et al., 2007); OBRA adapts this idea to arbitrary drawn bodies by tuning each class's controller through a per-entity profile.

Related work confirms that user-created drawings can be animated while preserving their original appearance. Smith et al. (2023) animate hand-drawn human figures by fitting a fixed humanoid skeleton to the drawing. OBRA differs in two ways that matter for its fifty-class vocabulary: it does not assume a known body plan — the physical structure is derived from the player's actual strokes rather than fitted to a template, so one pipeline can handle a bird, a snake, or a

crab — and its motion is driven by physics simulation rather than by mapping the drawing onto a pre-rigged animation.

A more recent line of work animates vector sketches by optimising motion trajectories directly, treating the trajectory itself as a differentiable quantity to be solved for (Zhu et al., 2025). That approach shares this study's commitment to keeping the drawing in vector form rather than replacing it, but it differs in when the motion is decided: a trajectory is solved for in advance, whereas OBRA's motion is produced frame by frame in response to player input and to contact with a world the drawing has not yet met. The distinction is not one of quality but of what the motion has to answer to.

OBRA integrates this into gameplay by binding the simulated body to its recognized class, its ConceptNet-derived ability, and the player's controls, and by keeping the rendered ink locked to the simulated segments so the drawing and its physics never drift apart. The result is not merely an animated image but an interactive, physically simulated game entity built from the player's own drawing.

2.6 Serious Games and Cultural Representations

Games built for purposes beyond entertainment are commonly described as serious games, a category in which learning, training, or cultural transmission is a design goal rather than a side effect. Reviews of the field consistently find that such games succeed when the intended content is carried by the mechanics themselves rather than delivered alongside them as text or cutscene, since a player learns what the game makes them do. This framing matters to OBRA because its subject matter is not decorative: the entities a player draws, the abilities those entities receive, and the settings in which they act are drawn from a specific cultural context, and the study's claim is that the drawing mechanic is what carries that content.

Philippine settings remain under-represented in commercial game development, and where they appear they are frequently reduced to visual motif. OBRA locates its five levels in recognisable Philippine places and situations — a

bahay kubo among rice terraces, a town pista, the sea, and Mayon — and ties progression to a family narrative rather than to a conquest structure. The study does not claim to measure cultural learning; no instrument in the battery assesses it, and the planned cultural-content consultation described in Section 4.2.1 is a design safeguard rather than an outcome measure. What the study claims is narrower and testable: that a drawing-driven mechanic can be built on a documented commonsense knowledge base and set in a specific cultural context without the setting becoming ornamental.

CHAPTER 3
TECHNICAL BACKGROUND

This chapter presents the technical foundations on which the proposed system will be built. It introduces the field of freehand drawing recognition and explains why that field matters, defines the core concepts, technologies, and class vocabulary the proposed system will use, states the mathematical models and theoretical principles that underpin classification, abstention, animation, and evaluation, surveys the existing technologies and frameworks together with their limitations, identifies the technical problems that remain unsolved in the domain, and illustrates the structure of the system through diagrams.

3.1 Introduction to the Domain

3.1.1 Overview of the Subfield

The proposed system belongs to the field of freehand drawing recognition, a branch of computer vision that assigns labels to hand-drawn line art rather than to photographs. It draws on four subfields at once. The first is supervised deep learning, which supplies the recognition method: a convolutional neural network trained on labeled examples. The second is knowledge representation, and specifically commonsense knowledge graphs, which supplies the means by which a recognized label is turned into an ability the game can act on. The third is human-computer interaction, which supplies the drawing canvas as an input device and the interaction loop in which the recognition result is consumed. The fourth is real-time two-dimensional rendering and physics simulation, which supplies the response the player sees once a drawing has been recognized.

Freehand drawings are treated as a distinct recognition problem rather than as a direct reuse of photographic classification. A photograph is rich in evidence: it carries color, texture, lighting, shadow, and background, and a classifier can draw on all of them. A drawing carries almost none of this. It is a small number of thin lines on an otherwise

empty page, and every person draws the same object at a different size, from a different angle, with a different level of detail and a different stroke order. Networks designed for natural images therefore do not transfer to this domain unchanged (Yu et al., 2015), and the evaluation practices appropriate to it differ as well.

The proposed system is a single application on a single computer. The game runtime and the recognition component are two co-located processes on the same host that exchange data over the host's loopback interface, which is not a network; no remote server, external connection, or second device is required at any point during play.

3.1.2 Importance of the Domain

The domain matters for two reasons. First, drawing is one of the most accessible forms of human expression, and a system that can read a drawing converts a low-barrier input into a control signal. This has direct application in education, in accessibility, and in games, where it removes the need for the player to learn a symbolic command vocabulary before acting. Second, drawings are genuinely hard to recognize, and progress on them is not obtained for free from progress on photographic classification (Eitz et al., 2012; Yu et al., 2015).

A drawing-driven game adds a requirement that recognition research alone does not impose. A classifier used for research may report its errors in a table after the fact; a classifier used as a game control must decide, immediately and on its own, whether it has understood the player at all. The proposed system therefore depends not only on a model that classifies well, but on a runtime that knows when the model is unsure, on a documented source of meaning that turns a label into an ability, and on an architecture that can run a machine-learning model alongside a game engine that has no facility for one, on a single machine and without leaving it.

3.2 Core Concepts and Definitions

3.2.1 Drawing Representation

A digital drawing can be stored in two forms. A raster representation stores the drawing as a grid of pixels and can be turned into an array of fixed width, height, and channel count, which is the form a neural network expects. A vector representation stores the drawing as a list of strokes, where each stroke is an ordered list of points. The vector form records how the drawing was made and stays editable, so it can be moved, scaled, and reshaped after it is drawn (Ha & Eck, 2017).

In plain terms, the raster form is a photocopy of the drawing and the vector form is a record of the pen movements that produced it. The photocopy is what allows the drawing to be identified, because that is the form the recognizer expects; the pen movements are what allow it to be redrawn in a new position, which is what animating it means. The two forms therefore serve different purposes, and the proposed system captures both at the same moment, sending the raster image to be recognized while retaining the strokes to be animated.

3.2.2 The Classification Task

Let x be a preprocessed drawing image and let Y be a fixed set of K category labels. A classifier is a function f(x) that returns one score for every label in Y. The task is to learn this function from examples whose labels are already known, so that the label with the highest score is the correct one. The classifier does not return only a label; it returns a full set of scores, and the proposed system reads three quantities from them: the top label, its probability, and the gap between that probability and the second highest. The formal statement of the task and of the quantities derived from it is given in Section 3.3.

3.2.3 Confidence, Margin, and Abstention

Confidence is the probability the model assigns to its top prediction. Margin is the difference between the top probability and the runner-up probability. The two describe different kinds of doubt. A model may be uncertain because the drawing resembles nothing it knows, in which case confidence is low across the board; or it may be uncertain because the drawing resembles two classes equally well, in which case confidence may look acceptable while the margin is small. A circle drawn slightly egg-shaped may divide its probability between a circle and a rounded creature, and neither number alone would catch it.

Abstention is the decision to make no prediction at all. A system that abstains when either quantity falls below a threshold is choosing a wrong answer's cost over an unhelpful answer's cost, which is the right trade in an interactive setting: a game that transforms the player into the wrong creature has misread them, whereas a game that asks them to draw more clearly has merely paused. The proposed system will therefore treat recognition as a decision with three outcomes: accept, abstain, or reject, rather than as a forced choice among fifty labels.

Confidence-based decision making has become a common strategy in machine learning systems where incorrect predictions may incur a greater cost than delayed decisions. Selective classification allows a model to abstain from making predictions when confidence is insufficient, improving the reliability of deployed systems by rejecting uncertain classifications rather than forcing potentially incorrect outputs (Geifman & El-Yaniv, 2017).

3.2.4 Preprocessing and Train-Serve Parity

Preprocessing is the step that conditions a drawing before the model looks at it. A player can draw anywhere on the canvas, at any size,

with any line thickness, but the model was trained on images of one fixed size and one fixed style. Preprocessing bridges that difference. It converts the drawing to grayscale, ensures that ink and background are the correct way around, crops away the empty space so that only the drawing remains, centers and pads it so that its aspect ratio is not distorted, resizes it to the dimensions the model expects, and finally rescales the pixel values into the numeric range the model was trained on.

Image preprocessing is a standard stage in computer-vision pipelines because neural networks are sensitive to differences in scale, alignment, and input representation. Consistent preprocessing minimizes the distribution mismatch between training and inference data, improving recognition accuracy and model stability (LeCun et al., 2015; Goodfellow et al., 2016).

Train-serve skew is the name given to the failure that occurs when the code that prepares a training image and the code that prepares a live input do not agree. It is a recognized hazard of deployed machine-learning systems precisely because it produces no error: the model receives an input it was never trained on, answers confidently, and answers wrongly, while every offline metric remains excellent (Sculley et al., 2015; Breck et al., 2017). A drawing is mostly empty space held together by a few thin lines, so unlike a photograph it has no color or texture to fall back on; a crop that cuts through a stroke, or a resize that thins a line until it disappears, removes the only evidence the model has. Train-serve parity is therefore obtained by construction rather than by testing: the proposed system defines preprocessing once, in a single shared routine, and calls that same routine from the training pipeline, from the offline evaluation, and from the live recognizer.

3.2.5 Generalization, Overfitting, and Held-Out Evaluation

A model is said to generalize when it performs well on drawings it has never seen. The opposite condition is overfitting, in which a model has effectively memorized its training examples and performs well on them while performing poorly on anything new. Because a memorizing model and a genuinely competent model are indistinguishable on the training data itself, performance can only be judged on data withheld from training (Goodfellow et al., 2016).

Three practices follow from this and are used in the proposed study. The first is data splitting: the available drawings are partitioned into a training set used to fit the model, a validation set used to monitor it during training, and a test set touched only once the model is final. The second is class balancing: because the source corpus contains far more drawings of some categories than of others, each class is capped at the same number of examples before splitting, so that the model is not simply rewarded for favouring populous classes. The third is data augmentation: mild rotations, shifts, and rescalings are applied to training images on the fly, so that the model is exposed to variation it would otherwise have to encounter by chance. Augmentation is applied to the training set alone; validation and test drawings are always seen unperturbed, since perturbing them would make the evaluation easier rather than more honest.

A held-out split of the training corpus is nonetheless a limited test, because its drawings were made by the same population under the same conditions. The proposed study therefore also evaluates against an independent corpus collected by different people under a different protocol, which is the closer analogue of a player at a canvas. The two evaluations answer different questions and are reported separately rather than merged.

3.2.6 Knowledge Graphs and Ability Attribution

A knowledge graph is a structured representation of facts in which concepts are nodes and the relationships between them are labeled, directed edges. ConceptNet is a commonsense knowledge graph of this kind, developed from the Open Mind Common Sense project and maintained as an open resource; it represents everyday semantic relationships between natural-language concepts through relations such as CapableOf, UsedFor, HasProperty, HasA, and IsA, so that applications can reason over ordinary knowledge rather than encoding it by hand (Speer et al., 2017).

Each fact in ConceptNet is stored as an assertion, a triple naming a start concept, a relation, and an end concept, and each assertion carries a weight. In plain terms, an assertion is a sentence of the form frog is capable of jumping, and its weight records how many independent contributors asserted it. The weight is therefore a measure of how widely a fact is agreed upon, and not a measure of how useful the corresponding ability would be in a game. This distinction governs how the resource is used in the proposed system: eligibility for an ability is decided first by whether a level actually requires that capability of the class, and weight is consulted only to break ties among the surviving candidates. Selecting the highest-weighted edge without a requirement filter would ground abilities that are well attested and unplayable.

Ability attribution is the name this study gives to the step that turns a recognized label into a capability the game can act on. It is performed offline. The researchers consult the graph during development, select the supporting assertions, record the relation and weight behind each selection, and store the result in a static lookup table. No query is issued to ConceptNet while the game is running, so the graph is a design-time source of documented meaning rather than a runtime dependency, and

the game's behavior does not vary with the availability of an external service.

Inheritance requires a guard, and the clearest illustration is the penguin. ConceptNet records that a penguin is a bird and that a bird is capable of flight, so an unguarded traversal of the IsA relation would grant a penguin the ability to fly. A traversal is therefore permitted only where the child class is not a documented exception to the inherited capability, and every traversal is recorded in the attribution table together with the parent concept it passed through. Where no usable assertion exists at any depth, the ability is hand-authored and explicitly marked as such, so that the provenance of every ability, whether graph-derived or authored, is documented rather than assumed.

3.2.7 Class Vocabulary and Ability

The proposed system will recognize a fixed vocabulary of fifty classes, grouped as shown in Table 1. Each recognized class is mapped to a corresponding ability or affordance in the game by the attribution procedure described in Section 3.2.6.

The vocabulary is closed in both directions, and this is the property that makes it load-bearing rather than incidental. An obstacle can only be authored against a class the recognizer is able to name, so the roster is at once a specification of what the model must learn and a specification of what a level designer is permitted to require. A puzzle whose intended solution lies outside the roster is not a difficult puzzle; it is an unsolvable one, and nothing in the system will say so. Section 3.5 treats this as a distinct failure mode.

The vocabulary is also the unit of progression. Because the proposed game allows the player to leave a level and return to it later, certain obstacles are gated behind a class the player has not yet been

taught to draw, and the roster therefore carries, for each class, the point at which that class becomes available. A gated obstacle is not a failure state: the system reports that the required concept is not yet known, and the player continues elsewhere.

Table 1

Class Vocabulary, Spawn Behavior, and the Relation Backing Each Group

Group	Classes	Spawns as	Ability relation
Creatures (20)	bird, bat, butterfly, spider, monkey, frog, fish, shark, octopus, crab, horse, elephant, snake, sea turtle, snail, ant, bee, pig, penguin, scorpion	Playable actor	per-class relations are given in the attribution table; the relation set is not uniform within a group. (See Appendix C)
Primitives (3)	circle, square, triangle	Drawable object	
Objects (27)	axe, sword, cannon, boomerang, flashlight, campfire, cloud, sun, fan, ladder, stairs, parachute, hot air balloon, bridge, sailboat, submarine, key, door, rake, scissors, clock, anvil, bucket, umbrella, tree, mushroom, wheel	Drawable object	

Every one of the fifty classes is trained from a Quick, Draw! label matching the name the game uses; no class is trained under a proxy label. The mapping from each class to its source label is nonetheless recorded in the entity manifest rather than assumed, so that this correspondence remains inspectable rather than presumed.

The relation column of Table 1 is not uniform, and this is a deliberate acknowledgement rather than an oversight. The CapableOf relation describes

what a thing can do, which fits creatures: a frog is capable of jumping. Objects are more often described by what they are for; a ladder is not capable of climbing, it is used for climbing, which is the UsedFor relation. The geometric primitives carry neither an agentive nor a functional edge of the usual kind, since a shape is not the sort of thing commonsense describes as acting, and they are instead grounded through structural relations. In total the study consults six affordance-bearing relations, CapableOf, UsedFor, HasProperty, HasA, ReceivesAction, and MannerOf, together with a single guarded IsA traversal where a class carries no usable edge of its own. Forty-eight of the fifty classes are grounded in this way; two, penguin and mushroom, are hand-authored. Any claim that a single ConceptNet relation supplies the abilities for all fifty classes would be false, and this chapter does not make it.

3.2.8 The Label-Order Contract

A trained classifier does not return the word frog. It returns a vector of numbers, and the position of the largest number is the answer. Meaning is supplied entirely by an external list that says which position corresponds to which label. If the model's output order and the game's roster disagree by even one position, every prediction is silently mistranslated: the model is right, the list is wrong, and no error is ever raised. The proposed system therefore treats the trained model, the label list, and the entity roster as one versioned artifact whose consistency is checked before anything runs, rather than as three files that happen to travel together.

Consistent label indexing is a recognized deployment requirement because the output of a classification model contains only numerical scores whose meaning depends entirely on the external label mapping used during inference. Errors in this mapping may silently invalidate predictions even when the model itself performs correctly (Sculley et al., 2015; Breck et al., 2017).

3.2.9 Local Client-Server Integration

A game engine and a machine-learning runtime are built for different purposes and, in this case, are written in different languages. The engine cannot execute the model directly, and embedding a deep-learning framework inside the engine would carry a runtime that is far heavier than inference requires. The proposed system therefore adopts a client-server arrangement in which the two concerns remain two-co-located processes of one deployed desktop application that agree on a message format, with the important qualification that both programs run on the same computer.

The client-server pattern describes any arrangement in which one program requests a service and another provides it. Ordinarily the two are on separate machines and communicate across a network. Here they are not. They communicate over the loopback interface, a facility every operating system provides by which a program may address the machine it is already running on. Data sent over loopback never reaches a network adaptor and never leaves the host. It is therefore accurate to describe the two components as client and server, and inaccurate to describe the exchange between them as network traffic. The exchange nonetheless follows the conventions of the architectural style Fielding (2000) set out for the web: a uniform interface addressed by resource, a request that carries everything needed to serve it, and a component that retains no state between requests. Those properties are adopted here for the discipline they impose rather than for the distribution they were designed to support — a stateless recognizer cannot accumulate hidden state that changes what a later drawing is recognized as, which is what makes one submission reproducible from its inputs alone.

The message itself uses two ordinary web conventions. The request and response bodies are formatted as JSON, a lightweight text format that is readable both by people and by machines and is widely

used for configuration and data interchange (Bray, 2017). Because JSON is a text format and an image is binary data, the drawing is Base64-encoded, a standard scheme for representing binary content as text, before being placed in the request body. The response carries the predicted label together with the full score vector, from which the runtime derives the confidence and margin of Section 3.2.3.

Two consequences follow and are stated here because Section 3.5 depends on them. First, the encoding, transmission, and decoding of the message are real costs that the model's own execution time does not include, which is why response time must be measured over the whole exchange. Second, because the request is dispatched without the game waiting for its reply, the recognition never occupies the frame in which the player is drawing.

3.2.10 Offline Training and Runtime Inference

Training and inference are separate activities with different requirements, and the proposed system separates them completely. Training is the process of fitting the model to labeled examples; it is computationally expensive, is performed once, and is done on the researchers' machine rather than on the player's. Inference is the process of applying the already-trained model to a new input; it must be fast, must happen many times during play, and requires none of the machinery that training requires.

The bridge between the two is a model interchange format. The trained network is exported from the training framework into a portable file that records the network's structure and its learned parameters, and this file is then executed at runtime by a lightweight inference engine that does not require the training framework to be installed. In plain terms, the training framework is the workshop and the exported file is the finished tool: the workshop does not need to be shipped with the tool. This

separation is what allows a model developed in one ecosystem to be served inside an application built in another, and it is the reason the game's runtime footprint is far smaller than its development footprint. It also means the model is fixed at export: no learning of any kind occurs during play, and the system's behavior does not change from one session to the next.

3.2.11 Procedural Stroke Animation

Procedural animation is motion produced by code acting on geometry while the program runs, rather than by playing back frames prepared in advance. In the proposed system the geometry being animated is the set of points that make up each stroke the player drew, and motion is produced by changing those points over time in response to movement and input. This matters because the artwork does not exist until the player draws it, so nothing can be prepared beforehand, and it distinguishes the approach from ordinary skeletal animation, in which bones are attached to artwork whose structure is already known.

The system produces this motion through physics simulation rather than by scripting stroke positions directly. The captured strokes are segmented and reconstructed into a small physical body: each segment becomes a rigid body with its own mass and collision shape, and adjacent segments are linked by joints, forming a bounded articulated structure, an active ragdoll, assembled entirely from the player's ink rather than from a predefined skeleton. Motion is then generated by driving those joints toward target poses with proportional-derivative control, the same servo-style control used in simple physics-based locomotion controllers (Yin et al., 2007), so that a gait such as walking, hopping, or flapping emerges from the interaction of these joint torques with gravity, contact, and the game world. This situates the approach within physics-based character animation, where actively controlled simulated bodies, including rag-doll characters, produce responsive motion without authored frames

(Geijtenbeek & Pronost, 2012). The control law itself is stated in Section 3.3.9.

Two properties follow from deriving the body from the drawing. Because the structure is unknown in advance, the number and arrangement of segments are inferred from the strokes when the body is built, and a fixed budget on bodies and joints keeps the simulation stable; where a drawing cannot be articulated reliably, the system degrades to a single intact physics body rather than simulating an unstable rig, so that the drawing remains playable and its ink stays aligned with the body. And because the visible ink is kept attached to the simulated segments, the drawing and its physics remain aligned throughout play. The rigid bodies, joints, and contact handling are provided by the game engine's built-in two-dimensional physics (Godot Engine contributors, n.d.).

3.2.12 Progression State and the Player Profile

Two kinds of state exist in the proposed system, and the distinction between them is a design commitment rather than an implementation detail.

Session state is everything that describes the current attempt at a level: the ink remaining, the tool currency equipped, the objects placed in the world, and the checkpoint most recently reached.

Profile state is everything that must survive the closing of the application. It comprises the set of classes the player has been taught and may therefore draw, the levels unlocked and completed, the narrative route chosen at each dialogue node, the collectibles recovered, and the accumulated behavioral counts defined in Section 3.3.8. Because the proposed game invites the player to leave a level and return to it after learning something elsewhere, and because the ending is selected from

the whole history of play rather than from the final level alone, profile state cannot be reconstructed from the current scene. It must be recorded.

Persistent application state enables interactive software to preserve user progress across multiple sessions while maintaining consistency despite interruptions or application updates. Separating persistent state from transient runtime state is a common architectural practice that improves software reliability and maintainability (Bass et al., 2022; Amershi et al., 2019).

Recording state on disk introduces the first component of the system whose correctness depends on data the application did not itself produce in the current run. A profile file may be absent on first launch, truncated by a power failure during a write, or written by an earlier version of the game whose schema has since changed. The proposed system will therefore write the profile atomically, by writing a temporary file and renaming it into place only once the write has completed, and will treat an unreadable or schema-incompatible profile as a new profile rather than as a fatal error. A player who loses a save is inconvenienced; a player who cannot launch the game is stopped.

3.2.13 Weighted Selection of Dialogue

The proposed game allows a level to be entered more than once, which creates a problem that has nothing to do with recognition: a companion character who delivers the same line on every visit makes the second visit feel like an error. The requirement is that repeated play produces different commentary without exhausting the authored material and without the selection becoming unpredictable to the researchers who must describe what a participant heard.

The mechanism adopted is a priority queue over the authored lines available in a given context. A priority queue is a data structure from which

elements are retrieved according to an assigned priority rather than the order in which they were inserted, and priority-based scheduling is widely used in computing systems to ensure that more important items are processed before lower-priority ones (Cormen et al., 2022; da Silva et al., 2020). The analysis of priority-driven scheduling under timing constraints originates with Liu and Layland (1973), whose treatment of periodic tasks under fixed and dynamic priority remains the reference account. The mechanism used here is a descendant of that idea rather than an instance of it: no line of dialogue carries a deadline, and a delivery that is late is merely late, so the queue borrows the selection discipline without inheriting the scheduling obligations that accompany it in a real-time system. Each line carries a weight. A line the player has not heard carries the maximum weight and is therefore selected first. When a line is delivered its weight drops to zero, and it recovers gradually as other lines are delivered, so that it becomes eligible again only after the unheard material has been exhausted. Selection takes the highest-weighted line, with ties broken at random from among the tied set.

Three properties of this arrangement matter to the study. First, it is a scheduling mechanism and not a learning one: no model is involved and no training data is required, so it adds nothing to the recognition problem. Second, insertion and extraction are logarithmic in the number of lines held, so the cost is negligible beside the frame budget described in Section 3.5 and the mechanism does not compete with recognition for time. Third, because the weights are stored in the profile and the recovery rule is deterministic, the sequence of lines a player hears is reproducible from their profile, which is what allows a play session to be described after the fact rather than merely observed. The mechanism is bounded by the material it is given: with few authored lines, demotion cycles back to repetition quickly, and the perceived variety is a function of the writing rather than of the queue.

3.3 Theoretical Foundations

3.3.1 Supervised Learning and the Classification Task

Drawing recognition is framed here as a supervised classification problem. In supervised learning a model is trained on examples whose labels are already known. Each example pairs an input, here a preprocessed drawing image, with a target label drawn from the fixed vocabulary. During training the model predicts a label, compares that prediction against the known target, and adjusts its parameters so as to reduce future error. Nobody ever writes down what a frog looks like; the model is shown many thousands of drawings labeled frog, many thousands labeled key, and so on, and it gradually adjusts itself until its predictions agree with the labels. The process is closer to how a person learns to read handwriting than to how a calculator works: the rule separating a four from a nine cannot easily be written down, but sufficient exposure to both makes the distinction reliable.

Formally, the study is given a set of N labeled examples and seeks the parameter values that minimize the average loss over that set, an objective known as empirical risk minimization (Goodfellow et al., 2016). The parameters are the weights and biases of the network; the loss is defined in Equation (4).

3.3.2 What a Convolutional Neural Network Does

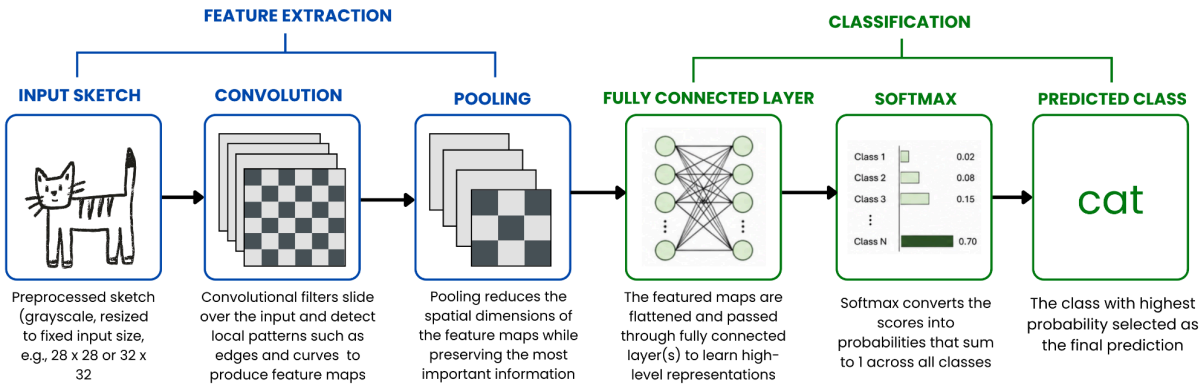
A convolutional neural network, or CNN, is a network that learns to recognize images from examples rather than from rules written by hand. It is built from layers. The first layer inspects small patches of the image and responds to very simple things, an edge in one place, a curve in another, a corner. The next layer inspects the output of the first and responds to combinations of those simple responses, a loop, a fork, a long line. Layer by layer the patterns detected become larger and more meaningful, until

the final layers are responding to whole shapes. This hierarchical feature learning is what makes CNNs effective for image classification (LeCun et al., 2015), and it suits sketches particularly well, because many recognizable drawings are built from simple line and curve arrangements.

Three ideas do most of the work. The first is the filter, a small pattern detector that is slid across the whole image so that the same feature is recognized wherever it appears; because one filter is reused at every position rather than a separate detector being learned for each, the network has far fewer parameters to learn than it otherwise would. The second is pooling, which shrinks a feature map by keeping only its strongest responses in each small region, reducing computation and conferring tolerance to small shifts in where a stroke happens to fall. The third is depth, the stacking of these operations so that later layers build on the abstractions formed by earlier ones. Figure 1 illustrates the resulting forward pass.

Figure 1

General Convolutional Neural Network (CNN) Classification Pipeline



Note. Figure created by the researchers based on LeCun et al. (2015).

As shown in Figure 1, the input sketch first passes through convolutional layers that apply learnable filters to detect local visual patterns. The resulting feature maps are refined through pooling operations, which reduce their spatial dimensions while preserving the most informative responses. The extracted features are then flattened and passed to fully connected layers, which combine the learned representations and produce one raw score for each class in the vocabulary.

Those raw scores, called logits, are unbounded and cannot be read as probabilities. The softmax function converts them into a probability distribution over the vocabulary, as shown in Equation (1): it exponentiates each score and divides by the sum of all of them, so that every value is positive and the values together sum to one.

$$p_k = \frac{exp(z_k)}{\sum_{j=1}^{K} exp(z_j)}, \; k = 1, \cdots, K \tag{1}$$

where:

p_k = the predicted probability of class k

z_k = the raw score (logit) of class k produced by the final layer

K = the number of classes, equal to 50 in this study

the values p_k sum to 1 by construction

The predicted label is then simply the class holding the largest of these probabilities. This conversion is what makes the quantities of Section 3.3.5 meaningful, since a difference between two raw scores has no fixed interpretation while a difference between two probabilities does.

Freehand drawing recognition nevertheless poses difficulties that this architecture does not remove. Sketches are abstract, incomplete, and highly variable across users; different users draw the same category with different stroke order, viewpoint, size, detail level, and symbolic style; and unlike natural photographs, sketches carry no material texture, color gradient, shadow, or background object. Eitz, Hays, and Alexa (2012) showed that human sketches vary widely across object categories, while Yu et al. (2015) demonstrated that models designed specifically for sketches can perform strongly on drawing-recognition benchmarks.

3.3.3 Training the Network

Training minimizes the categorical cross-entropy between the distribution the model predicts and the known answer. Because the correct answer places all of its weight on a single class, this measure reduces to the negative logarithm of the probability the model assigned to that class. The behavior of that quantity is what makes it appropriate: a confident correct prediction is penalized lightly, an unconfident correct prediction more heavily, and a confident incorrect prediction very heavily indeed, which is precisely the ordering a system that will later act on its own confidence requires (Goodfellow et al., 2016).

Three further mechanisms are used during training, each a standard choice rather than a contribution of this study. Parameter updates are computed by the Adam optimizer, which maintains running estimates of the gradient's recent magnitude and direction and adapts the step size for each parameter accordingly, so that a single global learning rate need not suit every weight. Dropout is applied as a regularizer: at each training step a randomly chosen subset of activations is set to zero, which prevents the network from depending too heavily on any single feature and reduces overfitting, and no units are dropped at inference. Geometric augmentation exposes the model to small rotations, shifts, and

rescalings of its training images, so that tolerance to how differently people draw is learned rather than assumed. The specific optimizer settings, dropout rate, and augmentation parameters are reported in Chapter 4.

3.3.4 Preprocessing as a Shared Transformation

Preprocessing, described in Section 3.2.4, is a deterministic transformation applied identically at training time and at play time. Its final step deserves particular emphasis, because it is the step at which a mismatch would be entirely invisible. Pixel intensities are rescaled into the numeric range on which the network was trained, first by dividing by the maximum intensity value so that every pixel lies between zero and one, and then by subtracting the mean of the training distribution and dividing by its standard deviation. A model trained on normalized input and served unnormalized input will answer confidently and wrongly, and no component of the system would report it. This is the concrete form of the train-serve skew described in Section 3.2.4, and the reason the transformation is written once and called from both the training pipeline and the live recognizer.

3.3.5 The Decision Rule

The softmax output supports two distinct measures of doubt, introduced in Section 3.2.3 and stated formally here because the thresholds placed on them are design decisions this study must defend rather than standard results it applies. Confidence is the largest probability in the distribution.

$$c(x) \;=\; max_{k}\, p_{k} = p_{1} \tag{2}$$

where:

c(x) = the confidence of the prediction for input x

p_1 = the largest predicted probability

p_k = the predicted probability of class k

Margin is the separation between the two leading probabilities. It detects the case in which the model is torn between two candidates while remaining nominally confident, which confidence alone cannot.

$$m(x) = p_1 - p_2 \quad (3)$$

where:

m(x) = the margin of the prediction for input x

p_1 = the largest predicted probability

p_2 = the second-largest predicted probability

The system acts on a prediction only when both quantities clear their thresholds; otherwise it abstains and returns the drawing to the player. This is a selective classification rule, in which the model is permitted to decline rather than forced to choose (Geifman & El-Yaniv, 2017). Both thresholds are held as configuration values so that they can be re-tuned against the confusion matrix without a code change, and a declined drawing costs the player no ink and no progress.

$D(x) = \hat{y}$ if $c(x) \geq T_c$ and $m(x) \geq T_m$; $D(x) =$ abstain otherwise (4)

where:

D(x) = the decision returned by the runtime for input x

$\hat{y}$ = the class with the highest predicted probability from Equation 1

τ_c = the confidence threshold, set to 0.60 in this study

τ_m = the margin threshold, set to 0.15 in this study

3.3.6 Threshold Selection

Equation 4 leaves two quantities unspecified: the confidence threshold and the margin threshold at which the system stops abstaining and acts. Neither has a standard value, because both encode a judgement about which failure is worse. A low threshold spawns entities the player did not ask for; a high one returns drawings the player believes are adequate. The study sets the confidence threshold at 0.60 and the margin threshold at 0.15, favouring the cheaper failure, and treats both as configuration values tuned against the confusion matrix rather than as fixed constants.

The direction of that trade-off follows the interaction literature rather than the classification literature. Users tolerate an imperfectly accurate system when its interface sets correct expectations about its errors, and frustration in recognition-based input is driven by the time taken to recover from a false positive rather than by how often a redraw is requested. A declined drawing therefore costs the player nothing but a second attempt, while an accepted wrong answer costs an unwanted spawn and the ink to undo it. The thresholds are set accordingly, and the evaluation reports the redraw rate so that the cost of this choice is visible rather than assumed.

3.3.7 Evaluation Measures

Because drawings can be ambiguous, evaluation does not rely on overall accuracy alone. Accuracy is the proportion of evaluation examples the model labels correctly, and it is reported, but it can conceal weak performance on individual classes. In this system such weakness would

appear not as a low score but as an obstacle the player cannot pass, which is why the study reports the per-class measures below alongside it and never in place of them (Sokolova & Lapalme, 2009).

$$Accuracy = \frac{1}{M} \sum_{i=1}^{M} \mathbb{1}[\hat{y}^{(i)} = y^{(i)}] \quad (5)$$

where:

M = the number of evaluation examples

$\hat{y}^{(i)}$ = the predicted class of the i-th example

$y^{(i)}$ = the true class of the i-th example

$\mathbb{1}[\cdot]$ = the indicator function, equal to 1 when its condition holds and 0 otherwise

Precision measures how often a prediction of a given class turns out to be correct, and therefore how much a class over-claims. It is the quantity that exposes the geometric primitives, into which poorly drawn members of other classes tend to collapse.

$$Precision_k = \frac{TP_k}{(TP_k + FP_k)} \quad (6)$$

where:

$Precision_k$ = the precision of class k

TP_k = true positives, examples of class k predicted as class k

FP_k = false positives, examples of another class predicted as class k

Recall measures how often examples of a given class are actually found, and it is the measure that matters most to this study: a class whose recall is low is a class whose associated ability the player cannot reliably obtain, whatever the aggregate figure says.

$$Recall_k = \frac{TP_k}{(TP_k + FN_k)} \tag{7}$$

where:

$Recall_k$ = the recall of class k

TP_k = true positives, examples of class k predicted as class k

FN_k = false negatives, examples of class k predicted as some other class

The two are combined into a single figure by the F1-score, the harmonic mean of precision and recall, which unlike an ordinary average remains low when either component is low. Because the evaluation sets are not equally populated across the vocabulary, the study also reports macro-averaged recall, the unweighted mean of the fifty per-class recall values, which weights every class equally regardless of how many examples it contributes. No aggregate figure is reported without the per-class table beside it, and the recall of the three geometric primitives is always shown individually rather than absorbed into a fifty-way mean.

$$F1_k = 2 \cdot \frac{(Precision_k \cdot Recall_k)}{(Precision_k + Recall_k)} \tag{8}$$

where:

$F1_k$ = the harmonic mean of precision and recall for class k

The confusion matrix records not merely how often the model is wrong but which classes it confuses with which. It is the instrument by which the similarity clusters hypothesized in Section 3.5 are tested and by which the thresholds of Equation 4 are tuned.

$$C_{k,l} = \sum_{i=1}^{M} \mathbb{1}[y^{(i)} = k] \cdot \mathbb{1}[\hat{y}^{(i)} = l] \tag{9}$$

where:

$C_{k,l}$ = the number of examples of true class k predicted as class l

M = the number of evaluation examples

$y^{(i)}$, $\hat{y}^{(i)}$ = the true and predicted class of the *i*-th example

$\mathbb{1}[\cdot]$ = the indicator function, equal to 1 when its condition holds and 0 otherwise

3.3.8 Derived Gameplay Measures

Three further quantities are defined here because Chapter 4 measures them and, in two cases, because the game itself consumes them in selecting an ending. Class diversity counts the distinct classes the player has drawn and has accepted at least once. It rises monotonically and never falls.

$$V = \left|\{k \in Y : n_k \geq 1\}\right|, \; 0 \leq V \leq 50 \tag{10}$$

where:

V = the number of distinct classes drawn and accepted at least once

Y = the class vocabulary

n_k = the number of accepted submissions recognized as class k

Redraw rate is the proportion of submitted drawings that the recognizer declined under the rule of Equation 4, that is, the number of submissions failing the confidence or margin threshold divided by the total number of submissions

$$R = \frac{N_{declined}}{N_{submitted}} \tag{11}$$

where:

R = the redraw rate

$N_{declined}$ = the number of submissions that failed the confidence or margin threshold

$N_{submitted}$ = the total number of submissions

Finally, because the model shares a processor with a game that has roughly sixteen milliseconds in which to draw each frame, the latency the player experiences is measured over the whole sequence from capture to spawn and not over model execution alone. Reporting inference time by itself would understate the wait by the cost of encoding, transfer, and decoding described in Section 3.2.9.

$$T_{e2e} = T_{pre} + T_{transfer} + T_{inf} + T_{post} \tag{12}$$

where:

T_{e2e} = the total response time experienced by the player

T_{pre} = time to capture and preprocess the drawing

$T_{transfer}$ = time to encode, transmit over loopback, and decode the request and response

T_{inf} = time for the model to execute

T_{post} = time to look up the ability and construct the animated body

3.3.9 Control of the Animated Body

The articulated body assembled from the player's strokes, described in Section 3.2.11, is driven by proportional-derivative control. Each joint is given a target angle, and a torque is applied in proportion to the error between that target and the joint's current angle, damped in proportion to its angular velocity so that the joint settles rather than oscillates. In plain terms, the proportional term is how hard the joint is pulled toward where it should be, and the derivative term is the brake that stops it overshooting. A gait emerges from the interaction of these torques with gravity and contact rather than from authored keyframes (Yin et al., 2007).

$$T_j = k_p(\theta^*_j - \theta_j) - k_d\dot{\theta}_j \quad (13)$$

where:

τ_j = the torque applied at joint j

θ^*_j = the target angle of joint j

θ_j = the current angle of joint j

$\dot{\theta}_j$ = the current angular velocity of joint j

k_p = the proportional gain, governing how strongly the joint is driven toward its target

k_d = the derivative gain, governing how strongly motion is damped

3.4 Existing Technologies and Frameworks

The proposed drawing-recognition pipeline consists of several software tools, datasets, and supporting technologies that are collectively used to train, deploy, and integrate the recognition model to the game. Each component serves a specific purpose from developing the machine learning model to game implementation and data storage. Before discussing their individual roles within OBRA, this section provides an overview to introduce each technology providing readers with the necessary technical background. *Table 2* then summarizes how each component is used in the proposed system together with its existing limitations.

3.4.1 The Game Runtime Layer

Godot Engine. Godot is a free and open-source game engine used for developing two- and three-dimensional games. It supplies the drawing canvas, input handling, real-time two-dimensional rendering, the scene graph, and the built-in physics that provides the rigid bodies, joints, and contact handling on which the animation of Section 3.2.11 depends (Godot Engine contributors, n.d.). Its principal limitation for this study is that it contains no machine-learning runtime, which is the reason recognition is performed by a separate co-located component rather than inside the engine.

GDScript. GDScript is Godot's integrated scripting language. All hand-written game logic is implemented in it: stroke capture, the construction of the rig from the player's ink, movement-driven animation, the ink economy, and the checkpoint system. It is statically typed in the manner used here, so that a mismatch between what a function returns and what its caller expects is caught before it can appear as a gameplay defect.

Aseprite. Aseprite is a specialized image editor and animation tool for pixel art, used to author the game's environment art, user-interface elements, and authored sprites. It is one of two tools in the project that are not fully open-source, being source-available under a proprietary licence that permits compilation from source at no cost; the other is the editor described in Section 3.4.4. It produces none of the player-facing creature artwork, which is drawn by the player at runtime.

3.4.2 The Recognition Service Layer

Python. Python is the host language of the local recognition service and of the offline training pipeline. It is used rather than GDScript for this half of the system because the machine-learning ecosystem the study depends on exists in Python and has no equivalent inside the game engine.

FastAPI, Uvicorn, and Pydantic. FastAPI is a modern Python web framework for building application programming interfaces, which derives validated endpoints directly from ordinary type annotations (Ramírez, n.d.). Uvicorn is the server that runs the resulting application, and Pydantic performs the validation of request and response schemas. Together they expose the model as a typed endpoint bound to the loopback interface and reachable only from the same machine. Their cost is that every prediction now carries encoding, hand-off, and parsing time in addition to model execution, which is why latency is measured end to end, as stated in Equation 12.

ONNX and ONNX Runtime. The Open Neural Network Exchange is an open standard format for representing machine-learning models so that they can be transferred between frameworks and executed on multiple platforms, while ONNX Runtime is the inference engine that executes an exported model without requiring the original training framework to be installed (PyTorch Contributors, 2023). This is the

mechanism described conceptually in Section 3.2.10. Its limitations are that not every operation can be exported, and that the exported model is fixed and less readily inspected than the definition it came from.

Pillow and NumPy. Pillow decodes the submitted image and performs the grayscale conversion, cropping, and resizing of the preprocessing stage, while NumPy provides the array handling for preprocessing tensors, dataset loading, and metric computation. These two libraries are what the shared preprocessing routine of Section 3.2.4 is written against, which is what allows one definition to serve both training and play.

3.4.3 The Offline Training Layer

PyTorch and torchvision. PyTorch is an open-source deep-learning framework that provides tools for building, training, and evaluating neural networks, with a define-by-run programming model and support for hardware acceleration (Paszke et al., 2019). torchvision supplies the image transforms used for the geometric augmentation described in Section 3.3.4. PyTorch is used offline only: it is too heavy to embed in the game runtime, which is precisely why training and inference are separated into different artifacts.

scikit-learn and Matplotlib. scikit-learn computes the evaluation metrics of Section 3.3.7, producing the per-class precision and recall figures, the classification report, and the confusion matrix; Matplotlib renders the confusion matrix as the figure reported in the study. Neither is present at runtime.

3.4.4 Data Sources and Supporting Technologies

Google Quick, Draw! dataset. Quick, Draw! is a large public collection of labeled freehand sketches gathered from millions of users playing Google's Quick, Draw! game, and is widely used for developing

and evaluating sketch-recognition systems (Ha & Eck, 2017). It supplies all training data for this study. Its limitations are inherent to how it was gathered: the labels are crowd-sourced and therefore noisy, the drawings were made under a time limit and are often unfinished, and its categories differ substantially in size.

TU-Berlin Sketch Dataset. The TU-Berlin Sketch Dataset is a benchmark collection of hand-drawn sketches covering 250 object categories, commonly used to evaluate the generalization performance of sketch-recognition models (Eitz et al., 2012). In this study it is used for evaluation only and never for training, which is what makes it an independent check rather than a second training corpus. It is small relative to Quick, Draw!, was collected under different conditions with markedly thicker strokes, and covers only forty of the fifty classes; the classes it does not cover are named individually rather than absorbed into an aggregate.

ConceptNet. ConceptNet is the commonsense knowledge graph described in Section 3.2.6, and it is the primary grounding source for abilities, with two documented classes using hand-authored mappings. It is consulted offline and resolved into a static lookup table, so it is a design-time dependency and not a runtime one. Its limitations are those of any crowd-derived resource: coverage is uneven, assertion weight measures agreement rather than usefulness, and two of the fifty classes carry no usable assertion at any traversal depth and must be hand-authored.

JSON. JSON is a lightweight text-based format for storing and exchanging structured data, readable by both people and machines (Bray, 2017). It carries the request and response bodies of the recognition endpoint and also serves as the on-disk format for the entity manifest, the label list, and the player profile. As a plain-text format on the player's own

machine, a profile file is editable by the player, so it cannot be treated as trustworthy evidence of how a session was played.

Priority queue. A priority queue is an abstract data structure in which elements are retrieved according to an assigned priority rather than the order of insertion, commonly implemented with heap-based algorithms that allow efficient insertion and removal of the highest-priority element (Cormen et al., 2022). It orders authored dialogue as described in Section 3.2.13, and its variety is bounded by the number of authored lines rather than by the structure itself.

Git. Git provides version control for source code and configuration under a trunk-based workflow, with datasets, runtime caches, and working documents excluded from tracking. It matters to this study beyond ordinary convenience because the model, the label list, and the entity roster form a single versioned artifact set whose consistency is described in Section 3.2.8.

Visual Studio Code. Visual Studio Code is the source-code editor in which the project is written. Its relevance to this study is that OBRA is written in two languages that live in different halves of one repository, and a single editor that understands both keeps them in view together: the Python recognition service and training pipeline, and the GDScript that implements the game logic. Through its language-server support it surfaces the static type information the project relies on, so that a mismatch between what a function returns and what its caller expects is reported while the code is being written rather than discovered during play, and it hosts the integrated terminal and version-control interface through which the training runs and commits are issued. Two qualifications belong with it. First, it contributes nothing to the shipped artifact; unlike every other technology in this section it is absent from the running system, and the project can be built and run without it. Second, Godot ships its own

built-in script editor, so the game half of the codebase can be edited in either environment, and the study does not depend on the choice. The Microsoft-branded distribution is free of charge but is released under a proprietary licence, although the underlying source is available under the MIT licence; it is therefore, with Aseprite, one of the two tools in the toolchain that is not fully open-source, and Section 4.8.3 should be read with that qualification.

Table 2

System Technologies and Constraints

Technology	Role	Limitation
Godot 4 (GDScript)	Captures strokes, renders them, dispatches recognition requests, and animates the result through its built-in 2D physics.	Has no built-in machine-learning runtime, so inference runs in a co-located Python component on the same machine, started by the game itself.
Python, FastAPI, Uvicorn, Pydantic	Expose the model as a type-validated endpoint bound to the host's loopback interface, reachable only from the same machine.	Add encoding, hand-off, and parsing time to every prediction, which is why latency is measured end to end (Section 3.3.7) rather than at the model alone.
ONNX / ONNX Runtime	Store the trained model in a portable format and execute it at runtime without the training framework.	Only some operations can be exported; the exported model is fixed and harder to inspect than its source definition.
PyTorch / torchvision	Define, augment, and train the CNN offline on labeled drawing images.	Too heavy to embed in the game runtime, so training and inference use separate artifacts.
Pillow / NumPy	Decode, crop, resize, and normalize images in the single shared preprocessing routine.	Shared use is a strength for parity but a single point of failure: a change here silently alters both training and play.
scikit-learn / Matplotlib	Compute and render the per-class metrics and the confusion matrix.	Offline only; they report on the model but observe nothing about live play.

Table 2 (Continued)

System Technologies and Constraints

Technology	Role	Limitation
Quick, Draw! dataset	Supplies large numbers of labeled freehand drawings for training.	Labels are crowd-sourced and noisy; drawings were made under a time limit and are often unfinished; categories differ in size.
TU-Berlin dataset	Independent evaluation set of carefully collected human sketches, used to measure generalization.	Small (20,000 sketches across 250 categories), collected under different conditions, and covers only 40 of the 50 classes.
ConceptNet	Commonsense knowledge graph supplying the documented ability attached to each recognized class, resolved offline into a static table.	Coverage is uneven and assertion weight measures agreement rather than usefulness; two classes carry no usable assertion and are hand-authored.
JSON	Carries the request and response bodies and persists the manifest, label list, and player profile.	Plain text on the host and therefore editable by the player; integrity is limited to a schema check.

3.5 Problem Context

Seven technical challenges define the difficulty of this domain, and none is fully resolved by the technologies surveyed above. They are stated here in the order in which they enter the pipeline, together with the design commitment by which the study proposes to contain each.

1. **Drawing is inconsistent and the data is imperfect.** People vary the stroke order, proportion, viewpoint, and style with which they draw the same thing (Eitz, Hays, & Alexa, 2012), and networks built for photographs do not transfer to this domain unchanged (Yu et al., 2015).

The Quick, Draw! drawings themselves were made under a time limit and are often unfinished, and its categories differ in size, so no accuracy figure means anything unless the sampling strategy is reported beside it. The implementation answers this by construction rather than by cleaning: no manual curation, nearduplicate removal, or outlier removal is performed, robustness to position and scale is obtained by cropping to the ink and re-centring, and every accuracy figure is reported with the per-class sampling cap that produced it.

2. **Some classes are hard to separate, and fifty classes are harder than ten.** The primitives circle, square, and triangle are what badly drawn versions of other classes collapse into. The vocabulary also contains dense clusters of its own: quadruped mammals with similar silhouettes (horse, elephant, pig, monkey), thin-bladed or elongated tools (axe, sword, scissors, rake), round forms (circle, wheel, sun, clock, hot air balloon), small winged creatures (bee, butterfly, bat), and aquatic bodies (fish, shark, octopus, crab, sea turtle). These five clusters are the ones named in Section 4.2.9, and they are stated there and here as a hypothesis the confusion matrix tests rather than as a result established in advance. The scale of the cost is documented rather than guessed: the closest published comparison on this dataset reports 89.02%, 87.22%, and 77.46% accuracy on three, ten, and fifty Quick, Draw! classes respectively (Ali et al., 2023), a figure this study's own exploratory scaling experiment corroborates independently (Section 4.2.4.1). Roughly ten points separate a ten-class vocabulary from a fifty-class one, and that is a ceiling to plan against rather than a defect to be surprised by. Recall for the primitives will be reported on its own rather than absorbed into a fifty-class average. The implementation plans against this ceiling rather than hoping to exceed it: the fifty classes were selected for separability from their category neighbours, per-class recall is reported for all fifty with the three primitives

shown individually, and no obstacle on a level-critical path may depend on a class whose held-out recall falls below 0.70.

3. **A classifier forced to choose will guess.** Given fifty labels and no way to decline, it returns one of them for a scribble and for an empty canvas alike, and in a game that wrong answer becomes an action the player never asked for. Thresholding on confidence and margin lets the system decline, but too low a threshold acts on guesses and too high a one refuses drawings that were perfectly clear. Where the thresholds belong is an empirical question, and the confusion matrix is what answers it. The implementation resolves this with a dual gate rather than a single threshold: a prediction is acted upon only when confidence reaches 0.60 and the margin over the runner-up reaches 0.15, both held as configuration values so they can be re-tuned against the confusion matrix without a code change, and a declined drawing costs the player no ink and no progress.

4. **Three failures make no sound.** If the code preparing a training image and the code preparing a player's drawing disagree by even a pixel, the model answers confidently and wrongly. If the list naming the model's numbered outputs is one position out of order, every prediction is mistranslated. To these the expanded design adds a third: if a level is authored against a class the roster does not contain, the obstacle is unsolvable, and neither the model nor the engine will report it, because from their point of view nothing is wrong. None of the three raises an error, and the evaluation metrics stay excellent throughout. They are avoided by construction rather than by testing: one preprocessing definition, one versioned artifact holding the model, the label list and the roster together, and a load-time check that every ability a level requires is supplied by a class in the roster.

5. **The runtime is unforgiving.** The model shares a processor with a game that has roughly sixteen milliseconds to draw each frame, so recognition performed inside that budget would stall it visibly — which is why recognition never runs on the main thread, and why what the player waits through is measured as the whole sequence from capture to spawn rather than the model alone (Miller, 1968; Nielsen, 1993). The drawing to be animated does not exist until it is made, which rules out sprite sheets and pre-authored skeletons; procedural stroke animation works with whatever geometry arrives, but degrades on disconnected strokes and unusual proportions. The implementation keeps recognition off the frame path entirely: the request is dispatched asynchronously and its response handled in a callback, so no frame waits on the recogniser, and the frame rate is required not to fall below fifty-five during a submission. Where a drawing cannot be articulated into a stable jointed body, the rig degrades to a single intact body and the degradation is recorded rather than hidden.

6. **The vocabulary is now a design dependency and not only a model dependency.** Each level offers three mutually exclusive narrative routes, and each route consumes classes of its own, so a single level draws on roughly three times the vocabulary a linear level would. Backtracking compounds this: a class introduced in a late level must remain recognizable when the player carries it back to an early one, which means no class may be tuned for the level that introduced it. Growing the roster to satisfy a puzzle is not free either, since every class added is one more opportunity for confusion for all the others, as the second challenge describes. The vocabulary must therefore be settled before level content is authored against it, rather than adjusted afterwards to accommodate a puzzle.

7. **Measurement is entangled with reward.** Telemetry that previously only observed the player now determines what the player receives: the ending is selected from the accumulated route history, the class diversity count,

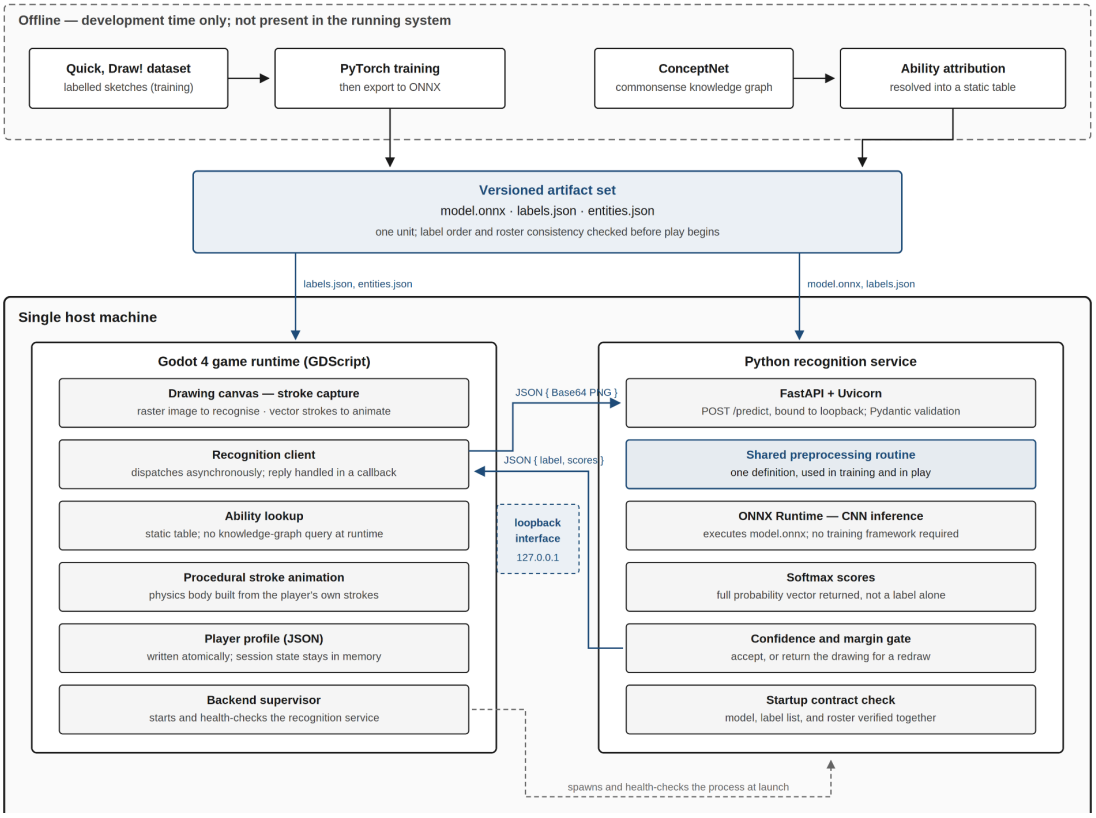
and the redraw rate. A measure the player has reason to move is no longer a clean measure of the system, and a redraw rate that the design rewards cannot at the same time serve as unqualified evidence of recognition quality. This is a research-design problem before it is an engineering one, and it is addressed in Section 4.10.3 rather than resolved here.

Together these challenges show that drawing-driven interaction is not only a model accuracy problem. It is also a data problem, a decision problem, a software-contract problem, a scheduling problem, a rendering problem, a content-design problem, and a measurement problem, and the reliability of the whole is bounded by the weakest of them.

3.6 Illustrations

Figure 2.

Component Architecture of the Proposed System

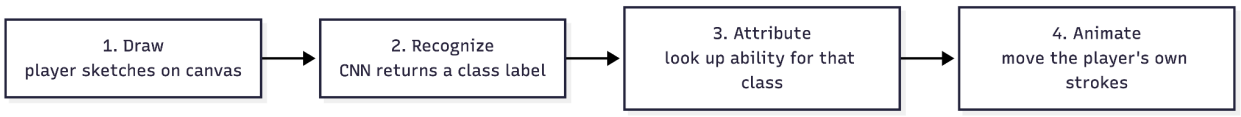


Note. The game runtime and the recognition backend are co-located processes on a single host and exchange data over the loopback interface, which is not a network. The trained model, the label list, and the entity roster form one versioned artifact set.

Figure 2 shows how the components are arranged. The Godot runtime owns the canvas, the game world, and the player profile; the Python recognition service owns the exported model and executes it on request; and the two exchange a single request and response per submission across the host's loopback interface, in the manner described in Section 3.2.9. The training pipeline sits outside the running system entirely, contributing only the exported artifact set, whose mutual consistency is checked before play begins for the reason given in Section 3.2.8.

Figure 3

The Four Stages of One Interaction



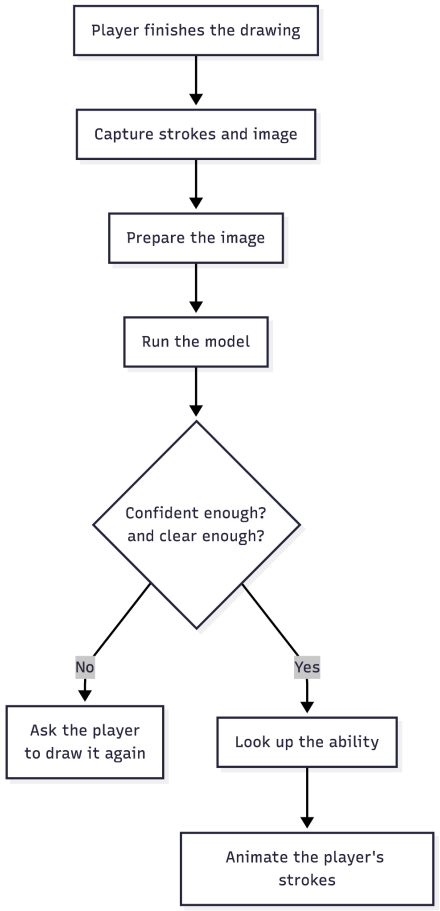
Note. The gate preceding the four stages withholds the drawing panel where the required class has not yet been taught.

Figure 3 shows what happens when the player finishes a drawing. The strokes are captured, the drawing is recognized as one of the fifty classes, that class is looked up in the ability table, and the ability is applied by animating the strokes the player drew. Nothing new is drawn for the player; the original strokes are what moves. The four stages are preceded by a gate; where an obstacle requires a class the player has not yet been taught, the drawing panel does not open and the system reports the missing concept instead. No stage of this sequence produces new artwork. The strokes captured at the first stage are the strokes animated at the last; the system contains no generative image model, no

sprite synthesis, and no substitution of developer- drawn art for the player's drawing.

Figure 4

The Request and Response Cycle



Note. The decision node applies the dual-gate rule of Equation 4.

Figure 4 traces a single recognition, including the decision the system makes about its own certainty. When confidence or margin falls short of the thresholds of Equation 4, the drawing is returned to the player rather than acted upon. When both clear their thresholds, the label is used. Running the model is only one step of several, which is why response time is measured from end to end, as described in Section 3.3.7.

CHAPTER 4
DESIGN AND METHODOLOGY

This chapter contains the research methodology used in the designing, developing, and evaluating of OBRA. It also details the research environment and respondents, sources and instruments for data gathering, and the procedures used to analyze and interpret the collected data. The chapter also discusses system concept, analysis and design, development model and approach, software development tools, project schedule, team responsibilities, and budget. Finally, it explains the verification, validation, and testing process to determine if OBRA's drawing-recognition, ability-attribution, drawing–to-vector conversion, and game components meet the system requirements.

4.1 Research Environment and Respondents

4.1.1 Research Environment

The study uses three publicly available data sources: the Google Quick, Draw! dataset as the primary training corpus, the TU-Berlin Sketch Dataset as a supplementary independent evaluation source, and ConceptNet as the commonsense knowledge source for grounding the entities' in-game abilities. Their characteristics, licenses, selection procedures, class coverage, and limitations are discussed in detail in Section 4.2.

The development environment is a single machine: a MacBook Pro with an Apple M1 Max chip, 64 GB of unified memory, and 2 TB of storage. Model preprocessing, training, and evaluation are performed in Python with PyTorch, using the Metal Performance Shaders backend for GPU acceleration. The game runtime is authored in Godot 4 using GDScript on the same machine. The system is developed on macOS and exported for both Windows and macOS, and the exported build for each platform is verified on that platform before any claim of compatibility is made.

The completed prototype will be deployed and operated on a single computer. The Godot game runtime and the local Python recognition component will run as separate processes on the same device and will communicate through the computer's loopback interface. The two processes exchange data asynchronously: the game runtime dispatches the recognition request and handles its response in a callback, so no frame of the game waits on the recognition component. Work performed on the game's main thread along the submission path is limited to a single frame's viewport capture, image encoding, and transport, which is why the game continues to render at full frame rate throughout the interval between a completed drawing and the appearance of the entity it becomes. The prototype depends on the required local Python environment and model files being available on the host computer, and it writes a single profile file to the host's user data directory in which progression is stored between sessions, with research telemetry written to a separate anonymous per-session log.

Testing will be conducted in the Computer Laboratory of DCISM, University of San Carlos - Talamban Campus, Cebu, Philippines, where participants play the finished game under observation and consistent conditions.

4.1.2 Research Respondents

The study depends on human respondents to answer what the model metrics cannot: whether OBRA, as a playable whole, meets its objectives of usable drawing-based control, immersive play, and a satisfying player experience. Model accuracy alone cannot establish that the draw-to-ability mechanic feels right, that the levels hold attention, or that the game is comfortable to operate.

Sample Size

The evaluation will recruit a minimum of thirty (30) participants. Three considerations determine this number. First, the study uses its instruments descriptively rather than inferentially. Scores from the System Usability Scale (Brooke, 1996), the short version of the User Experience Questionnaire (Schrepp, Hinderks, & Thomaschewski), 2017, the Immersion Experience Questionnaire — Short Form (Cutting, Aung, Perrett, & Cairns, 2025), the selected components of the Game Experience Questionnaire (IJsselsteijn, de Kort, & Poels, 2013), the Godspeed Animacy subscale (Bartneck, Kulić, Croft, & Zoghbi, 2009) , and the QUESI Perceived Familiarity (Hurtienne & Naumann, 2010) subscale will be reported as means with 95% confidence intervals and compared against published benchmarks (Naumann & Hurtienne, 2010); they will not be subjected to significance testing between subgroups. Thirty is the conventional minimum for this class of use. Johanson and Brooks (2010) recommend thirty representatives of the population as a reasonable minimum for pilot and preliminary instrument-based work, and Hertzog (2008), examining the precision of estimates obtained from samples of between ten and forty per group, supplies the confidence-interval reasoning by which a sample of this size is judged adequate for descriptive aims. Second, the instrument on which the study's usability target rests stabilises well below thirty. Tullis and Stetson (2004), comparing five usability questionnaires across 123 participants, found that the System Usability Scale reproduced the conclusion of the full dataset in more than ninety per cent of resamples at sample sizes of twelve to fourteen — the earliest of the five instruments they examined to reach that threshold. A sample of thirty therefore carries substantial margin above the point at which the instrument becomes stable, and that margin extends to the shorter subscales administered alongside it.

Third, thirty supervised sessions generate enough drawing submissions for descriptive live metrics on the classes actually exercised. rather than as a single aggregate figure. Because Section 4.2.1 commits the study to reporting per-class precision and recall together with a confusion matrix, the number of submissions matters as much as the number of participants. Thirty sessions played against a fixed level set yield sufficient observations for the classes exercised on the critical path to be interpreted individually, with the support count stated for each.

What the Sample Size Does Not Support

No a priori power analysis was conducted. The study therefore makes no claim to be statistically powered for inferential hypothesis testing between subgroups, and no such comparison is reported. The sample describes the cohort that played the game; it does not license generalisation to a wider population. This constraint is restated among the limitations in Section 1.4.

Inclusion Criteria

A prospective participant is eligible for the evaluation if all of the following conditions are met:

1. is at least eighteen (18) years of age
2. is a currently enrolled student of the University of San Carlos, or a member of the same campus community
3. is able to operate a mouse or trackpad without assistance;
4. is able to read and respond in English, the language in which all instruments are administered
5. gives written informed consent to participate and to the recording of anonymous session telemetry
6. is available for a single supervised session of approximately sixty (60) minutes

Exclusion Criteria

A prospective participant is excluded from the evaluation if any of the following conditions applies:

1. is a proponent of this study, the faculty adviser, or any other person who has had prior sight of the level design, the class roster, or the ability-attribution table
2. has taken part in a pilot session conducted on the same build
3. has an uncorrected visual impairment that prevents use of the drawing canvas
4. has a motor impairment that prevents freehand drawing with a pointing device. This is an exclusion from the present evaluation only. It is made because the study measures the drawing mechanic rather than its accessibility, and it is recorded among the limitations rather than presented as a property of the design
5. declines the recording of anonymous session telemetry, without which the objective measures described in Section 4.2.1 cannot be obtained

Sampling Technique and Recruitment Procedure

Participants will be obtained by stratified random selection from a screened volunteer pool. The pool is one of convenience, drawn from a population to which the researchers have practical access; the selection of participants within that pool is random and stratified. The procedure has five steps.

1. A short screening questionnaire is circulated to students of the Department of Computer, Information Sciences and Mathematics and to the wider University of San Carlos

community, and remains open for a fixed recruitment window. It collects only what is needed to determine eligibility and stratum membership.

2. Every respondent is assessed against the inclusion and exclusion criteria stated above. Ineligible respondents are removed from the pool and are not contacted further.
3. Each eligible volunteer is assigned to one of four strata formed by crossing two self-reported variables: drawing confidence (lower or higher) and gaming familiarity (lower or higher). The stratification is deliberate. It ensures the recognition mechanic is exercised by hesitant and confident drawers alike, which a sample drawn without regard to drawing confidence would not guarantee.
4. Within each stratum, participants are drawn at random until a quota of seven to eight is filled, giving a minimum of thirty in total. Random selection within strata removes the researchers' discretion over which individual volunteers are invited.
5. A reserve list is maintained in randomly assigned order within each stratum, and is used to replace participants who withdraw or do not attend, so that the stratum quotas are preserved.

Participation is voluntary throughout and may be withdrawn at any point without consequence. Responses are recorded anonymously under a session identifier that carries no personally identifying information, and results are reported only in aggregate. No personal data beyond the coarse background needed to describe the sample and to assign strata is collected at any stage.

Pre-Session and Post-Session Administration

Each participant completes one supervised session, framed by two sets of instruments.

Before play, the participant completes a short pre-session form. This records the background used to describe the sample and to confirm stratum assignment (age band, programme of study, self-reported drawing confidence, self-reported gaming familiarity, and any prior exposure to drawing-based games). It is administered before play so that the participant's answers are not coloured by the session that follows.

After play, the participant completes the post-session instruments described in Section 4.2.1: the System Usability Scale, the User Experience Questionnaire - Short Version, the Immersion Experience Questionnaire - Short Form, the selected components of the Game Experience Questionnaire, the QUESI Perceived Familiarity subscale, and the Godspeed Animacy subscale. These are administered after play because each measures a judgement that can only be formed through use; a usability or immersion rating collected before the participant has operated the system would have no referent.

In-game telemetry is recorded automatically throughout the session and requires no action from the participant. The instruments and the telemetry are analysed together, as set out in Section 4.2.1.

Expert reviewers

Expert reviewers will be recruited by open call rather than by invitation, the research team having no existing professional network in game development. A recruitment notice describing the study, the review task, its expected duration, and the absence of compensation will be posted to public online communities of Philippine and international game developers, including the Indie Game Developers Club of the Philippines

and the Game Developers Association of the Philippines on Facebook, and to game development communities on Reddit. Respondents who express interest will be sent a formal invitation letter endorsed by the thesis adviser, together with the review materials and the review form. The text of the recruitment notice is reproduced in Appendix A; screenshots of the posts as published will be added upon recruitment.

Three to five reviewers are sought. The number follows from the method rather than from the instrument: a single evaluator identifies approximately thirty-five percent of the problems present in a design, five independent evaluators approximately seventy-five percent, and returns diminish beyond five (Nielsen & Molich, 1990; Nielsen, 1994). More than five are invited, since not every respondent who expresses interest returns a completed form.

Reviewers are eligible if they are practicing game developers or recent graduates of game development programmes. Each reviewer records their role, affiliation, and years of experience on the form, together with a link to a portfolio, published game, or professional profile. These links are checked by the research team against the experience claimed, and the results report for each reviewer whether their stated experience was corroborated by a publicly available source. No identity documents are collected. Verification is therefore corroborative rather than conclusive, and this is disclosed as a limitation of an open-call recruitment strategy.

Participation is voluntary and unpaid. Reviewers may decline, withdraw, or leave any principle unanswered, and are told that the materials describe a game still in development. Reviewers are named in the thesis only with consent; those preferring not to be named are identified by role and years of experience.

Cultural content respondents

The cultural content of the game is evaluated by students of the Department of Anthropology, Sociology and History of the University of San Carlos. Purposive sampling is used: respondents are selected for their disciplinary training in Philippine society, culture, and history, which is the qualification that makes their judgement of the game's cultural content meaningful, rather than for representativeness of the general player population. Thirty respondents are sought for the main administration, thirty being the conventional minimum for reliability analysis of a questionnaire (Bujang et al., 2018) and matching the sample size of the gameplay evaluation.

A pilot test precedes the main administration. Fifteen respondents from the same population complete the questionnaire and are excluded from the main administration, so that forty-five respondents are required in total. Fifteen falls within the range of ten to thirty recommended for survey and instrument pilots (Isaac & Michael, 1995; Hill, 1998). Johanson and Brooks (2010) recommend thirty where a pilot is intended to produce stable reliability estimates; the present pilot is intended primarily to identify problems of wording, comprehension, and completion time. Pilot responses are examined item by item — the mean, standard deviation, rate of "cannot judge" responses, and inter-item correlation within each dimension — together with Cronbach's alpha per dimension and the observed completion time. Items that respondents find ambiguous, that fail to discriminate, or that correlate poorly with their paired item are revised or replaced before the main administration; the pilot's internal-consistency figures are treated as indicative rather than confirmatory, and Cronbach's alpha is computed again on the main administration and reported there.

Respondents review the game's storyboard, level descriptions, visual references, and a recorded demonstration of the drawing mechanic;

where a playable build is available, they play it. The materials each respondent reviewed are recorded on the questionnaire and reported, since a respondent who has played the game is answering from a different basis than one who has not. Participation is voluntary and anonymous.

4.1.3 Ethical Considerations and Data Management Plan

Because the study collects data from human participants, its protocol, informed consent form, questionnaire battery, and data-handling procedure will be submitted for ethical review to the Research Ethics Committee of the University of San Carlos, which may be reached at rec@usc.edu.ph. No participant is recruited, and no session is conducted, before clearance is granted. The study is conducted in accordance with the Data Privacy Act of 2012 (Republic Act No. 10173).

Minimizing the risk of harm. The study foresees minimal risk, none of it exceeding the risks of ordinary computer use, and no physical, economic, or legal risk is anticipated. Two minor discomforts are disclosed to participants rather than left unstated. The first is mild frustration, since the recogniser will sometimes decline a drawing and ask for it again; the frequency of that decline is one of the study's own measures, redrawing costs the player no ink and no progress, guided clues are available, and no obstacle on a level-critical path can be passed only by drawing one specific class. The second is fatigue or eye strain, as with any period of screen use. A researcher is present throughout each session, and a participant who appears to be in distress, discomfort, or fatigue is asked whether they wish to continue, reschedule, or stop. No participant is deceived at any point, no personal, sensitive, or medical question is asked, and any item may be skipped. A participant may pause, rest, withdraw, or decline to answer at any time without penalty and without giving a reason; where a session is terminated for any reason, the partial record is discarded and excluded from the analysis unless the participant states otherwise.

Obtaining informed consent. Before any data are collected, a researcher goes through the informed consent form with the participant, answers any question arising from it, and leaves the participant a copy to keep. The form explains the purpose of the study, the nature of the intervention, the procedure to be followed and its approximate duration of sixty minutes, the manner in which participants were selected, the two kinds of data recorded, the absence of direct benefit, the arrangements for confidentiality, and the period for which the data will be held. Participation is voluntary, and the form states that a decision to take part or not to take part has no bearing on a participant's grades, academic standing, or relationship with the Department. Consent is recorded by signature on the certificate of consent before the pre-session form is administered. In recognition of their time, each participant receives a token of one hundred pesos (₱100.00) together with light refreshments during the session; the token is not conditional on completing the session. Questions about the research or about a participant's rights may be directed to the faculty adviser at cfpena@usc.edu.ph or to the University of San Carlos Research Ethics Committee at rec@usc.edu.ph.

Protecting privacy and confidentiality. Personal information is minimised at the point of collection rather than removed afterwards: no participant's name, student number, electronic mail address, or any other identifying detail is entered into the game or written on any questionnaire, and each questionnaire is identified by a randomly generated session code in place of a name. That code links a participant's game record to their questionnaire responses and to nothing else. The signed consent form is the only document bearing a participant's name; it is held in a cabinet under lock and key, separately from all research data, and no document anywhere associates a name with a session code. All computer-based research files are encrypted or password-protected and are held on the researchers' own machines rather than on removable

media, access being limited to the four student researchers and the faculty adviser named in the consent form; each of them signs an undertaking of non-disclosure before access is granted. Any unforeseen or incidental data arising during a session is placed in a separate folder under the same protections. The player's drawings themselves are not retained: they exist only for the duration of a session and are discarded when it ends. The game makes no network connection during play, so no session data leaves the laboratory machine on which it is generated. Results are reported only in aggregate, and no individual participant is identifiable in the thesis, in any presentation, or in any publication arising from it.

Monitoring, retention, and disposal. A logbook is kept of any event of data transfer or sharing, recording what was transferred, to whom, when, and for what purpose. Three (3) years after the completion of the study, that is, by 31 December 2029, all electronic research data will be irreversibly deleted, the deletion extending to the recycle bin, and all paper documents, including the signed consent forms, will be vertically shredded. A participant may withdraw at any time, before or after the session, by quoting their session code, in which case their record is deleted at that point and not replaced.

4.2 Research Instrument and Sources of Data

This section describes both the researcher-made programs used to prepare the recognition data and the standardized external corpora on which the model is built and evaluated, together with the gameplay-evaluation instruments used in the validation phase. The data-preparation instruments and each data source are profiled in turn below, and their validity, coverage, and limitations are stated with them.

4.2.1 Research Instrument

A. Data-preparation instruments

Because the study administers no survey to human participants at the model-building stage, there is no conventional

human-subject instrument for that phase. In its place, a set of researcher-made programs acquires and conditions the drawing data, each with a single, auditable responsibility: a manifest that declares the fifty classes and their source labels and serves as the single source of truth for the vocabulary; a loader that guarantees the label order used in training is exactly the order the game uses to interpret the model's output; an acquisition routine that downloads only the selected classes from the public Quick, Draw! source; a training routine that samples, balances, splits, trains, exports, and records the model together with its labels, metadata, and metrics; a shared preprocessing routine that converts any incoming drawing into the exact form the model was trained on; and an evaluation routine that scores the trained model both on the held-out Quick, Draw! test split and on the independent TU-Berlin corpus, mapping TU-Berlin's categories onto the roster through the manifest's field. No routine trains or fine-tunes on TU-Berlin; the corpus is loaded only after training has completed, which is what preserves it as an independent measure of generalization. The single most important property of this group is that one preprocessing routine is written once and reused by training, offline evaluation, and the live game, which is what keeps training and serving from diverging. These programs do not create data; they select, retrieve, balance, split, adapt, and normalize pre-existing standardized corpora.

B. Gameplay-Evaluation Instruments

The gameplay-evaluation phase uses four standardized questionnaires — the SUS, the UEQ-S, and the IEQ-SF administered in full, and selected components of the GEQ — together with two validated subscales borrowed from adjacent fields, and an automatic in-game telemetry log. Together they cover

what the study needs to observe: whether the game is usable, whether it is immersive, whether the wider player experience is positive, whether the stroke-animated drawing is believable and its granted ability matches what the player expected, and — objectively, without self-report bias — whether players can actually complete the game and how recognition performs in real play. Each questionnaire and subscale is administered after a defined play session; the telemetry is recorded automatically throughout.

System Usability Scale (SUS) (Brooke, 1996). The SUS measures perceived usability, that is, how easy and comfortable the system is to operate. It is a ten-item questionnaire with alternating positive and negative statements rated on a five-point agreement scale, producing a single score from 0 to 100 (Brooke, 1996). It is technology-independent and among the most widely used usability instruments in the field, with strong reported internal-consistency reliability (Cronbach's α of roughly 0.91) across many studies (Brooke, 1996; Lewis & Sauro, 2009). A score above 68 is conventionally read as above average (Sauro, 2011). In OBRA it will measure the usability of the drawing canvas, controls, and menus.

User Experience Questionnaire – Short Version (UEQ-S) (Schrepp, Hinderks, & Thomaschewski, 2017). The UEQ-S measures user experience on two scales, pragmatic quality and hedonic quality, using eight semantic-differential items rated from −3 to +3. It is the validated short form of the twenty-six-item UEQ (Laugwitz, Held, & Schrepp, 2008), and reproduces the full instrument's overall score closely (Schrepp et al., 2017), who report internal-consistency reliability of α = .85 for pragmatic quality and α = .81 for hedonic quality. It is administered because the SUS addresses operability alone: a system can be easy to operate and

still be dull, and a study whose central claim is that drawing-as-gameplay is engaging needs a measure that separates whether the game works from whether it is worth playing. The short form is used rather than the full UEQ because its authors intend it for scenarios in which a user-experience measure is added to an already lengthy battery, which is the case here. Following the same authors' guidance, the pragmatic and hedonic scores are reported separately and no combined score is reported.

Immersion Experience Questionnaire — Short Form (IEQ-SF) (Cutting, Aung, Perrett, & Cairns, 2025). The IEQ-SF measures immersion, the sense of involvement, of losing awareness of the real world, and of being absorbed in the task. It is an eleven-item short form of the thirty-one-item Immersion Experience Questionnaire (Jennett et al., 2008), organized around the subfactors of involvement, real-world dissociation, and challenge; validation studies report that it functions very similarly to the full-length original while being far quicker to administer (Cutting et al., 2025; Jennett et al., 2008). It directly addresses OBRA's objective that drawing-as-gameplay produces an absorbing experience rather than a mechanical one. Cutting et al. (2025) report internal-consistency reliability of $\alpha = .82$ for the eleven-item scale, and $\alpha = .77$, $.75$, and $.68$ for the Involvement, Real-World Dissociation, and Challenge subfactors respectively. The per-subfactor figures are stated because the instrument's author, in granting permission for its use, recommended that Cronbach's alpha be computed per subscale; that recommendation was made in relation to translated administration and does not bind the present study, which administers the scale in its original English, but the source reliabilities are reported here for completeness.

Game Experience Questionnaire (GEQ) (IJsselsteijn, de Kort, & Poels, 2013). The GEQ measures the multidimensional player experience across components such as competence, sensory and imaginative immersion, flow, tension, challenge, and positive and negative affect (IJsselsteijn et al., 2013), with per-component internal consistency reported across the published literature reviewed by Law et al. (2018). Because each GEQ component is scored as an independent average of its own items, administering a subset of components is methodologically sound; the study therefore administers only the three components that map to its objectives — Competence, Sensory and Imaginative Immersion, and Positive Affect (16 of the 33 Core items). Flow and Challenge are omitted because absorption and difficulty are already captured by the IEQ-SF, and Tension and Negative Affect are omitted because recognition-related friction is captured objectively by the decline-and-redraw telemetry; the In-Game module is not used. One limitation is disclosed rather than hidden: the GEQ's factor structure has been questioned in later psychometric work (Law et al., 2018), so its component scores are treated as descriptive indicators of experience rather than as strictly validated subscales, and are interpreted alongside the SUS, IEQ-SF, and telemetry rather than on their own.

Believability and Ability-Expectation. Two further constructs are specific to OBRA's contribution and are not covered by any single games questionnaire: whether the stroke-animated drawing reads as a living, moving entity, and whether the ConceptNet-derived ability matches what the player expected the drawn thing to be able to do. Rather than author new, unvalidated items, each construct is measured with an established, validated subscale borrowed from an adjacent field. Animation believability is

measured with the six-item Animacy subscale of the Godspeed series, a set of semantic differentials (dead–alive, stagnant–lively, mechanical–organic, artificial–lifelike, inert–interactive, apathetic–responsive) widely used in human–robot interaction (Bartneck, Kulić, Croft, & Zoghbi, 2009). Ability-expectation alignment is measured with the three-item Perceived Familiarity subscale of the QUESI, whose items derive from the precondition of prior knowledge — whether a system behaves as the user's existing world-knowledge predicts (Hurtienne & Naumann, 2010). Reliability evidence for the Animacy subscale comes from work subsequent to its publication: Ho and MacDorman (2010) report internal consistency of approximately .88 for animacy, and a later review of applications of the series reports Cronbach's alpha for animacy in the range .70 to .76 (Stroessner, 2020). Two limitations are nevertheless disclosed. The Godspeed animacy and anthropomorphism indices correlate strongly ($r \approx .89$; Ho & MacDorman, 2010), so the Animacy score is read as a descriptive indicator of believability rather than as an independent construct; and QUESI Perceived Familiarity is the closest validated proxy for the ability-expectation construct rather than a purpose-built measure of it, so it is interpreted alongside the per-class recall telemetry rather than on its own.

In-game telemetry. Complementing the self-report questionnaires, the game logs objective behavioral and performance data automatically during every session, so that claims about playability and recognition rest on what players actually did and not only on what they reported. Five families of measures are captured.

(1) **progression** — Level completion rates and time-on-level per player; the narrative route selected at each dialogue node; the number of returns to a previously

completed level; and the collectibles recovered. Time-on-level is reported within route and never pooled across routes, since the routes differ in the amount of content they contain by design.

(2) **Recognition performance in real play** — the acceptance rate, the rate at which drawings are declined and redrawn, and the distribution of model confidence and margin, recorded for every submission. Ground truth is available for only part of this record. An affordance-based obstacle states the capability it requires and not the class that must supply it, so a submission against such an obstacle carries no intended label and contributes to the acceptance, decline, and confidence figures alone. Obstacles that solicit a class by name — the tutorial obstacles and the concept-gated obstacles identified in Section 4.5.4.6 — do record an intended label, and those submissions alone yield per-class precision and recall from live play. Live per-class figures are therefore reported separately from the held-out and TU-Berlin figures, restricted to the classes so solicited, and each carries its support count.

(3) **Responsiveness** — the end-to-end recognition response time, measured from stroke submission to the spawned entity becoming visible in the world, since this whole-path figure and not model inference time alone is what the player experiences.

(4) **Play style profile.** The class diversity count and the redraw rate defined in Section 3.3.8, together with the route history from which the ending is selected.

(5) **Gameplay Performance.** The frame rate sustained during each submission, the utilisation of the

processor and, where the platform exposes a counter, of the graphics processor, and the resident memory of the game runtime and the recognition component reported separately. These are sampled at one-second intervals for the duration of the session and reported as a median and a peak. Because recognition is dispatched asynchronously, a frame rate that holds through the submission window is also the evidence that no frame of the game waits on the recognition component.

The fourth family requires an explicit methodological caution, because it is the point at which a research measure and a game reward coincide. Class diversity, together with route history and collectibles, determines which of the four endings the player receives; redraw rate is research telemetry only and does not affect the ending.

A participant who learns this has an incentive to draw badly on purpose, at which point the redraw rate ceases to measure recognition quality and begins to measure the participant's interest in an ending. Three measures are therefore taken. The conditions attaching to the endings are not disclosed to participants before or during the session. Recognition figures are computed from the first supervised session only, and any free play that follows is recorded separately and excluded from the recognition analysis. The post-session questionnaire asks participants directly whether they deliberately submitted drawings they expected to be rejected, and sessions answering in the affirmative are reported separately rather than discarded silently.

C. Expert Design Review Instrument

The instruments above observe the system and its players. Neither captures the judgement of people who build games professionally, which is what identifies design problems while they are still cheap to correct. A third instrument is therefore administered to a panel of game development practitioners. Reviewers are not participants; they receive no intervention and are not part of the human-participant sample.

Principles of Game Playability (PLAY). PLAY measures playability — the degree to which a design supports an engaging and operable player experience — in fifty principles across three categories: Game Play; Coolness, Entertainment, Humour and Emotional Immersion; and Usability and Game Mechanics (Desurvire & Wiberg, 2009). It is the refined successor to the Heuristics to Evaluate Playability of Desurvire, Caplan and Toth (2004), developed to generalise across game genres and to be usable earlier in development.

PLAY is selected over general usability heuristics, and over its own predecessor, for three reasons.

It was validated empirically rather than by face validity alone. Fifty-four gamers rated high- and low-ranked games, drawn from Metacritic aggregate scores, against 116 candidate principles; forty-eight differentiated significantly between high- and low-ranked games at $p < .0004$ (Desurvire & Wiberg, 2009). A later study found that evaluators and designers using PLAY identified substantially more problems, fixes, effective design elements, and enhancements than the same participants working without heuristics, with all four effects significant at $p < .001$ (Desurvire & Wixon, 2013).

It was designed for the stage this study is at. Its authors state that it was created to assist developers throughout the design process, particularly at the beginning of the concept phase when changes to the design are less costly — which is the condition under which OBRA is reviewed here.

And it was recommended by its author. Permission to use HEP was granted by Dr. Heather Desurvire on August 11, 2026, who in the same correspondence directed the research team to PLAY as the more refined instrument. The study follows that recommendation.

All fifty principles are administered in their published wording, in the order printed in Table 1 of the source. Nothing is reworded, merged, renamed, omitted, or added. Although the authors state that PLAY may be used modularly, with categories removed where a game lacks the relevant component, no such removal is made here: OBRA has a story, an interface, and a control scheme, and every category therefore applies. Where a principle cannot be judged from the materials supplied, the reviewer records this rather than guessing, and the study reports it.

Each principle is rated met, partially met, not met, or cannot assess. Items rated partially met or not met receive a severity rating on Nielsen’s (1994) five-point scale, 0 for a non-problem through 4 for a catastrophic one, with a written description of the problem. The form also carries seven open-response questions on the drawing mechanic — the team’s own, and identified as such — and a problem log. Reviewers receive the game design plan, the system flowchart, a recorded demonstration, and a prototype build; Part A records whether each reviewer played the build. Completion takes approximately sixty minutes.

PLAY is a principle inventory, not a psychometric scale, so Cronbach's alpha is neither reported by its developers nor meaningful for it. Reliability here is a function of the number of independent evaluators: averaged across six projects, one evaluator found about thirty-five percent of the problems present and five found about seventy-five percent, with returns diminishing beyond five (Nielsen & Molich, 1990; Nielsen, 1994). The study recruits three to five reviewers, working independently of one another and of the proponents. Results are reported as the unique problems identified, the number of reviewers identifying each, the mean severity, the open-question answers, and the team's disposition of each problem.

Three limitations are disclosed. Roughly a quarter of the problems present are expected to go undetected even at five reviewers, so the review establishes that the identified problems exist, not that others do not. A subset of principles concerns qualities that emerge only across an extended play session — enduring play, the pacing of mastery, replay — and reviewers working from artifacts rather than a full playthrough are expected to record these as cannot assess; the number of such responses is reported per principle. And the review is formative and does not substitute for the gameplay evaluation in Section 4.2.1B.

Copyright in the source is held by the authors. Written permission to use the instrument was granted by Dr. Heather Desurvire on August 11, 2026 conditional on credit and on documentation of any changes; both conditions are met, the instrument being administered without alteration and fully cited. The correspondence is reproduced in Appendix H. Independently, under Republic Act No. 8293, Section 175, copyright does not extend to

any idea, procedure, system, method, or concept, even where embodied in a protected work.

D. Cultural Content Questionnaire

The instruments above measure whether the system works and whether it plays well. Neither addresses whether the game's cultural content is accurately and appropriately handled, and that question cannot be answered by the research team, whose own membership is the source of the material under review. A fourth instrument is therefore administered to students of anthropology, sociology, and history.

No validated instrument exists for assessing the cultural appropriateness of a game's depiction of a specific culture. Existing cultural instruments measure attributes of the respondent — cultural identity, cultural awareness, acculturation — rather than the adequacy of a cultural depiction, and instruments developed for cultural-heritage games measure engagement and learning rather than accuracy. The study therefore developed its own instrument, grounded in a published framework rather than composed from scratch.

The **Cultural Content Questionnaire** measures perceived cultural appropriateness, the degree to which the game's representation of Philippine settings, figures, relationships, and values is accurate and defensible to those settings. Its structure is taken from the Ecological Validity Model of Bernal, Bonilla and Bellido (1995), which specifies eight dimensions along which work developed in one cultural setting must be examined for adequacy in another (language, persons, metaphors, content, concepts, goals, methods, and context) and which distinguishes surface-level adaptation, reaching only names, images, and settings, from

deep-structure adaptation, reaching the values and meanings beneath them. That distinction is one this study must be prepared to fail: a game whose Filipino content resides in setting, story, characters, and art direction, while its core mechanic is culturally neutral, is by that description a candidate for surface-level adaptation, and the questionnaire asks the question directly rather than assuming the answer.

The eight dimensions are retained as published, with two items written by the research team under each, giving sixteen dimension items. Five further items address the individual levels, since a dimension may be handled well across the game and poorly in one place. Respondents rate each of the twenty-one items on a five-point agreement scale, coded 1 for strong disagreement through 5 for strong agreement, with higher scores indicating greater cultural appropriateness; a “cannot judge” option records that the respondent does not consider themselves qualified to answer, and is reported rather than treated as missing data. Dimension scores are the mean of their two items. Six open-response questions follow, including a direct question on whether the game reaches beyond surface-level representation.

Because respondents’ regional knowledge will vary, the questionnaire records how familiar each respondent is with the Cordillera, Visayan festivals, coastal fishing communities, Philippine folklore, and the Bicol region. Ratings are reported alongside that familiarity profile, so that judgements of a setting are weighed by the knowledge of those making them.

Content validity rests on the derivation of the instrument rather than on expert ratings. Every dimension and its definition are taken from the published framework, and each item is written to

operationalise the dimension it sits under; no dimension was added, removed, merged, or renamed. Item clarity and comprehension are checked in the pilot, and items that respondents find ambiguous are revised before the main administration. Formal expert content-validity ratings were not obtained, and this is disclosed as a limitation. Reliability is assessed by Cronbach's alpha computed per dimension on the main administration; because each dimension carries two items, the coefficients are reported alongside item-level results rather than in place of them.

Under Republic Act No. 8293, Section 175, copyright protection does not extend to any idea, procedure, system, method, or concept, even where embodied in a protected work. The eight dimensions are a framework of this kind, the items administered are the research team's own composition, and no item is reproduced from the source. The framework is fully cited.

Three limitations are disclosed. Respondents are students rather than credentialed experts, so the instrument reports informed judgement rather than authoritative assessment. Respondents drawn from a single Cebu-based department will be strongest on Visayan material and weakest on Cordillera and Bicol content, which the familiarity profile makes visible. And where respondents review design materials rather than a playable build, they cannot judge how the content reads in motion, in audio, or in the pacing of play.

4.2.2 Sources of Data

Table 3

Data Sources at a Glance

Data source	Type	Role in OBRA	Used for training?	License	Citation
Google Quick, Draw!	Image corpus (sketch bitmaps)	Primary training corpus for the sketch classifier	Yes – initial training	CC BY 4.0	Google Creative Lab (2016); Ha & Eck (2017/2018)
TU-Berlin Sketch Dataset	Image corpus (raster sketches)	Independent test dataset	No	Cited to original authors; mirror states no license	Eitz, Hays, & Alexa (2012)
ConceptNet 5.5	Commonsense knowledge graph	Knowledge base for ability attribution (resolved offline into a static ability table)	No (knowledge lookup)	CC BY-SA 4.0	Speer, Chin, & Havasi (2017)

4.2.3 Primary Data Source (Training Corpus)

Table 4

Dataset Profile: Google Quick, Draw! (Primary Training)

Parameter	Description
Source Platform	Public Google Cloud Storage release, maintained by Google Creative Lab
Dataset Name	Google Quick, Draw! Dataset (numpy_bitmap variant)
Total Instances	6,982,163 drawings across the 50 enabled classes (raw total recomputed after download)

Table 4 (Continuation)

Dataset Profile: Google Quick, Draw! (Primary Training)

Parameter	Description
Instances Used	2,000,000 drawings (capped to 40,000 per class before splitting)
Features per Instance	784 pixel intensities (a 28 × 28 grayscale raster, flattened)
Data Format	NumPy .npy bitmap arrays, dtype uint8, one file per class
On-disk Size	≈ 5.1 GB (50 files)
Class Distribution	Imbalanced as downloaded (117,999–253,791 per class; ≈ 2.15 : 1), rebalanced to an equal 40,000 per class by capping
Class Retained	50 of the 345 categories in the full release
Target Variable	Class index 0–49 (the fifty-class label)
License	Creative Commons Attribution 4.0 (CC BY 4.0)
Role in OBRA	Primary corpus used to train the sketch classifier

Basis for the Selection of the Training Corpus

Four considerations determined the choice of Google Quick, Draw! as the primary training corpus. First, scale: the release supplies roughly fifty million labelled drawings across 345 categories (Ha & Eck, 2017), of which 6,982,163 fall within the fifty enabled classes, so every class can be capped to an equal quota without any class being exhausted. Second, licensing: the corpus is distributed under a Creative Commons Attribution 4.0 licence, which permits use and redistribution in an academic study on the sole condition of attribution and imposes no cost, consistent with the study's commitment to a toolchain reproducible at zero licensing cost. Third, form: the numpy_bitmap release is already rendered to 28 × 28 grayscale rasters under a common ink convention, so the corpus arrives normalised in the respects that matter to the classifier and requires no rendering step of the researchers' own that could diverge from the one

used at play time. Fourth, precedent: the corpus is the standard benchmark of the freehand sketch recognition literature and the basis of the recognition results reviewed in Chapter II (Ha & Eck, 2017; Ali et al., 2023), which makes the per-class figures reported here comparable with published results on the same data rather than only with themselves.

The corpus also carries limits that the study accepts rather than conceals. The drawings were produced in a browser game under a twenty-second limit, so many are unfinished; the labels are crowd-sourced and therefore noisy; and the categories are imbalanced as downloaded, ranging from 117,999 to 253,791 drawings per class, a ratio of roughly 2.15 to 1. The first two are treated as label noise the model is expected to tolerate, since a player at a canvas draws under comparable haste. The third is corrected by capping each class at an equal 40,000 drawings rather than by loss re-weighting, and that cap is reported alongside every metric derived from the resulting pool.

The training data is the Google Quick, Draw! dataset, a collection of freehand doodles gathered through a browser game in which players drew a prompted object within a twenty-second limit. It was compiled and is maintained by Google Creative Lab and is distributed under a Creative Commons Attribution 4.0 (CC BY 4.0) license. The study uses the dataset's pre-rendered bitmap release, in which each drawing is already supplied as a small grayscale raster, retrieved directly from Google's public storage for the selected classes.

4.2.4 Dataset Profile and Characteristics

Each class is supplied as one array of drawings, every drawing a 28 × 28 grayscale bitmap with ink rendered bright on a dark background and centered to roughly fill the frame by Google's own pipeline. All fifty enabled classes were downloaded in full, totaling 6,982,163 drawings (approximately 5.1 GB), of which only a capped subset is used for training

(Table 5). The categories are imbalanced as downloaded, ranging from 117,999 drawings (butterfly) to 253,791 drawings (penguin), an imbalance of roughly 2.15 to 1, which the per-class sampling cap described in Section 4.2.6 later rebalances.

The fifty classes fall into three functional groups: twenty creatures spawned as playable, movement-driven entities; twenty-seven objects spawned as tools, structures, and vehicles; and three geometric primitives spawned as physics construction objects.

Table 5

Class Distribution Data

Entity (game id)	Quick, Draw! source label	Drawings (raw)	On-disk (MB)
circle	circle	122,876	91.9
square	square	125,145	93.6
triangle	triangle	123,170	92.1
bird	bird	133,572	99.9
bat	bat	118,114	88.3
butterfly	butterfly	117,999	88.2
spider	spider	209,447	156.6
monkey	monkey	127,633	95.4
frog	frog	159,047	118.9
fish	fish	134,150	100.3
shark	shark	126,050	94.2
octopus	octopus	150,152	112.3
crab	crab	126,930	94.9
horse	horse	178,286	133.3
elephant	elephant	126,969	94.9
snake	snake	122,273	91.4
sea_turtle	sea turtle	119,876	89.6
snail	snail	133,757	100
ant	ant	124,612	93.2
bee	bee	120,890	90.4
pig	pig	186,770	139.6

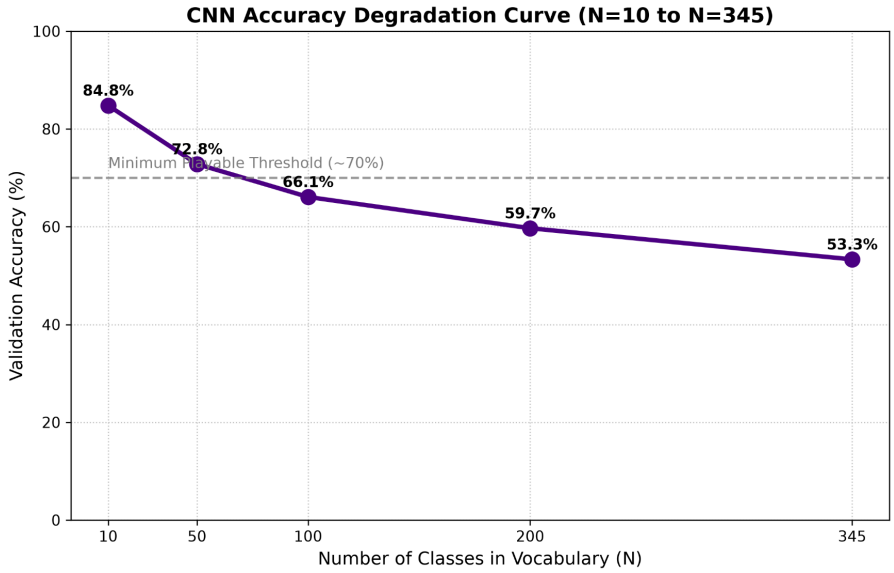
Table 5 (Continued)
Class Distribution Data

Entity (game id)	Quick, Draw! source label	Drawings (raw)	On-disk (MB)
penguin	penguin	253,791	189.8
scorpion	scorpion	165,689	123.9
axe	axe	124,122	92.8
sword	sword	123,802	92.6
cannon	cannon	141,394	105.7
boomerang	boomerang	142,682	106.7
flashlight	flashlight	239,763	179.3
campfire	campfire	133,395	99.7
sun	sun	133,781	100
cloud	cloud	120,265	89.9
fan	fan	136,158	101.8
ladder	ladder	125,389	93.8
stairs	stairs	128,981	96.4
parachute	parachute	127,319	95.2
hot_air_balloon	hot air balloon	126,350	94.5
bridge	bridge	133,010	99.4
sailboat	sailboat	136,506	102.1
submarine	submarine	124,362	93
key	key	160,965	120.4
door	door	120,230	89.9
rake	rake	154,639	115.6
scissors	scissors	123,390	92.3
clock	clock	120,536	90.1
anvil	anvil	126,231	94.4
bucket	bucket	124,064	92.8
umbrella	umbrella	124,084	92.8
tree	tree	144,721	108.2
mushroom	mushroom	142,167	106.3
wheel	wheel	136,659	102.2
Total		**6,982,163**	**5220.6**

Note. On-disk sizes are mebibytes (1 MB = 1,048,576 bytes) as reported by the filesystem for the .npy bitmap arrays. Values are rounded to one decimal place.

4.2.4.1 Validation of the 50-Class Vocabulary against HCI Usability Thresholds

Figure 5
CNN Accuracy Degradation Curve



As preliminary work supporting this proposal, Figure 5 plots validation accuracy against vocabulary sizes, the researchers conducted an exploratory scaling experiment to confirm that a fifty-class vocabulary is recognizable at a level the game's interaction loop can tolerate. A lightweight CNN, separate from the PyTorch/ONNX pipeline described in Section 4.8, was trained in TensorFlow/Keras across five vocabulary sizes drawn from Quick, Draw!'s full category list as nested prefixes in their native ordering — 10, 50, 100, 200, and 345 classes — at 2,000 samples per class.

Validation accuracy fell from 84.8% at N=10 to 72.8% at N=50, 66.1% at N=100, 59.7% at N=200, and 53.3% at N=345, with the steepest single loss occurring between N=10 and N=50.

As the vocabulary grows, softmax divides its probability mass among more competing classes, lowering both top-1 confidence and the top-two margin independent of any change in visual distinctness. Because the game's abstention rule (Section 3.2.3) gates on confidence and margin rather than raw accuracy, redraw frequency is expected to rise faster than accuracy falls as N grows. A minimum accuracy floor was set above this compounding point: 70%, a round-number analog to the 68-point "above average" convention already used for the SUS (Brooke, 1996), applied here to recognition accuracy instead of perceived usability.

Figure 6

OBRA Curated 50-Class Selection: Confusion Matrix Validator

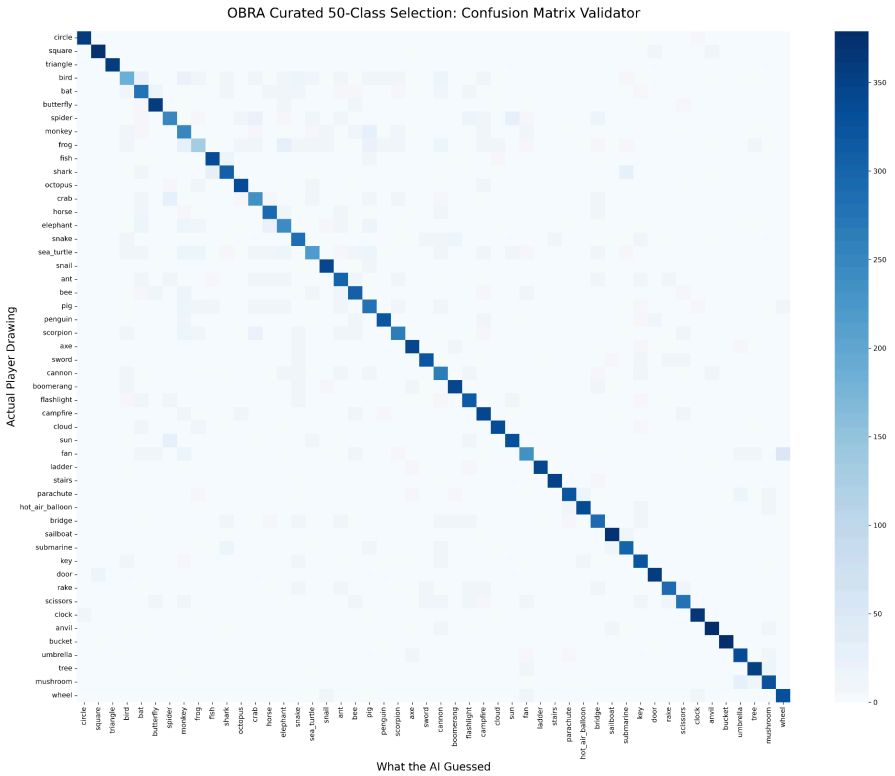


Figure 6 shows the confusion matrix for OBRA's curated fifty-classes roster. A second model, trained under the same

procedure and the same 2,000-samples-per-class budget on OBRA's actual curated fifty-class roster rather than an alphabetical slice, reached 77.3% accuracy on a held-out test set, 4.5 points above the arbitrary N=50 baseline, isolating the effect of curation from the effect of sample size. This figure independently corroborates the closest published comparison on Quick, Draw!, which reports 77.46% accuracy at fifty classes (Ali et al., 2023; Section 3.5). The confusion matrix, drawn from the same held-out set, shows a strong diagonal with confusion concentrated between visually adjacent classes, indicating that selecting each class for separability from its category neighbors measurably improves on an arbitrary fifty-class vocabulary of the same size. Kocielnik, Amershi, and Bennett (2019) found that users can accept an imperfectly accurate AI system when the interface sets correct expectations about its errors; OBRA's dual-gate rule (confidence ≥ 0.60, margin ≥ 0.15) does this by turning a misclassification into a redraw rather than a wrong spawn. Henderson et al. (2022) and Alghofaili et al. (2024) further report that user frustration in recognition-based input is driven by false-positive recovery time, not redraw frequency — which is why the 0.60 gate is set conservatively, favoring the cheap failure over the costly one.

The fifty classes also match OBRA's level structure: twenty creatures split into aerial, aquatic, and land locomotion types, three primitives, and twenty-seven objects across combat, elemental, traversal, and utility roles, giving each level environment at least one viable morph or tool without exceeding the accuracy budget above.

4.2.5 Data Schema and Feature Definitions

The training data is unstructured image data, so the schema is described in terms of image properties and the derived model input rather than named columns.

- **Resolution and channels:** 28 × 28 pixels, one grayscale channel.
- **Storage:** each drawing is a fixed-length row of pixel intensities (0–255); the class label is not stored inside the data but is supplied by the class and fixed to the roster's ordering.
- **Independent variables (inputs):** the pixel-intensity grid of each drawing, reshaped to the model's input and rescaled from 0–255 to a 0–1 range before entering the network.
- **Dependent variable (target):** the class index 0–49, produced in the roster's order and mapped to the corresponding in-game entity.

4.2.6 Data Inclusion and Exclusion Criteria

The rules below determine which parts of the large Quick, Draw! corpus are kept. They are applied in code, not by hand, so they are reproducible.

Inclusion. A drawing is usable if it belongs to one of the fifty enabled classes; if it comes from the pre-rendered bitmap release, so that it is already normalized to a common resolution and ink convention; and if it falls within the per-class sampling cap. For training, 40,000 drawings from each class are randomly selected without replacement, which rebalances the naturally imbalanced corpus into an equal quota and yields a balanced training pool of 2,000,000 drawings.

Exclusion. A drawing is filtered out if its class is not enabled; if it exceeds the 40,000-per-class cap; or, at inference and evaluation time, if the canvas contains no ink, in which case it is rejected before reaching the model.

Resulting balanced split. After capping, the pooled 2,000,000 drawings are shuffled with a fixed random seed and partitioned 80/10/10 into training, validation, and test sets, giving 1,600,000 training, 200,000 validation, and 200,000 test drawings with near-equal per-class support. The test set is held out and touched only once, for the final per-class figures; the validation set is used during training to monitor for overfitting.

4.2.7 Supplementary Testing Set

Table 6
Dataset Profile: TU - Berlin Sketch Dataset

Parameter	Description
Source Platform	Computer Graphics group, Technische Universität Berlin; accessed via a third-party Hugging Face mirror (`sdiaeyu6n/tu-berlin`), disclosed as the retrieval route only
Dataset Name	TU-Berlin Sketch Dataset (human sketch benchmark)
Total Instances	20,000 sketches across 250 object categories (80 sketches per category)
Instances Used	Only the sketches in categories that map to OBRA's vocabulary (see mapping in 4.2.7)
Features per Instance	Raster images at high native resolution with markedly thicker strokes; normalized to the model's 28 × 28 grayscale input before use
Data Format	Raster image files
Class Coverage	Maps to 40 of the 50 OBRA classes (same-named TU-Berlin folders)
Role in OBRA	OBRA Independent test set for generalization
License	No license stated by the mirror; the corpus is cited to its original authors

TU-Berlin was selected as the supplementary testing set for one reason the training corpus cannot satisfy: its sketches were drawn by different people, under a different protocol, without a time limit, and none has been seen by the model at any point in training (Eitz, Hays, & Alexa, 2012). A generalisation figure computed on a held-out split of the training corpus measures performance on drawings made the same way as those the model learned from; a figure computed on TU-Berlin measures performance on drawings made differently, which is the closer analogue of

a player at a canvas. Its 250 categories map to 40 of OBRA's 50 classes, so the coverage is partial by construction: that coverage figure is stated wherever a TU-Berlin result is reported, the ten classes without coverage are named rather than absorbed into an aggregate, and no TU-Berlin figure is presented as a fifty-class result.

To test generalization on sketches drawn under a different protocol, the study also uses the TU-Berlin sketch dataset (Eitz, Hays, & Alexa, 2012): 20,000 human-drawn sketches across 250 object categories, collected by the Computer Graphics group at Technische Universitat Berlin. It is used only for evaluation and never for training, and is accessed through a third-party mirror on Hugging Face (https://huggingface.co/datasets/sdiaeyu6n/tu-berlin); the mirror is a re-upload with no stated license, so the corpus is cited to its original authors and the mirror is disclosed solely as the retrieval route.

TU-Berlin's category set does not align cleanly with the fifty OBRA classes. The manifest's per-entity evaluation_labels field encodes the label-mapping that model/evaluate_folder.py applies when scoring against it:

- bird folds TU-Berlin’s two bird categories — flying bird and standing bird — into the single OBRA bird class.
- Ten classes have no same-named TU-Berlin folder and are evaluated only on the held-out Quick, Draw! split: the three geometric primitives (circle, square, triangle), plus bat, campfire, stairs, rake, clock, anvil, and bucket.
- All remaining classes map to their single same-named TU-Berlin folder; in total, 40 of the 50 classes have a TU-Berlin test folder.

TU-Berlin also differs in representation; it distributes high-resolution raster images with markedly thicker strokes, so its images are normalized to the same 28 x 28 tensor through the shared backend/preprocess.py

before inference, ensuring the independent test set is seen through exactly the same preprocessing as the live game.

TU-Berlin was selected as the independent evaluation set for one reason that the training corpus cannot satisfy: it was drawn by different people, under a different protocol, without a time limit, and it has never been seen by the model at any point in training. A generalisation figure computed on a held-out split of the training corpus measures performance on drawings made the same way; a figure computed on TU-Berlin measures performance on drawings made differently, which is the closer analogue of a player at a canvas. Its category set does not cover the whole vocabulary, so the covered subset is defined by the manifest and enumerated in Section 4.2.7, the coverage figure is stated wherever a TU-Berlin result is reported, and classes without coverage are named individually rather than absorbed into an aggregate

4.2.8 Tertiary Data Source: ConceptNet (Ability-Attribution Knowledge Base)

ConceptNet is a knowledge graph rather than a training corpus, its profile is described in terms of concepts, relations, and assertions rather than rows or images.

Table 7

Data Source Profile: ConceptNet

Parameter	Description
Source Platform	Open-source project of Luminoso Technologies, originating at the MIT Media Lab under the Commonsense Computing Initiative
Resource Name	ConceptNet 5.7.0, a multilingual commonsense knowledge graph. Attribution is resolved against the version 5.7.0 assertions dump; the resource is cited to Speer, Chin, and Havasi (2017), which introduces the 5.5 release and remains its canonical description.

Table 7 (Continued)
Data Source Profile: ConceptNet

Parameter	Description
Structure	Natural-language concepts (nodes) linked by labeled, weighted, directed assertions (edges)
Size	Over 21 million assertions across 83 languages; only the English subset is used
Relation Set	A closed set of 36 core relations; OBRA consults six affordance-bearing relations: CapableOf, UsedFor, HasProperty, HasA, ReceivesAction, and MannerOf, together with IsA for inheritance traversal where a class carries no usable edge of its own. (See Appendix C)
Data Format	Assertion triples of the form (start concept, relation, end concept) with a confidence weight; accessed through the ConceptNet REST API and downloadable edge dumps
Output Taken into OBRA	The ability term(s) attached to each entity by the relevant relation (e.g., fish → *swim*, axe → *chop*), selected offline and stored in a static ability lookup table
License	Creative Commons Attribution-ShareAlike 4-0 (CC BY-SA 4.0)
Role in OBRA	Knowledge base that grounds each recognized entity's in-game ability in real-world commonsense knowledge

Unlike the two sketch corpora, which supply what OBRA can *recognize*, ConceptNet supplies what a recognized entity can *do*. It is the knowledge base behind this study's knowledge-grounded ability attribution: once the classifier has identified a drawing as one of the fifty classes, that class's in-game ability is not invented arbitrarily but is drawn from real-world commonsense knowledge encoded in ConceptNet.

Table 8

Data Coverage

Measure	Count
Classes with a ConceptNet-derived ability	48 of 50
Classes hand-authored (penguin, mushroom)	2
Total abilities attributed	117
Mean abilities per grounded class	2.4

ConceptNet is not used as a training corpus and is not queried during gameplay. The researchers consult it during development, select appropriate ability terms, document the supporting assertions, and store the resulting mappings in a static lookup table. Where ConceptNet supplies more than one candidate assertion for a class, selection proceeds by game requirement first and assertion weight second. Only assertions naming a capability that a level actually requires of the class are eligible; among the surviving candidates, weight breaks ties. This ordering is deliberate. A ConceptNet weight counts how many independent contributors asserted an edge, so it measures how widely a fact is agreed upon, not how useful the corresponding ability is as a game mechanic. Taking the highest-weighted edge without a requirement filter would ground abilities that are well attested and unplayable. The selected assertion, its relation, and its weight are reported for every class, so that each attribution can be audited against the source; the weight documents corroboration and does not by itself justify an ability. Where no usable assertion exists at any depth — as for penguin and mushroom — the ability is hand-authored and explicitly marked as such, so that the provenance of every ability, whether knowledge-graph-derived or hand-authored, is documented rather than assumed. The three geometric primitives are not among the hand-authored cases. None carries a

CapableOf edge, and a shape is not the kind of thing commonsense describes as capable of acting; each is instead grounded through a structural relation — circle through HasProperty (round), triangle through HasA (three sides), and square through both HasA (four sides) and a UsedFor edge (walked on). Extending the relation set beyond CapableOf and UsedFor is precisely what makes these three classes groundable rather than authored.

Table 9

Attribution Modes

Mode	Relation(s)	Rule	n
Direct	CapableOf, UsedFor	The ability is stated on the class itself.	84
Property	HasProperty	A documented property implies the behavior it produces under the game's physics.	8
Part	HasA	A documented part implies the affordance that part provides.	9
Action	ReceivesAction, MannerOf	An action the thing receives, or an action the thing is a manner of performing, implies the affordance.	7
Inherited	IsA traversal	No usable edge on the class; the ability is taken from a named parent concept.	9

4.2.9 Data Quality and Limitations

- **Label noise.** Quick, Draw! labels are crowd-sourced and the drawings were made under a time limit, so some are unfinished, ambiguous, or off-prompt. No manual cleaning is performed beyond the automated inclusion and exclusion rules; the model is expected to tolerate this noise.
- **Imbalance handled by capping.** The raw corpus is imbalanced, but the fixed 40,000-per-class cap converts it

into a balanced training pool, so balance is achieved by sampling rather than by loss re-weighting.

- **Distribution shift, measured but not corrected.** The training drawings were elicited under a timed browser game, whereas OBRA's canvas is untimed. Evaluating TU-Berlin's untimed, thicker-stroke sketches measures how far performance falls under that shift for the forty classes TU-Berlin covers. It does not correct the shift, and it says nothing about the ten classes TU-Berlin does not cover. The study reports the size of the gap rather than claiming to have closed it.
- **Low native resolution and a denser vocabulary.** Because the source bitmaps are 28 x 28, fine detail that distinguishes visually similar classes is largely unavailable to the model. At fifty classes the vocabulary deliberately contains several confusable clusters, namely quadruped mammals (horse, elephant, pig, monkey), thin-bladed or elongated tools (axe, sword, scissors, rake), round forms (circle, wheel, sun, clock, hot air balloon), small winged creatures (bee, butterfly, bat), and aquatic bodies (fish, shark, octopus, crab, sea turtle). The strongest off-diagonal confusions are expected within these clusters, a hypothesis the confusion matrix will test, and it is for this reason that per-class recall rather than a single accuracy figure is what the evaluation reports.

4.3 Research Procedure

OBRA is a software artifact whose value is only partly captured by model metrics; the rest is captured by whether people can play it and enjoy it. Its research procedure is therefore an iterative game-development procedure, not only a data pipeline. The study moves through five phases: (1) requirements and design, in which the game concept, class vocabulary, level structure, and system

architecture are specified; (2) data preparation, in which the training drawings are gathered and treated; (3) system development, in which the recognition model is trained and the game is built and integrated with it; (4) verification and internal testing, in which the assembled system is checked against its functional requirements; and (5) validation through gameplay evaluation, in which human participants play the finished game and the study measures whether its objectives are met. The phases are iterative rather than strictly linear: findings from testing and early play feed back into design and development before the final evaluation.

The data-related phase is executed almost entirely by code rather than by hand. There is no manual selection, editing, or relabeling of individual drawings, which makes that part of the pipeline deterministic and reproducible.

4.3.1 Gathering of Data

4.3.1.1 Data Extraction

For each of the fifty enabled classes named in the manifest, the acquisition program requests one pre-built file directly from the public Quick, Draw! storage location. Only the enabled classes are requested, so of the several hundred categories in the full release, exactly fifty are retrieved. The download is designed to be repeatable and safe: files already present are skipped rather than fetched again, and each file is only treated as final once it has transferred completely, so an interrupted download never leaves a partial file that looks complete.

4.3.1.2 Storage and Organization

The downloaded files are written to a single local data folder, one file per class, named after that class's source label, giving a flat and self-describing layout. The raw corpus (roughly 5.1 GB) is kept out of version control and treated as a re-obtainable artifact whose canonical copy remains the public source; anyone can regenerate the exact local dataset by re-running the acquisition program

against the manifest. The order and interpretation of the classes are governed centrally by the manifest, so the filenames themselves never have to encode class order.

4.3.1.3 Integrity Check

The source does not publish per-file checksums, so integrity is enforced through a set of automatic checks that together catch truncation, corruption, and structural mismatch. The number of files must match the number of enabled classes; a missing class is caught at training time with an explicit error. Because each file is finalized only after a complete transfer, any file present under its final name is guaranteed to be whole. Every file is validated as it loads, and a wrong-shaped or corrupted file fails loudly rather than feeding malformed data to the model. Finally, an automated contract check confirms that the exported label set contains exactly the fifty classes in the same order as the manifest, and that every ability referenced by a level is supplied by a class in that manifest, so that neither the label order used to read the model's output nor the content authored against the roster can drift from the intended specification.

4.3.2 Treatment of Data

Treatment converts the raw bitmaps into the exact form the classifier consumes and prepares balanced, non-overlapping partitions for training and evaluation. Because the source is a collection of images rather than a spreadsheet, several conventional data-cleaning steps either do not apply or take an image-specific form.

4.3.2.1 Data Cleaning

Individual drawings cannot contain missing cells, since each is a dense grid of pixel values, so there are no null entries to impute. "Missingness" exists only at the class level, as an absent file, and is handled as an integrity error rather than by imputation. A

different kind of empty input — a canvas with no ink — is rejected outright at inference and evaluation time, so blank submissions never reach the model. No near-duplicate removal is performed within a class; visually similar drawings are retained as legitimate within-class variation, and each class's influence is instead bounded by the per-class sampling cap. No statistical outlier removal is applied either: freehand drawings (sketches) are expected to vary widely, and unusual or mislabeled drawings are treated as label noise the model must tolerate. Robustness to position and scale is handled constructively instead, by cropping each image to its ink and re-centering it, and by exposing the model to rotated, shifted, and rescaled variants during training.

4.3.2.2 Data Transformation and Feature Engineering

The class names are text categories and must be represented as numbers for the network, so each class is assigned an index equal to its position in the manifest order, and these indices are consumed directly by the training loss. Pixel intensities are scaled from their original 0-to-255 range into a 0-to-1 range, and — crucially — this same scaling is applied identically at training time and at play time through the shared preprocessing routine, so the model always sees inputs on the same scale. Incoming images are cropped to their ink, resized to the model's small input frame while preserving aspect ratio, and, where necessary, inverted so that bright ink sits on a dark background, matching the convention the model was trained on. To make the model tolerant of how differently people draw, mild geometric augmentation (small rotations, shifts, and rescalings) is applied on the fly during training only; validation and test drawings are always seen unperturbed.

4.3.2.3 Data Splitting

Before any split is made, each class is capped to the same number of drawings, which converts the naturally imbalanced download into an equal, balanced pool. The balanced pool is then shuffled with a fixed random seed and partitioned 80/10/10 into training, validation, and test sets, yielding a large, near-equally represented test set. The fixed seed makes the split reproducible; the three sets are disjoint; and the test set is held out and touched only once, for the final precision, recall, and confusion-matrix figures, while the validation set is used during training to monitor for overfitting. K-fold cross-validation is not used: with a balanced pool this large, a single hold-out split gives stable estimates without the several-fold increase in training cost that k-fold would incur. Generalization is instead checked a second and stronger way — by evaluating the trained model on the independent TU-Berlin corpus, drawn by different people under a different protocol and never seen during training, which measures performance under genuine distribution shift rather than on held-out samples of the same distribution.

4.3.3 System Development and Integration

With the model trained and exported, development moves to building the game and integrating recognition into it. The game runtime is authored in the engine described in Section 4.8, with the drawing canvas, level structure, and the draw-to-ability mechanic implemented against the fixed fifty-class vocabulary so that the entities the game can spawn are exactly the classes the model can recognize. The exported model is served by the local recognition component, which the game launches and health-checks at startup; the game submits a completed drawing to this component and receives back a predicted class, a confidence value, and a margin. When confidence or margin falls short of threshold, the drawing

is returned to the player to redraw rather than acted upon; when both clear threshold, the recognized class is looked up in the ability table and the player's own strokes are animated as the resulting in-game entity. Development proceeds iteratively: components are prototyped, integrated, and revised as recognition behavior and gameplay feel are observed together, since the quality of the experience depends on the two working as one rather than on either in isolation.

4.3.4 Verification, Validation, and Gameplay Evaluation

Verification asks whether OBRA was built correctly against its functional requirements: whether drawings are captured, recognized, mapped to abilities, and animated as specified, whether the redraw threshold behaves, and whether levels can be completed. This is carried out by the researchers through internal and functional testing before any external participant is involved.

Validation asks the harder question of whether the correctly built system actually achieves its objectives for real players, and this is where the human respondents of Section 4.1.2 take part. Each participant will play a defined set of OBRA levels under observation. While they play, the game will automatically record the telemetry described in Section 4.2.1: level completion rates, time-on-level, the model's per-class precision and recall in live play, its confidence and redraw behavior, and the end-to-end recognition response time. Immediately after the session, each participant will complete the post-session instruments: the SUS (in full) for perceived usability, the UEQ-S (in full) for pragmatic and hedonic quality, the IEQ-SF (in full) for immersion, the selected GEQ components for the wider player experience, and two validated subscales — the Godspeed Animacy subscale for the believability of the animated drawing and the QUESI Perceived Familiarity subscale for the fit between the granted ability and their prior expectations.

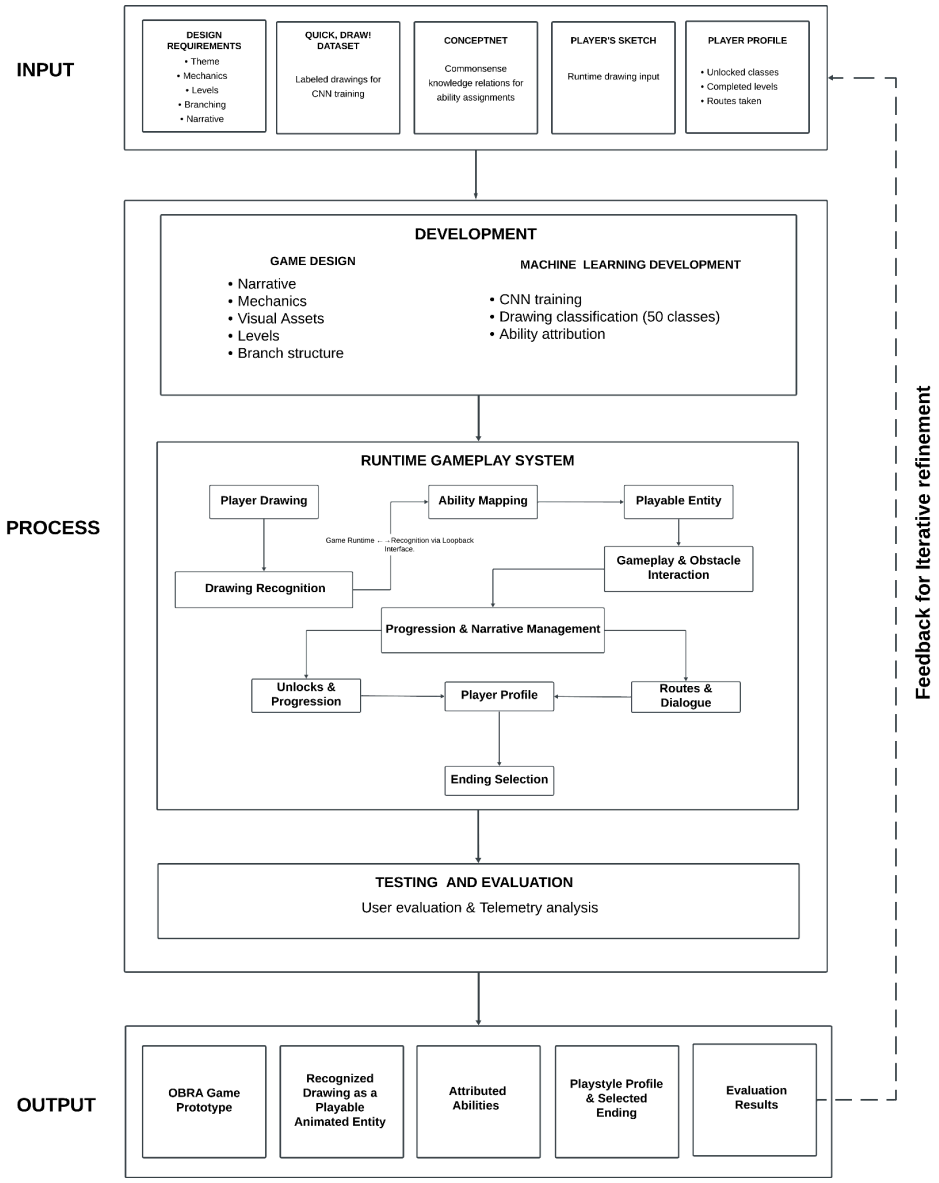
Analysis will then bring the subjective and objective evidence together. The questionnaire scores will be summarized against their established benchmarks — for instance, the SUS score against the above-average threshold — and read alongside the telemetry, so that a claim such as "the drawing mechanic is usable" rests on both the players' ratings and their observed completion behavior. Recognition performance in real play will be reported per class for the classes solicited by name with a confusion matrix and never as a single accuracy figure, with the primitive classes reported individually. Where results fall short of an objective, the shortfall will be traced to a specific cause — a usability obstacle, a low-recall class, or a latency problem — and used to state clearly which objectives OBRA meets and which remain open. Because the model is trained on secondary data and no player contributes to building the system, this evaluation phase is the study's phase in which human respondents evaluate the built systems, and it is the phase that actually validates whether the research objectives have been achieved.

4.4 Concept

The conceptual framework of OBRA is based on the Input-Process-Output model. The model is appropriate to this study because OBRA's operation is a transformation with a clearly bounded beginning and end: the player supplies a freehand drawing, the system converts it into a classification and an attributed ability, and the result is returned to the game world as a playable entity. Each stage is separable and independently measurable, which is what allows the study to evaluate recognition accuracy, response time, and player experience as distinct outcomes rather than as a single undifferentiated result. Figure 7 presents the framework.

Figure 7

Conceptual Framework of OBRA



The ***input layer*** consists of the resources required to design, train, and operate the system. It comprises the design requirements of the game, namely its theme, mechanics, level and branch structure, and narrative content; the Quick, Draw! dataset, which provides labeled sketches for model training; ConceptNet, which is the basis for grounding abilities through commonsense knowledge relations; the player's drawing, which serves as the real-time input during gameplay; and the persisted player profile, which supplies the classes

already unlocked, the levels already completed, and the routes already taken, and which is therefore an input to every session after the first.

The **process layer** shows how these inputs are made into a usable system. Game design determines the narrative, mechanics, visual assets, levels, and the branch structure of each level. Machine learning development trains the convolutional neural network to classify drawings into the fifty enabled classes. Progression and narrative management gates obstacles against the classes the player has unlocked, resolves the route chosen at each dialogue node, selects dialogue by weighted priority, and maintains the profile from which the ending will be computed. All these components are integrated through the loopback interface between the game runtime and the recognition component. Testing is conducted through user evaluation and telemetry analysis to evaluate how effectively the system performs.

Lastly, the ***output layer*** comprises the final outputs and evaluation results produced by the system. These include the game prototype, the recognized drawing as a playable animated entity, the grounded ability assigned to each recognized class, the playstyle profile and the ending it selects, and the evaluation results gathered from questionnaires and telemetry. These outputs form the criteria for evaluating whether the system recognizes player drawings accurately, assigns appropriate abilities, incorporates those abilities into gameplay, and provides a usable and engaging player experience.

The framework also includes a feedback loop, in which the results from user testing and telemetry analysis are analyzed in order to improve the system. Recognition accuracy, completion rates, time-on-level within route, redraw behavior, and player feedback are used to determine whether recognition, ability mapping, progression, and the game's mechanics function as intended, and the findings inform the design requirements, model performance, level and branch design, and overall gameplay experience of the next iteration.

4.5 Analysis and Design

This section presents the technical blueprint of the OBRA software. It first states the functional and non-functional requirements the system must satisfy, then describes the overall system architecture and how its parts interact, and finally details the design through a set of standard diagrams (structural, use-case, data-flow, and activity) together with the initial user-interface designs. All descriptions are drawn from the current system design, with the entity manifest at game/config/entities.json treated as the single source of truth for the roster and its mappings.

4.5.1 Requirement Analysis

The requirements below were derived from the behavior of the two parts that make up the application: the game runtime, built in Godot 4 with GDScript, and the local recognition component, a Python service running the trained model under ONNX Runtime. The two run together as a single application on one machine.

Functional Requirements

1. **Freehand drawing capture.** The system shall present an in-game drawing panel on which the player draws freehand strokes, capturing both the resulting image and the ordered strokes that produced it.
2. **Sketch recognition.** The system shall classify a submitted drawing into one of fifty classes and return the recognized entity together with a confidence score and the margin between the top two candidates.
3. **Ambiguity gating.** The system shall reject a prediction and prompt the player to redraw when the top confidence falls below 0.60 or the margin between the top two candidates falls below 0.15.

4. **Creature transformation.** The system shall spawn or replace a playable creature skinned with the player's own strokes, at the player's current position.
5. **Movement-driven animation.** The system shall animate the drawn strokes by physics simulation rather than by playing prepared frames: the strokes are segmented into rigid bodies linked by joints, and motion is produced by driving those joints toward target poses with proportional-derivative control, as specified in Section 3.2.11. Gaits (walk, hop, fly, swim, climb, slither) shall be produced only in response to movement and input, and the joints shall relax toward their drawn rest pose when motion stops, rather than playing a perpetual preset loop.
6. **Drawable objects.** The system shall spawn recognized objects as physics bodies whose collision shape is rebuilt from the submitted drawing, and shall place construction, structure, and vehicle objects through a preview-then-commit interaction.
7. **Ink economy.** The system shall enforce a limited ink budget of six units. Creature transformation at a checkpoint is free. A **tool** costs one unit of ink on its first successful recognition, is then held in the toolbelt, and is thereafter selectable and reusable at no ink cost and with no redraw. A **placeable object** — structure, vehicle, traversal prop, elemental, or primitive — costs one unit of ink on **each** placement and is not retained; to place it again the player redraws it. Whether a class is a tool or a placeable is declared in the entity manifest rather than in gameplay code. Tool acquisition persists across levels and sessions, since a tool acquired later opens a concept-gated area earlier.

8. **Checkpoints and reset.** The system shall save and restore a full snapshot of ink, toolbelt, placed objects, and puzzle state on reset.
9. **Toolbelt and interaction.** The system shall maintain a toolbelt (key, axe, umbrella, flashlight), let the player cycle and select tools, and resolve tool and fixture interactions in the world.
10. **Local recognizer lifecycle management.** The game shall verify that the local recognition component is reachable through a health check and shall start it automatically as a co-located component, surfacing its readiness to the player.
11. **Roster-governed vocabulary.** The system shall load and validate the class roster at startup, refusing entries whose scenes or profiles are missing or malformed, so that the trained model's label order and the game's set of spawnable entities always agree.

Non-Functional Requirements

1. **Interactive latency.** The end-to-end response time — from capturing the drawing, through the local hand-off to the recognition component, preprocessing, inference, and the response, to spawning the entity — shall remain low enough for interactive play, and both parts shall expose timing instrumentation so this can be measured.
2. **Single-machine monolithic deployment.** The application shall be built and run as one unit on a single Windows or macOS machine, with the pretrained recognizer running locally and started by the game itself; no remote server, external network, or second device is required at runtime.
3. **Correctness and validation.** Invalid input shall be rejected with meaningful errors (an undecodable image, or an empty

canvas), and the model's output size shall be checked against the number of labels at startup.

4. **Training–runtime parity.** The same preprocessing shall be used at training time and at runtime, so the model always sees inputs prepared in exactly the same way.
5. **Maintainability and single source of truth.** The roster, scene bindings, and entity profiles shall live only in the class roster, so that adding or changing a class does not require editing several components.
6. **Robustness and graceful degradation.** When a silhouette cannot be built from the drawing, the system shall fall back to an authored collision shape rather than fail.
7. **Privacy.** The system shall process only anonymous drawings, keep all data within the host so that nothing leaves the machine, and store no personally identifying information.

4.5.2 System Architecture

OBRA is a monolithic desktop application: it is built, deployed, and run as a single unit on a single machine. Sketch recognition is performed by a pretrained convolutional neural network that runs locally inside the application as its recognition component; the model is trained and evaluated against an independent external test set offline in advance (see *Treatment of Data* and *Development Approach*), and at runtime the application only loads and runs it, never trains it. Internally the application is modular, comprising a Godot 4 game runtime and a co-located Python inference component, but the two run together on one machine and communicate over the loopback interface, so the system is monolithic in deployment rather than a distributed client–server product. There is no conventional database management system; persistent state takes the file-based and in-memory forms described below.

Game runtime. The Godot 4 project, developed using GDScript, captures the player's strokes, renders the world, runs all gameplay logic

and physics, and performs all animation locally. It is responsible for input capture, for handling the local recognition request, for starting and health-checking the recognition component, for loading the roster and instantiating the correct scene for each recognized entity, and for holding the gameplay state — ink, toolbelt, checkpoints, and object placement — that must survive when one playable actor is replaced by another.

Recognition component. A small local service, launched and supervised by the game itself, exposes two endpoints on the loopback interface: one for a health-and-metadata check and one for classification. On a classification request it decodes the submitted image, preprocesses it to the model's fixed input size, runs the pretrained network under ONNX Runtime, applies a softmax, maps the top class to an in-game entity through the shared roster, and returns the recognized entity together with its confidence, the runner-up margin, and the entity's scene and behavior profiles. It holds no session and stores nothing; it is an internal component of the application, not a remote server.

Persistence (no database). The system deliberately uses no database. Player progress is instead persisted locally through a single structured save file, the *player profile*, stored in the host operating system's user-data directory. The profile is a schema-versioned record that captures the player's completed levels, unlocked levels, the distinct recognizer-accepted classes the player has drawn, submission and decline counts, and route and collectible state. The file is written atomically — the updated profile is first written to a temporary file and then swapped into place — so an interruption during a save, such as a power loss or crash, cannot corrupt existing progress. On launch, a profile that is absent, empty, unreadable, or of an incompatible version is treated as a new profile rather than as a fatal error. This file-based save fulfills the requirement of persisting progression between play sessions without the operational overhead of a database server, which is unnecessary for a single-player, locally run prototype. It is kept separate from the anonymous

per-session gameplay telemetry, which is collected solely for research analysis and does not form part of player progression.

Why a local recognition component at all. Godot has no built-in machine-learning runtime, while the training ecosystem (PyTorch, ONNX) is in Python. Running the pretrained model as a small local component bridges the Python inference stack and the GDScript game without leaving the machine: the boundary between them is a loopback call, an internal integration mechanism rather than a network deployment. The recognizer is started automatically by the game and bound to the loopback interface, so the whole system launches and runs as one application on one machine.

Figure 8

System Architecture (component and deployment view)

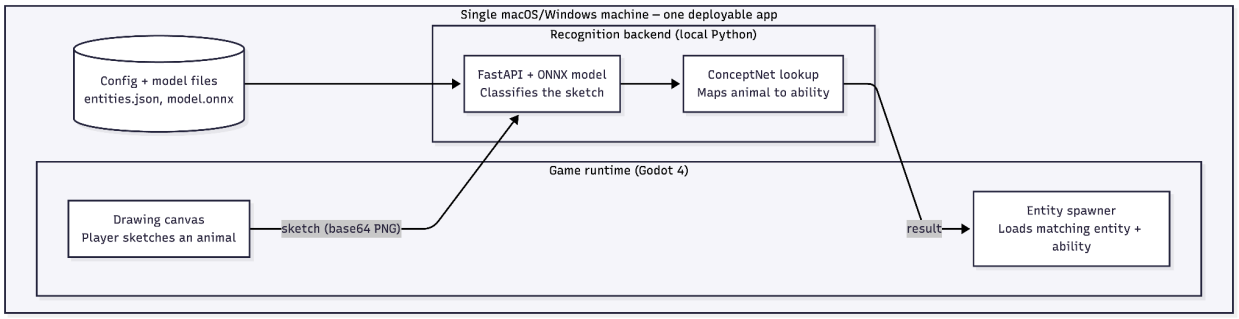


Figure 8 shows these parts together as one deployable application on a single machine, with the loopback boundary drawn inside it rather than across a network.

Figure 9

Request and Response Cycle (sequence view)

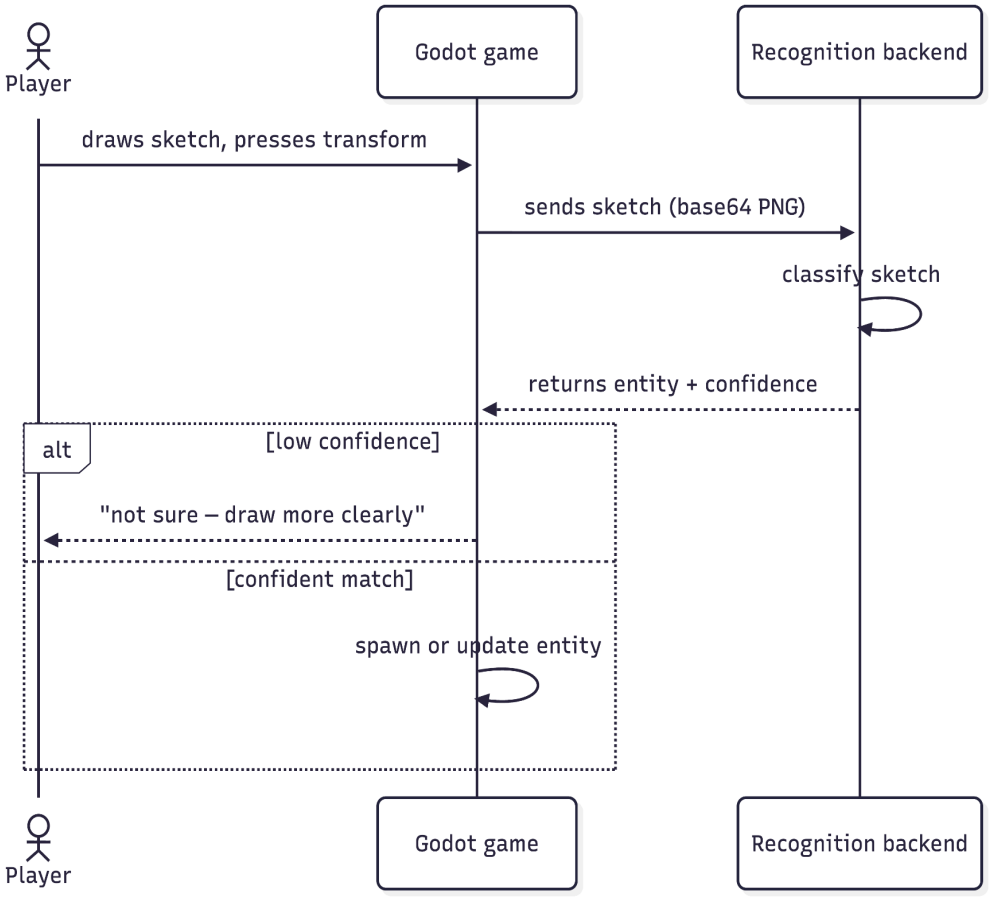


Figure 9 follows a single submission through the three participants in the exchange. The player draws and presses Transform; the game encodes the canvas as a Base64 PNG and sends it over loopback; the recognition backend classifies it and returns the entity together with its confidence. The response then splits. On a low-confidence result the game keeps the panel open and asks the player to draw more clearly, spending no ink and spawning nothing. On a confident match the game spawns or updates the entity in place. The two branches are shown as alternatives of the same step because the decision belongs to the game rather than to the model, which reports a number and nothing more.

Figure 10

Client and Backend Development

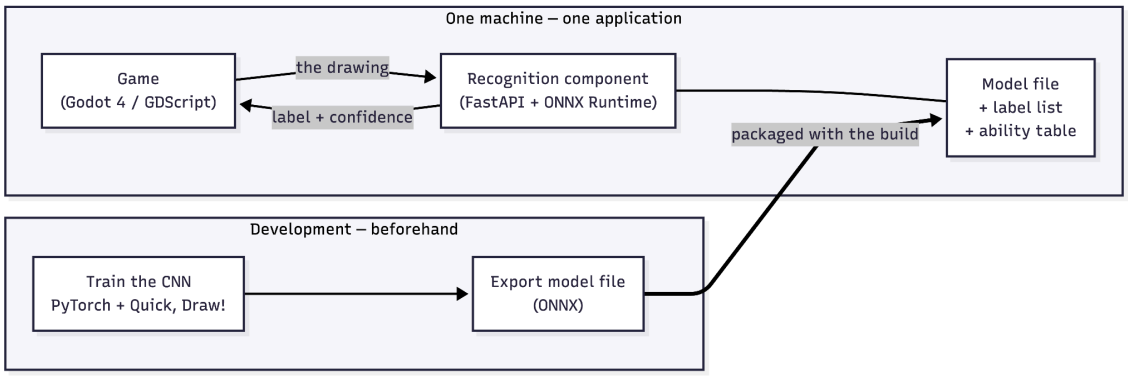


Figure 10 shows how a Python inference component and a GDScript game come to live inside one application. The model is trained beforehand and only its exported file crosses into the running system. At play time the game starts the recognition component beside itself and speaks to it across the loopback interface, so nothing leaves the machine.

4.5.3 Detailed Design Diagrams

4.5.3.1 Structural (Class) Diagram

The client is organized around two families of entities and a set of coordinating components. Playable creatures share a common base that owns the skinning of the player's strokes, the silhouette collision, and the forwarding of motion to the runtime rig; on top of this base, each of the twenty creatures specializes its own locomotion. Drawable objects share a separate base with a physics lifecycle and rebuild their collision shape from the submitted drawing. A level coordinator wires together the drawing panel, the roster, the recognition supervisor, and the state manager, while the state manager holds all gameplay state that must survive the replacement of one actor by another.

Figure 11

Structural / Class Diagram (key types)

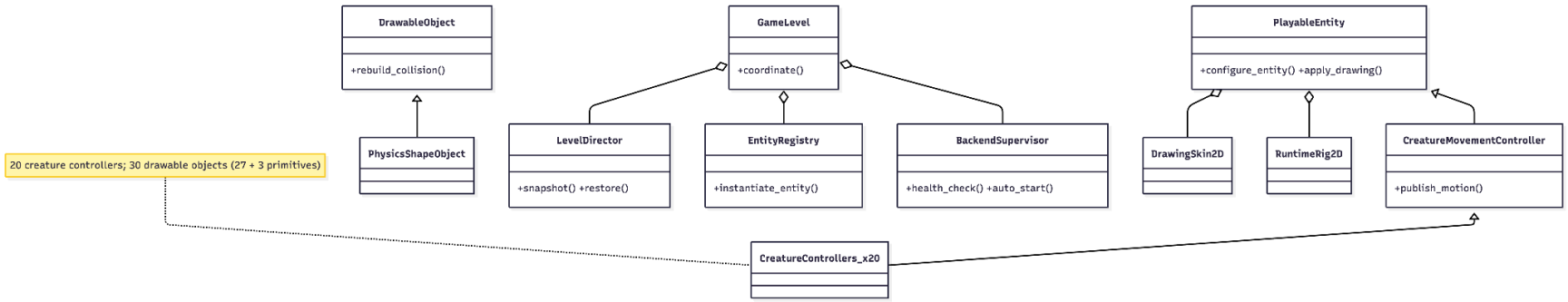


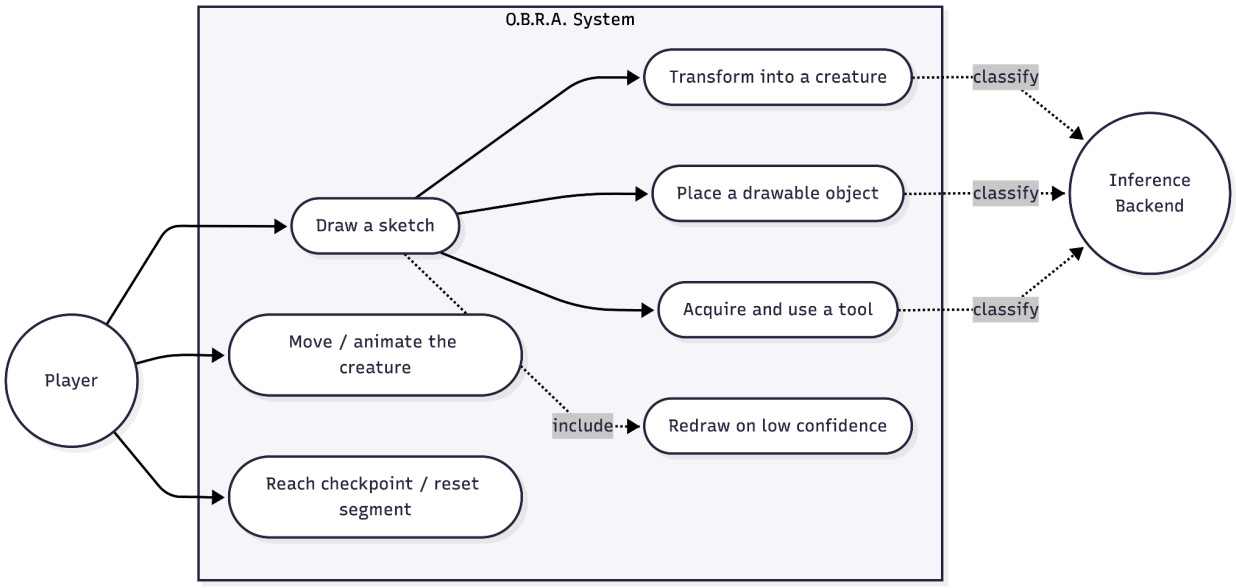
Figure 11 shows the key types only. Open diamonds denote aggregation; the hollow triangle denotes specialisation. GameLevel coordinates the state-holding and supervising components, PlayableEntity assembles the components that skin, rig, and animate the player's own strokes, and PhysicsShapeObject rebuilds its collision shape from the submitted drawing. The annotation records the roster the design supports: twenty creature controllers and thirty drawable objects, being twenty-seven objects and three primitives.

4.5.3.2 Use Case Diagram

The single human actor is the Player; the recognition component is a supporting (secondary) actor that fulfills every recognition-dependent use case. Drawing is the entry point, which branches into transforming into a creature, placing an object, or acquiring a tool, and includes the redraw case when a prediction is too uncertain. Figure 12 shows the actors and cases together.

Figure 12

Use Case Diagram

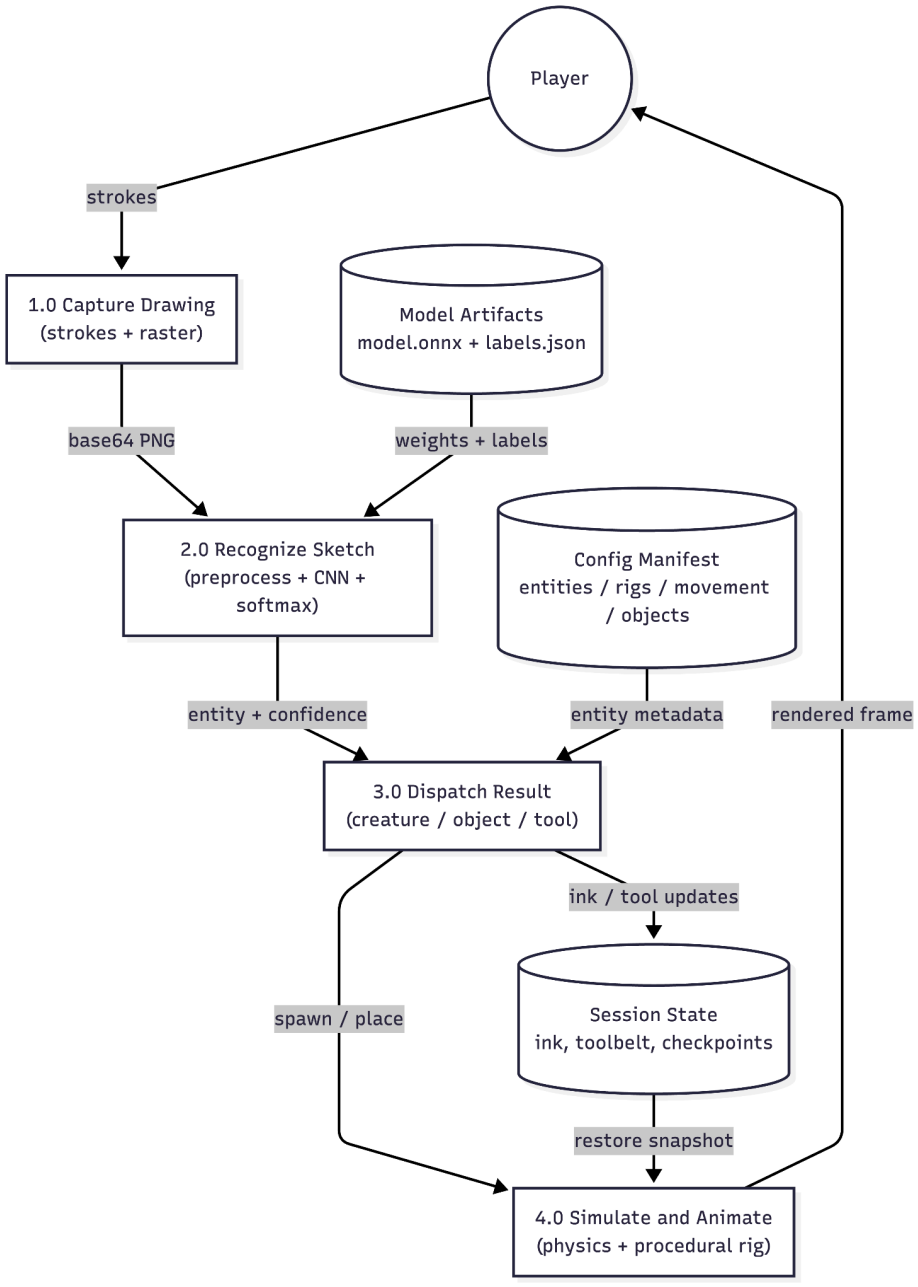


4.5.3.3 Data Flow Diagram

At level 1, four processes move data from the player's strokes to a rendered frame. Recognition reads the model artifacts; dispatch reads the class roster; and simulation reads and writes the in-session gameplay state.

Figure 13

Data Flow Diagram (Level 1)

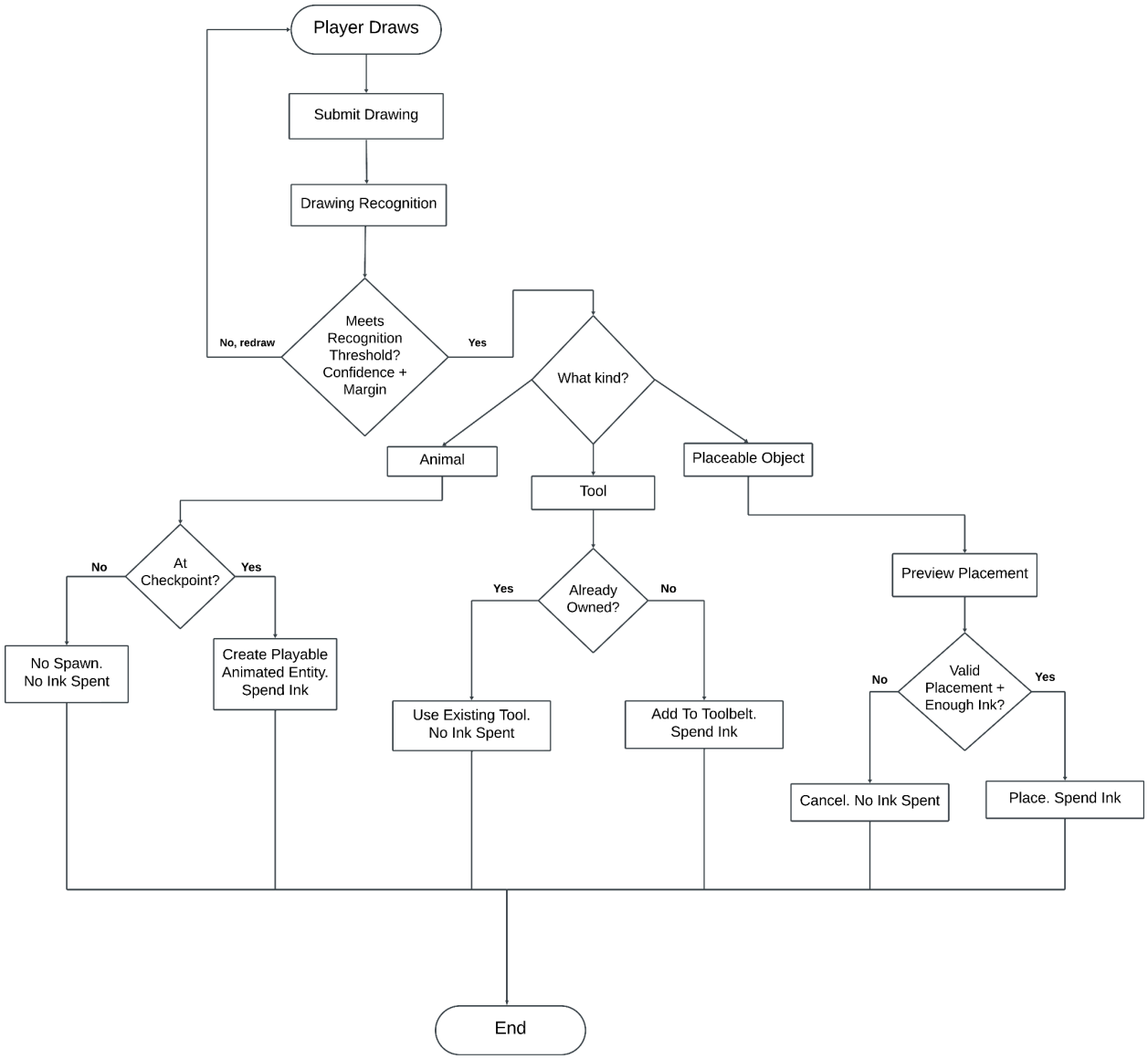


4.5.3.4 Activity Diagram

The activity traces one full interaction, from opening the drawing panel to the outcome, showing the confidence gate and the three dispatch branches: creature, tool, and placeable object.

Figure 14

Activity Diagram (draw to outcome)



4.5.4 Game Design

4.5.4.1 Game Overview

Logline

Step into the magical paintings of your missing grandmother. Using her enchanted paintbrush, draw objects and creatures that come to life to solve puzzles. Navigate branching memories, revisit past paintings to uncover hidden secrets, and unravel the mystery of her disappearance.

Core Concept

OBRA merges deep learning with non-linear gameplay. The player draws on a canvas; a CNN recognizes the sketch (from an expanded 50-class vocabulary); and a static lookup table, resolved from ConceptNet before release, supplies its real-world property. Driven by dynamic dialogue choices, the player navigates a branching story where they can take aggressive, empathetic, or pragmatic routes—some of which bypass entire sections of the game.

4.5.4.2 Core Gameplay Mechanics

The Draw-to-Ability Loop

1. Encounter: The player faces an obstacle.
2. Draw & Recognize: The player draws; the CNN classifies the drawing and returns a class, a confidence, and the margin between the top two candidates. The result is accepted only if confidence ≥ 0.60 and margin ≥ 0.15; failing either, the game declines and asks for a redraw, and no ink is spent.

Adaptive Difficulty by Acceptance Threshold

The dual gate of 0.60 confidence and 0.15 margin is a correctness floor and does not change. Above it sits a second, per-level acceptance threshold that rises as the player advances, so that a drawing which would satisfy an

early level may be asked for again in a later one. The early levels accept a recognition at the floor; the middle levels raise the required confidence; the final level requires the highest. The threshold is held in the level definition as a configuration value, not in code, so it can be re-tuned against the confusion matrix without a rebuild.

This is a difficulty parameter and is documented as such. It is not a measure of drawing quality and is never reported as one: a higher threshold means the game is asking for a clearer drawing, not that the player has drawn badly. Recognition metrics are reported against the fixed floor throughout, so that the per-level threshold cannot inflate or deflate any figure in Section 4.10.3.

To guarantee that no level becomes unpassable, the elevated threshold decays after a bounded number of consecutive declines on the same obstacle, falling back toward the floor by a fixed step per attempt until it reaches the floor, where it remains. A player who cannot satisfy the raised threshold therefore always converges on a level that is passable at the floor, in keeping with REQ-4.3-9. Declined attempts continue to cost no ink and no progress at every threshold.

3. Attribute & Execute: The static ability table, resolved from ConceptNet offline, before the game ships, and committed, supplies the ability for the recognized class. No knowledge-graph query is made while the game is running. The player Summons a tool (e.g., Axe, Ladder, Umbrella) or Morphs into a creature (e.g., Spider, Bird, Turtle).

The Studio Hub & Backtracking

- Lola's Studio (The Hub): The game is anchored in Lola's real-world art studio. The paintings on the easels act as level portals.
- Backtracking: Some areas inside early paintings are blocked by conceptual obstacles that the player doesn't know how to solve until Lolo teaches them a concept in a later level (e.g., learning to draw a Flashlight in a later level lets the player return to Level 1 and light a pitch-black cave to claim its Hidden Flower).
- Replayability: Players can re-enter any unlocked painting to try a different narrative path or find missing Hidden Flowers.

Affordance-Based Obstacle

An obstacle does not name the class it wants. It states the affordance it requires, and accepts any recognised class whose attributed ability satisfies that affordance. A ravine requires the ability to span or to cross; a bridge satisfies it, and so does a ladder laid flat, a tree felled across the gap, or a hot air balloon that carries the player over. A dark passage requires illumination, which a flashlight, a campfire, and the sun all supply. The obstacle is authored against the affordance, the roster supplies every class that satisfies it, and the level is completable by any of them.

This follows directly from the attribution design. Because abilities for the forty-eight grounded classes derive from documented ConceptNet assertions, with penguin and mushroom documented as hand-authored exceptions, the set of classes satisfying a given affordance is computed from the ability table rather than enumerated by a designer, and the load-time roster check in Section 4.10.2 verifies that at least one class in the roster

satisfies every affordance on a level's critical path. The player therefore chooses what to draw rather than guessing what the designer had in mind, which is the difference between a puzzle about the world and a puzzle about the developer.

Guided Clues

A player who stalls at an obstacle is offered a clue rather than an answer. The clue system engages after a bounded number of declined or unsuccessful attempts at the same obstacle and escalates in three stages: the first restates the affordance in plainer language, the second names the category of thing that would satisfy it, and the third names one class that does. Each stage is offered rather than imposed, and the player may dismiss it and continue unaided. The stage reached is recorded in telemetry, so that a level completed only after a third-stage clue is distinguishable in the results from one completed unaided — a distinction that matters to the completion figures reported under the fifth evaluation aspect.

4.5.4.3 The Tri-Path Narrative & Skipping System

Instead of a linear hallway, levels are designed with "Dialogue Nodes." When the player reaches a critical juncture, Lolo pauses time to talk. The player's dialogue choice physically alters the level layout, opening one path and permanently locking/skipping the others for that playthrough.

The Three Paths

- Path A: The Artist / Empathy Route
    - *Trigger:* Choosing compassionate, lore-seeking dialogue.
    - *Effect:* Opens the longest, most puzzle-heavy route. Unlocks deep lore cutscenes and Hidden Flowers.

- Path B: The Pragmatist / Skip Route
    - *Trigger:* Choosing impatient, objective-focused dialogue ("Let's just get out of here").
    - *Effect:* Skips scenes. This opens a shortcut that bypasses major puzzle rooms and narrative cutscenes entirely. It guarantees a fast completion time but forfeits lore and collectibles.
- Path C: The Protector / Confrontation Route
    - *Trigger:* Choosing aggressive, protective dialogue ("I'll destroy whatever is in our way").
    - *Effect:* Routes the player into hazard-heavy or combat-lite zones solved by destruction (Axe, Sword, Cannon, Anvil) rather than traversal.

Background Telemetry

While the player actively chooses paths, the game runtime's telemetry subsystem records:

- Class Diversity Score (0 to 50): Number of distinct roster classes successfully recognised and accepted at least once, from 0 to 50.
- Redraw Rate: How often the CNN rejects their drawings.

4.5.4.4 Priority Queue Dialogue & Replay System

To ensure high replay value and prevent repetitive interactions when players backtrack or replay levels, OBRA uses a Priority Queue Data Structure for its narrative delivery.

Ambient & Hint Dialogue (The Shuffle Bag)

Whenever Lolo comments on the environment or the player's drawings, the game pulls from a priority queue of dialogue strings.

- High Priority: Unseen dialogue lines (Base weight +100).

- Low Priority: Recently played lines (Weight drops to 0, slowly recovers over time).
- *Result:* A player repeating Level 1 three times will hear Lolo tell three different stories about the rice terraces.

Path Priority (Nudging the Player)

The game uses a global save file to track which paths the player took previously, then applies priority to the Dialogue Prompts.

- *Example:* If a player always chooses the "Pragmatist/Skip" route, on their next replay, Lolo will actively suggest the unexplored path: *"We rushed past this last time... shouldn't we stay and fix the bridge today?"*

4.5.4.5 Playstyle Profiles & Multiple Endings

At the climax at Mayon Volcano, the game resolves the ending from the routes the player took at the four dialogue nodes in Levels 1 through 4, combined with their telemetry data. Level 5 contains no dialogue node — its layout is *generated from* the dominant route rather than adding another choice to it. Because there are four nodes in total, no two routes can each be taken three or more times, so Endings A, B, and C are mutually exclusive by construction.

- Ending A: The Masterpiece (Perfect Ending) (Requires Empathy Path 3+ times, all Flowers, and High Class Diversity: > 25 unique classes drawn). Lola is saved, Lolo's spirit is restored, player inherits the brush.
- Ending B: A Rushed Sketch (Speedrunner Ending) (Requires Pragmatist Path 3+ times, and Low Class Diversity: < 10 unique classes drawn). Lola is saved, but Lolo's spirit fails to manifest. Paintings burn.

- Ending C: The Protector’s Canvas (Defensive Ending) (Requires Protector Path 3+ times). Lolo sacrifices his spectral form to shield the MC and Lola.

- Ending D: The Persevering Spirit (Meta-Ending) — the default ending. Reached whenever no route ending's conditions are met: that is, when no single route was taken three or more times, or when a route reached three but its diversity or collectible condition was not satisfied. Lola laughs warmly at the MC's messy sketchbook, validating the player's real-world struggle with the drawing AI and remaining proud of the effort. Precedence, highest first: C → A → B → D. Because D is the fallback rather than a threshold, the ending rules are total: every reachable combination of routes and telemetry resolves to exactly one ending.

4.5.4.6 Expanded Branching Level Design

Milestone Rewards and the Challenge Curve

Progression is marked by rewards granted at fixed points rather than at random, so that a player always knows what the next milestone is and what completing it will give. Four milestone types are defined. Completing a level unlocks the next painting in Lola's studio and grants a permanent addition to the toolbelt, a tool being paid for once and kept thereafter. Recovering all Hidden Flowers within a level unlocks that level's memory cutscene, which carries narrative rather than mechanical reward. Reaching a stated class-diversity count unlocks entries in the studio's sketchbook, which records every class the player has successfully drawn. Completing the fourth dialogue node resolves the playstyle profile that determines the ending, as described in Section 4.5.4.5.

Difficulty rises across the five levels along three independent axes, so that later levels are harder in kind and not only in quantity.

Traversal complexity increases from the single vertical climb of Bahay Kubo to the timed, multi-stage ascent of Mayon. The acceptance threshold rises level by level as set out in Section 4.5.4.2. And the affordances demanded broaden: early obstacles require a single common affordance that many roster classes satisfy, while later obstacles require affordances satisfied by fewer classes, or two affordances in sequence. The ink budget of six units is unchanged throughout, so the same resource covers a widening demand, which is what makes the later levels a test of what the player has learned to draw rather than of how much they can draw.

Level 1: Bahay Kubo (The Awakening)

- Atmosphere: Warm, golden-hour sunlight. Lush green Banaue Rice Terraces. The music is a slow, acoustic *bandurria*.
- Mechanic Focus: Basic vertical traversal and getting comfortable with the CNN drawing interface.
- The Dialogue Node: The Broken Hanging Bridge
    - *Context:* A massive chasm separates the terraces. The hanging bridge is rotted away.
    - *Choice 1 (Empathy): "Let's restore her work."* -> Path A (The Puzzle Route): The player must draw a series of Ladders and Bridge. Unlocks a memory cutscene showing Lola painting this exact bridge.
    - *Choice 2 (Pragmatist): "Let's just climb down and go around."* -> Path B (The Skip): Skips the bridge puzzle and cutscene entirely. The player morphs into a *Spider*, drops down the gorge wall, and crawls straight to the level exit.

- *Choice 3 (Protector): "I'll carve us a new path."* -> Path C (The Hazard Route): The player summons an *Axe* to chop a massive dead tree, creating a makeshift bridge that triggers a collapsing-platform escape sequence.

- Backtracking Secret: At the bottom of the gorge is a pitch-black cave. The player must return here from the Hub later, after learning to draw a *Flashlight* (Concept: Illumination), to melt away magical shadows and claim Hidden Flower 1.

Level 2: Pista (The Festival of Colors)

- Atmosphere: Lively afternoon with *banderitas* (bunting) and crowded streets. Sinulog festival drumbeats that dynamically change based on movement.

- Mechanic Focus: Navigating tight spaces and moving platforms (parade floats).

- The Dialogue Node: The Blocked Plaza

  - *Context:* A massive, broken parade float is blocking the only street forward. A crowd is panicking.

  - *Choice 1 (Empathy): "Let's help them move it."* -> Path A: Player draws a *Horse* or *Wheel* to fix the float. They join the parade, unlocking a memory of Lolo and Lola's first date in the crowd.

  - *Choice 2 (Pragmatist): "I'm going over them."* -> Path B (The Skip): Player morphs into a *Bird* or summons a *Hot Air Balloon*. They fly high above the rooftops,

bypassing the plaza, the crowd, and the puzzle entirely.

- *Choice 3 (Protector): "I'll clear the back alleys."* -> Path C: Player turns away from the crowd and goes through the slums. Requires summoning *Swords* or *Axes* to smash through barricaded doors and territorial stray dogs.

- Backtracking Secret: A flooded drainage canal runs under the plaza. The player must return later and morph into a *Fish* (learned in Level 3) to swim through the narrow pipes to find Hidden Flower 2.

Level 3: Dagat (The Open Sea)

- Atmosphere: A stormy, violent ocean inspired by Coron's jagged cliffs. The player starts on a fragile *Bangka* (outrigger boat). Rain and thunder dominate the audio.

- Mechanic Focus: Water physics, buoyancy, and aquatic morphs.

- The Dialogue Node: The Maelstrom

  - *Context:* A massive whirlpool, shadowed by the silhouette of a *Bakunawa* (sea serpent), blocks the strait.

  - *Choice 1 (Empathy): "It's frightened. Let's guide it."* -> Path A: Player morphs into a fish to soothe the beast, triggering a beautiful underwater ballet sequence.

  - *Choice 2 (Pragmatist): "Dive under it."* -> Path B (The Skip): Player draws a *Submarine* or *morphs into a*

crab to descend to the ocean floor and bypass the whirlpool.

- *Choice 3 (Protector): "Brace for impact!"* -> Path C: The player remains aboard the *bangka* and repeatedly summons a *Bucket* to remove incoming water while avoiding debris and maintaining the vessel's balance through the whirlpool.

- Backtracking Secret: High on a limestone cliff is a nesting ground. The player must combine the *Spider* climb and *Bird* flight (advanced stamina mechanics) to reach Hidden Flower 3.

Level 4: Dilim (The Forest)

- Atmosphere: Eerie, pitch-black jungle inspired by Mt. Makiling. Glowing bioluminescent flora. Sound design is claustrophobic, with snapping twigs and heavy breathing.

- Mechanic Focus: Illumination, evasion, and timing under pressure.

- The Dialogue Node: The Aswang Confrontation

  - *Context:* The Aswang (a manifestation of Lola's grief) corners the MC and Lolo at the edge of a cliff.

  - *Choice 1 (Artist/Empathy): "Wait... look at its eyes."* -> Path A: Player draws a *FlashLight*. The Aswang shrinks, revealing it is just a manifestation of Lola's sadness. Unlocks the deepest lore fragment of the game.

    - *Choice 2 (Pragmatist): "Jump!"* -> Path B (The Skip): Player morphs into a *Bird* and dives off the cliff into the fog. The boss encounter is skipped completely. The fastest route, but denies closure.
    - *Choice 3 (Protector): "Stand behind me, Lolo!"* -> Path C: Player stays in the arena. A combat-lite sequence begins where the player uses a *Sword or Boomerang* to interrupt the creature's attacks and summons a *Campfire* to keep the arena illuminated until the manifestation retreats.
- Backtracking Secret: A heavy stone door at the start of the forest requires a *Key*. If the player favored animal morphs and never mastered object summoning, they must practice drawing a high-confidence Key and return to unlock Hidden Flower 4.

Level 5: Mayon (The Summit & The Climax)

- Atmosphere: Apocalyptic but beautiful. The volcano is erupting, but instead of lava, it is spewing vibrant, wet oil paint (crimson, gold, cerulean) into the sky.
- Mechanic Focus: A final exam of the player's drawing speed and CNN mastery.
- The Final Layout (Telemetry-Generated):
    - There are no dialogue choices here. The physical layout of Level 5 is determined entirely by the paths the player chose in Levels 1-4.
    - *If Pragmatist (mostly skipped):* The path up the volcano is a straight, violently crumbling slope. The

player must run and draw instantly to survive the speedrun.

- *If Artist (mostly empathy):* The path is a series of broken, floating frescoes. The player must pause, observe, and draw specific, diverse objects to "repair" the path to the summit.
- *If Protector (mostly confrontation):* the ascent is a hazard gauntlet through erupting debris. Falling rock is deflected with an Umbrella or absorbed as a Sea turtle, and collapsed obstacles are cleared with an Axe or Sword.

- The Climax: Upon reaching Lola at the crater, the cinematic ending (A, B, C, D) triggers based on the final telemetry profile. Hidden Flower 5 is sitting plainly beside Lola's easel—a free reward for those who made it to the end.

4.5.4.7 Audio & Visuals

- Art Style: 2D hand-painted environments mimicking the 8-bit pixel art style. The player's drawings appear as raw, black with white outline, that starkly contrast with the painted world, emphasizing that the player is an "intruder" altering the canvas.
- Dynamic Audio Engine: Traditional Filipino instruments (Kulintang, Bandurria) mixed with melancholic piano and strings. The music dynamically strips away layers if the player is speedrunning (Pragmatist Path), and adds rich, emotional strings if they are exploring and engaging empathetically (Artist Path).

Figure 15

Game Design Flow

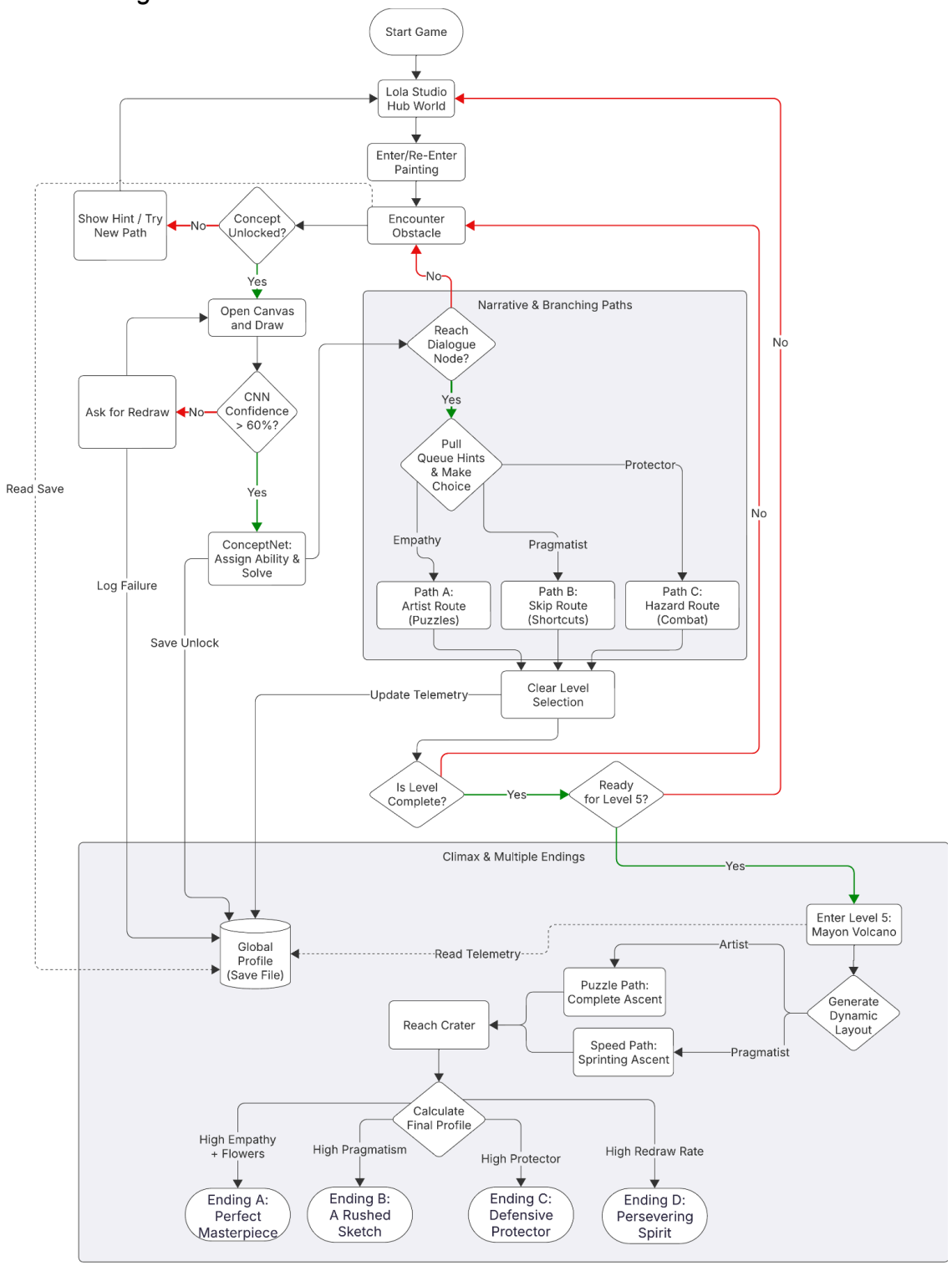


Figure 15 traces the whole game as a single flow, from launch through the hub to the endings. The player enters a painting and meets an obstacle, and the first decision is whether the concept it requires has been taught: if it has not, the drawing panel does not open and the game points the player toward another path. If it has, the canvas opens and the drawing is classified. A result below the confidence gate returns to the player as a redraw and is logged as a failure; a result above it is assigned its ability and applied to the obstacle. Each cleared obstacle updates telemetry and route history, and the loop returns to level selection. The lower half of the figure shows what those accumulated records are for: on reaching the final level, the route profile, class diversity, and redraw rate together select among Endings A through D. The same counters that measure the system therefore also determine what the player receives, which is the entanglement discussed in Section 3.5 and addressed in Section 4.10.3.

4.6 Development Model

This study adopts the Iterative and Incremental Development Model. In this model, the system is not constructed in a single pass, but instead developed through repeated cycles, each of which produces a working increment of the system that is evaluated and then refined in the cycle that follows. Rather than requiring a complete and fixed specification before implementation begins, the model permits requirements and design to evolve as knowledge is gained from each iteration. Within individual iterations the study employs a prototyping approach, in which a working component is built and exercised directly rather than specified exhaustively in advance.

4.6.1 Justification of the Model

The iterative and incremental model suits this study for four reasons.

1. The machine learning component is inherently iterative. A convolutional neural network cannot be specified correctly in advance. It must be trained, evaluated against per-class precision,

recall, and a confusion matrix, and then retrained with adjusted data or parameters until acceptable performance is reached. The fifty-class vocabulary of this study makes this especially so, since the study anticipates that the primitive classes will absorb misclassifications from the remainder of the vocabulary, and this behavior only becomes visible once a confusion matrix exists. A linear model such as Waterfall, which presumes that requirements are fully known before implementation begins, cannot accommodate a component whose correctness is discovered through measurement.

2. OBRA consists of clearly separable increments: the sketch classifier, the ability-attribution table, the stroke-capture and canvas subsystem, the procedural animation of the player's strokes, the game levels, and the loopback integration between the game runtime and the recognition component. Each can be built and validated on its own before being composed with the others.

3. The central runtime challenge of the system cannot be resolved by specification. Because the artwork animated at runtime does not exist until the player draws it, procedural stroke animation must be exercised against real, unanticipated drawings in order to discover the conditions under which it degrades. Conditions such as disconnected strokes, unusual proportions, and drawings that do not conform to the animation rules are found by iteration, not by design.

4. The game's mechanics depend on player experience, which cannot be evaluated from a specification. A working increment must exist before it can be played, tested, and refined. This concern is not incidental: for a game, the experience the system provides *is* its

chief outcome, unlike a productivity application whose value can be assessed against a functional specification (Cutting et al., 2025).

The Prototyping Model alone was considered and rejected as the governing life cycle, since it emphasizes the elicitation of requirements through mock-ups and does not by itself account for the repeated retraining and measurement that the machine learning component demands. Prototyping is therefore employed as a technique within iterations rather than as the overarching model.

4.6.2 Phases of the Model

Each iteration of the study proceeds through the following phases:

1. **Planning and Requirements Elicitation**- The scope of increment is defined.

   Across the project, this phase established the game concept, the fifty-class vocabulary recorded, the abilities attributed to each class, and the target platforms.

2. **Analysis and Design** - The increment is designed.

   This produced the four-stage drawing-to-ability interaction, the monolithic architecture setup, the request-response cycle, and the training pipeline.

3. **Implementation** - The increment is built.

   Across iterations this comprised: the stroke-capture canvas built on SubViewport and Line2D; the SketchCNN classifier trained on Quick, Draw! Bitmaps; the static ability-attribution table resolved offline from ConceptNet (Speer, Chin, & Havasi, 2017); the recognition backend served through FastAPI and ONNX Runtime, RuntimeRig2D, which constructs a runtime rig from the player's

own stroke geometry, the spawning of playable and obstacle actors; the game environment and levels; and the telemetry subsystem.

4. **Testing** - The increment is verified.

    Unit tests exercise components in isolation, integration tests exercise the exchange across the loopback interface, and regression tests confirm that previously completed increments remain intact.

5. **Evaluation and Refinement** - The increment is assessed against its intended behavior.

    Model metrics, response-time measurements, and playtest observations are reviewed, and the findings inform the requirements and design of the subsequent iteration. It is this feedback that distinguishes an iterative process from a merely incremental one. Because the evaluation instruments adopted by this study are designed for repeated administration during iterative game design and testing (Keebler et al., 2020), they may be applied at the close of successive iterations rather than only at the study's conclusion.

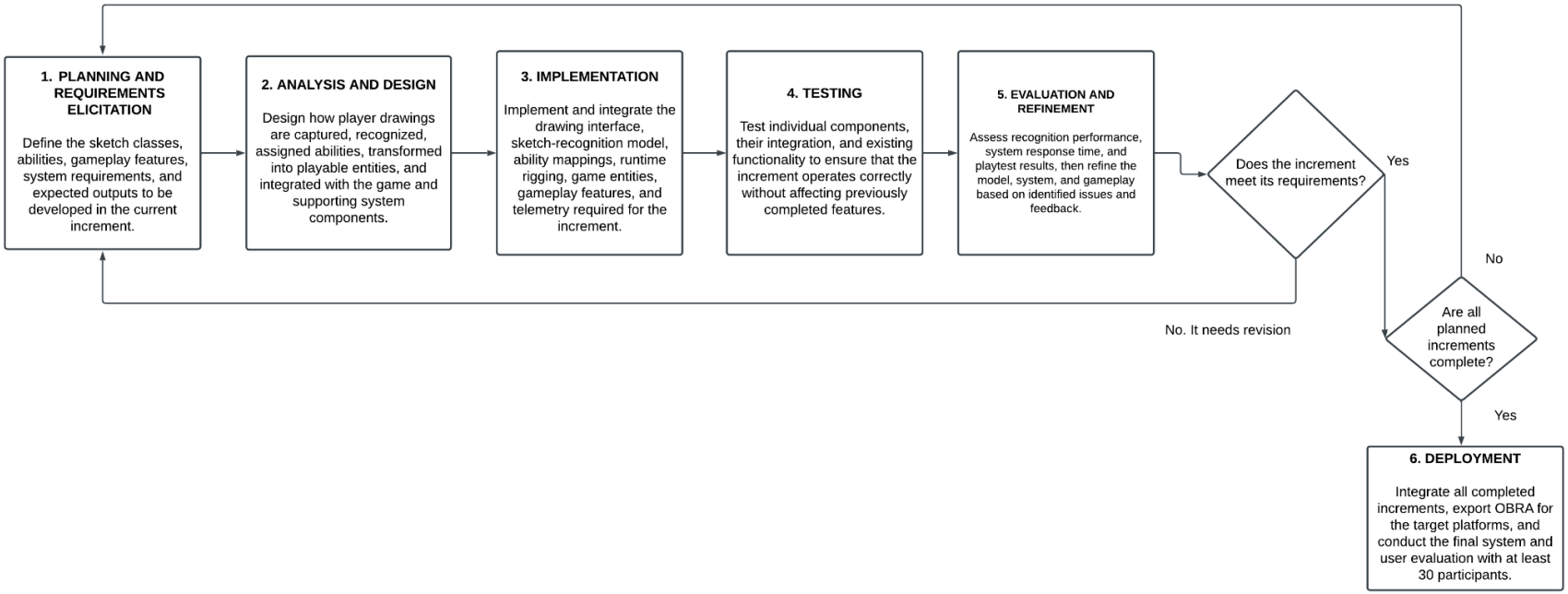
6. **Deployment**

    Once the increments are composed into a complete system, the game is exported for Windows and macOS and evaluated through user testing with a minimum of thirty participants.

4.6.3 SDLC Diagram

Figure 16

Iterative and Incremental Developmental Model for OBRA



Note. Adapted from Pressman (2010). Software Engineering: A Practitioner's Approach (7th edition). McGraw-Hill Science, Engineering & Mathematics.

Figure 16 shows the five activities that make up one increment — planning and requirements elicitation, analysis and design, implementation, testing, and evaluation and refinement — followed by the two decisions that govern the loop. The first asks whether the increment meets its requirements; if it does not, the increment returns to analysis and design rather than proceeding. The second asks whether all planned increments are complete, and only then does the project move to deployment and the evaluation sessions. The model is drawn this way because recognition quality and gameplay feel cannot be assessed separately: an increment is only judged complete once the model and the level content that depends on it have been exercised together.

4.7 Development Approaches

This section describes the engineering approach behind OBRA: the architectural style that shapes the system, the programming paradigms and design patterns used in each layer, the way components are integrated and data is handled, the security and quality practices applied, and the version-control strategy. The account reflects the system as actually implemented across the game runtime (Godot 4 / GDScript), the inference backend (FastAPI + ONNX Runtime), and the shared Python tooling; the project's own engineering conventions are recorded in AGENTS.md, and the entity manifest (game/config/entities.json) is treated as the single source of truth.

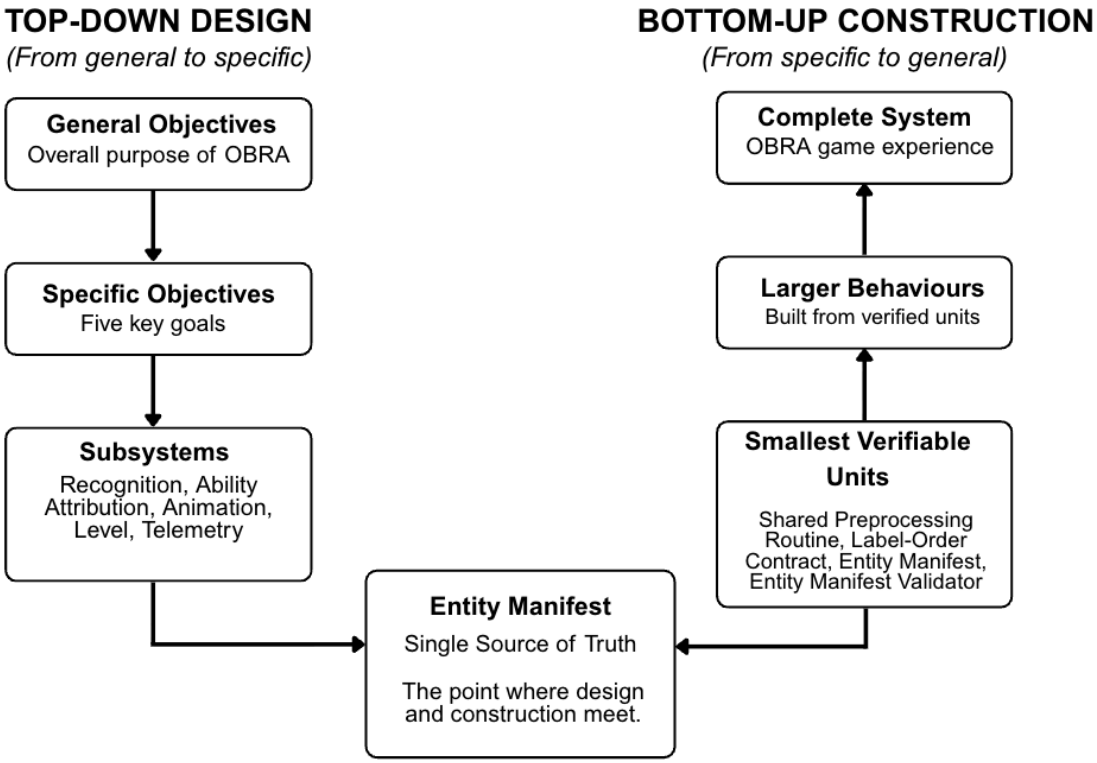
A guiding design decision in OBRA is to keep the game and machine-learning components separate in function while still operating as one integrated system on the same computer.

Design Direction

OBRA is designed top-down and built bottom-up, and the two directions meet at the entity manifest. *Figure 17* illustrates the relationship between the two design directions and their convergence at the entity manifest.

Figure 17

Design direction approach of OBRA



The design proceeds top-down: the study begins from the general objective, decomposes it into the five specific objectives, and decomposes those in turn into the subsystems that satisfy them — recognition, ability attribution, procedural animation, level structure, and telemetry. Each requirement in the specification is traceable upward to the objective that motivates it, which is what makes the objective traceability check in Section 4.10.3 possible at all. Nothing is built that cannot be traced to a stated aim.

Construction proceeds bottom-up. The smallest verifiable units are completed and tested first — the shared preprocessing routine, the label-order contract, the entity manifest and its validator — and larger

behaviours are assembled from units already known to be correct. This direction is chosen deliberately, because the three faults the study most needs to avoid are silent ones: a preprocessing routine that diverges by a pixel between training and play, a label listing one position out of order, and a level authored against a class the roster does not contain. None of the three raises an error, and none is visible from the top of the system; each is caught only where the unit itself is exercised. Building upward from verified units places a test at exactly the level where such a fault can still be seen.

The iterative and incremental life cycle described in Section 4.6 accommodates both directions: each increment is planned top-down from the objectives and implemented bottom-up from its units, and the entity manifest serves as the fixed point where the decomposition and the construction must agree.

This separation helps organize the system and guides the programming and design choices used throughout the project.

4.7.1 Architectural Style

OBRA adopts a monolithic architecture: it is built, deployed, and run as a single application on a single machine, with a pretrained recognition model served locally inside it. The application is internally modular, comprising:

- A game runtime (Godot 4 / GDScript) that holds all user interaction, gameplay logic, physics, and animation, organized as a component-based scene graph.
- A local recognition backend (FastAPI + ONNX Runtime) that loads a pretrained convolutional neural network and answers classification requests on the loopback interface. The model is

trained offline beforehand and only loaded at runtime, never trained.

These two parts run together on one machine and communicate over a local loopback HTTP call; the game itself starts and supervises the recognizer. The system is therefore monolithic in deployment , one unit, one machine, rather than a distributed client-server product, and the internal loopback boundary is an implementation detail, not a network architecture. It is not a three-tier application either, because it has no database tier: its persistence consists of static configuration and pretrained model files, a local file-based player-profile save that carries progression across sessions, and in-memory state for transient in-level data.

This structure was chosen for a concrete reason documented in AGENTS.md: the game engine has no built-in machine-learning runtime, and the training ecosystem is Python. Serving the pretrained model as a co-located local component bridges the Python ML stack and the GDScript engine without leaving the machine, so training-side and runtime-side code can evolve independently while the whole system still ships and runs as a single application.

4.7.2 Software Development Paradigms

The system is intentionally multi-paradigm, applying the paradigm that best fits each layer rather than forcing one style throughout.

- Object-oriented programming (client). The Godot client is predominantly object-oriented. Behavior is organized into class hierarchies with encapsulated state and shared base classes: playable creatures derive from PlayableEntity through CreatureMovementController, and drawable props derive from DrawableObject through PhysicsShapeObject. Common responsibilities (skinning, collision rebuilding, motion publishing,

lifecycle) live in the base classes, and specific creatures or shapes override only what differs. A limited amount of OOP also appears in Python: the immutable EntityDefinition dataclass and the PyTorch nn.Module network and Dataset classes.

- Component-based (composition) design. On top of inheritance, the client uses Godot's node-and-scene model as a component system. An entity is a composed scene (a body node, a DrawingSkin, a habitat probe, collision shapes), and its tunable behavior is supplied as data components (rig, movement, and object profiles). This favors composition over deep inheritance and lets designers change feel by editing JSON rather than code.

- Functional and procedural style (data pipeline). The Python data and inference code leans functional and procedural. Preprocessing is a pure function (image bytes to tensor), softmax is a pure function, and the training and download scripts are straightforward procedural pipelines built from small, single-purpose functions. This keeps the pipeline deterministic and easy to reason about and test.

- Event-driven (reactive) style (intra-client). Communication inside the client is signal-based rather than call-based: components emit named signals (for example drawing_ready, entity_prediction_received, backend_ready, ink_changed, checkpoint_changed) and interested nodes subscribe. This decouples producers from consumers and is discussed further under Integration.

- Declarative, data-driven configuration. Much of the system's behavior is expressed as data, not code. The roster, scene bindings, physics parameters, and animation tuning all live in versioned JSON, following the project convention to "use those files for per-entity feel before hardcoding behavior."

4.7.3 Design Patterns and Coding Philosophies

Several recognizable patterns organize the client.

- A Registry-plus-Factory (EntityRegistry) loads the manifest and instantiates the correct scene for a given entity id, hiding scene wiring from callers.
- A Template Method structures motion — CreatureMovementController defines the motion-publishing frame while each creature supplies its own locomotion.
- Strategy-via-data lets the pose_driver field and the movement and object profiles select animation and physics behavior by configuration rather than by branching.
- Godot signals implement Observer / Publish–Subscribe throughout, and LevelDirector acts as a Mediator that centralizes the state which must survive an actor replacement, capturing and restoring checkpoint snapshots (Memento).

The overarching philosophy is discipline through contracts and typing: statically typed GDScript and Python, small single-responsibility scripts, schema-versioned data files, fail-fast validation, and a single shared preprocessing routine that guarantees train–serve parity.

4.7.4 Integration and Data Handling

Integration happens at two levels, each with a paradigm suited to it.

- Between the game and its local recognizer: a local request-response HTTP interface. The game runtime and the co-located recognition backend integrate through a small HTTP interface over the loopback address on the same machine. The

game issues an asynchronous POST /predict carrying a JSON body, plus a GET / health-and-metadata request used for readiness. Requests and responses are JSON; the drawing is carried as a Base64-encoded PNG inside the JSON body, and the response returns the recognized entity together with confidence, margin, runner-up, per-class probabilities, and the entity's scene, rig, movement, and object profile paths. Because the contract is plain HTTP and JSON, the two languages (GDScript and Python) interoperate cleanly on one host: this local boundary is what lets a Python inference component sit inside a GDScript application without leaving the machine. The recognizer is stateless, so each request is self-contained and it holds no session.

- Inside the game runtime: event-driven signals. Within Godot, components integrate through the observer pattern using typed signals instead of direct coupling. The capture, recognition, dispatch, and animation stages hand off through emitted signals (drawing to draw_panel to game_level, then to the spawned actor via apply_drawing), which keeps each stage independently replaceable.

Data handling emphasizes validation and typed contracts at every seam:

- Input is validated at the trust boundary. The backend uses a Pydantic request model, strictly validates the Base64 payload, rejects an empty canvas (HTTP 422) and an undecodable image (HTTP 400), and checks the model's output width against the label count at startup.

- Configuration is validated on load. Both shared/entities.py and the game's EntityRegistry validate required fields, allowed enumerations, schema versions, and the existence of referenced scenes and profiles before anything runs.

- A model-to-game contract is enforced. validate_model_labels asserts that labels.json matches the manifest order exactly, and a repository test locks this invariant in place.

- Formats are chosen per role. NumPy .npy for training arrays, ONNX for the portable model, JSON for configuration and API messages, PNG/Base64 for images in transit, and Godot scenes (.tscn) for composed entities. Data acquisition itself is handled idempotently and safely (skip-if-present, temp-file-then-rename) so it can be safely re-run.

4.7.5 Security and Quality Approaches

The system is a single-user prototype confined to one local machine, and its security posture is scoped accordingly:

- Privacy by design. Only anonymous drawings are processed and the software and research datasets collect no personally identifying information; signed consent forms are stored separately from session data collected. The only data persisted to disk is an anonymous local player profile that records game progression, together with anonymous session telemetry; both stay on the host machine and neither contains personal data, so nothing that identifies the player ever leaves the device.

- Input validation and defensive decoding. Every external input crossing into the backend is validated and defensively decoded (strict Base64, guarded image decode, empty-canvas rejection, startup output-width check), so malformed input yields a clean error rather than undefined behavior.

- Local trust boundary. The game runtime and the recognition backend communicate only over the machine's loopback interface, so requests never leave the host, and there are no runtime calls to

third-party services. CORS is enabled on the backend; it is intentionally permissive (open origins), a deliberate, documented trade-off appropriate to a local, single-user prototype rather than a production default. Likewise, the game's automatic start of the local recognizer spawns a process from a resolved virtual-environment interpreter, which assumes a trusted local machine.

Quality is pursued through validation, reproducibility, and parity rather than heavy process:

- Automated contract tests. A unittest suite (tests/test_manifest_contract.py) enforces structural invariants: exactly fifty labels in manifest order, twenty playable creatures and thirty drawable objects, and the presence of the required per-kind profiles.
- Schema versioning and fail-fast checks. Rig, movement, and object profiles declare a schema_version that is validated on load, and missing model, label, scene, or profile files abort startup with actionable messages.
- Reproducibility. Training uses a fixed random seed and records model_metadata.json and metrics.json (including the classification report and confusion matrix), so results can be reproduced and audited.
- Train / evaluate / runtime parity. A single preprocessing function is shared by the training-time evaluation and the live backend, removing a common source of train-serve skew.
- Static typing and living documentation. Typed GDScript and Python type hints catch mismatches early, and README.md plus AGENTS.md document the stable contracts and workflow and are updated when the architecture changes.

- Instrumentation. Optional end-to-end timing logs on both the game and the recognition backend support the latency measurement the study requires.

4.7.6 Version Control Strategy

The project uses Git for version control, hosted on GitHub. Development follows a trunk-based workflow: the main branch is the always-integrated line, short-lived branches carry larger or riskier changes and are merged back, and work is committed in small, feature-scoped increments. Re-downloadable datasets, local runtime caches, and thesis working documents are excluded through .gitignore, so version control tracks only source and configuration.

4.8 Software Development Tools

This section lists the tools used to build OBRA, the layer each belongs to, whether it was reused or written for this study, and the licence under which it is distributed. Because OBRA is a single monolithic application with a pretrained model served locally, front-end and back-end refer to two co-located parts of one program on one machine rather than to separately deployed systems.

Table 10

Software Development Tools

Software	Version	Layer	License	Purpose
Godot Engine	4.7	Front-end	MIT	Game engine and editor providing the runtime, drawing canvas, input handling, real-time 2D rendering, physics, and scene graph
GDScript	Godot 4.7	Front-end	MIT	Godot's integrated scripting language; implements all hand-written game logic, stroke capture, procedural rigging, movement-driven animation, and the ink and checkpoint systems

Table 10 (Continued)

Software Development Tools

Software	Version	Layer	License	Purpose
Python	3.12.6	Back-end	PSF License 2.0	Host language of the local recognition service and the offline model-training pipeline
FastAPI	0.139.0	Back-end	MIT	Web framework exposing the local recognizer (GET / and POST /predict) with type-validated requests
Uvicorn	0.50.2	Back-end	BSD 3-Clause	ASGI server that runs the FastAPI recognition application on the loopback interface
Pydantic	2.13.4	Back-end	MIT	Validates the request and response schemas of the recognition endpoint
Pillow (PIL)	12.3.0	Back-end	MIT-CMU	Decodes the submitted PNG and performs grayscale conversion, cropping, and resizing during preprocessing
NumPy	2.5.1	Back-end and offline	BSD 3-Clause	Array handling for dataset loading, preprocessing tensors, and metric computation
PyTorch (torch)	2.12.1	Offline	BSD 3-Clause	Deep-learning framework used to define and train the drawing classifier offline
torchvision	0.27.1	Offline	BSD 3-Clause	Image transforms and data augmentation during training (random affine perturbations)
ONNX	1.22.0	Offline	Apache 2.0	Open model-interchange format the trained classifier is exported to
onnxscript	0.7.1	Offline	MIT	Supporting library for exporting the PyTorch model to the ONNX format
scikit-learn	1.9.0	Offline	BSD 3-Clause	Evaluation metrics: per-class precision and recall, classification report, and confusion matrix

Table 10 (Continued)

Software Development Tools

Software	Version	Layer	License	Purpose
Matplotlib	3.11.0	Offline	Matplotlib License (PSF-based, BSD-compatible)	Renders the confusion-matrix figure reported in the study
Git	2.50.1	Development tooling	GPL 2.0	Version control for source code and configuration under the trunk-based workflow
Aseprite	1.3.17	Asset creation	Proprietary EULA (Commercial Licence Purchased)	Pixel-art image editor and animation tool used to author the game's sprites, environment art, and UI assets, licensed under the Aseprite EULA

Note. Layer indicates where each tool applies within the single monolithic application: the front-end game runtime, the co-located back-end recognition service, the offline training pipeline that runs separately from the game, development tooling, or asset creation. Version numbers reflect the environment used during implementation. All runtime and training dependencies are free and open-source; GitHub is used on its free tier for version control and code hosting. Aseprite is source-available proprietary asset-authoring software compiled from its public repository at no cost. None of the three is required to build or run the delivered system.

4.9 Project Management

The development of OBRA is undertaken by a team of four researchers across two academic terms, following the iterative and incremental model described in Section 4.6. Project management is organized around three concerns: the scheduling of activities across the development timeline, the assignment of responsibilities among the members of the research team, and the

management of the costs incurred in the conduct of the study. Each is described in the subsections that follow.

4.9.1 Schedule and Timeline

The study is conducted over two academic terms. The first, spanning the summer term of A.Y. 2025–2026, is devoted to the formulation and documentation of the proposal, culminating in the thesis proposal defense. The second, spanning the first semester of A.Y. 2026–2027, is devoted to the implementation and evaluation of the system, culminating in the thesis defense and the submission of the hardbound manuscript. Figures 18 and 19 present the schedule of activities for each term respectively. The star (★) denotes a milestone.

4.9.1.1 Proposal Phase

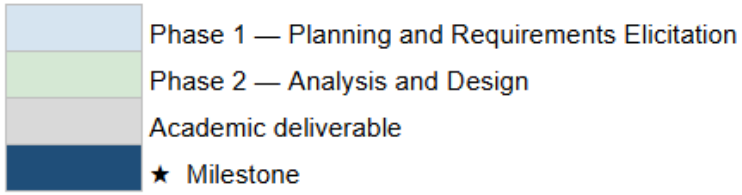
Figure 18 sets out the proposal term across June, July, and August 2026 in weekly columns, with each activity labelled by the phase of the development model to which it belongs. The proposal term constitutes the first pass through the model's opening phases. Planning and Requirements Elicitation covers the establishment of the title and themes, the review of related literature, the technical background, and the research objectives, all of which serve to define the scope of the work to be built. Analysis and Design covers Chapter 4 and the Software Requirements Specification, in which the drawing-to-ability interaction, the system architecture, and the training pipeline are designed. The remaining phases Implementation, Testing, Evaluation and Refinement, and Deployment — fall in the implementation term and are shown in Figure 18. The chapters are scheduled in the order the study needs them rather than the order in which they appear in the manuscript: Chapters 2, 3, and 4 precede Chapter 1, because the rationale can be written only once the technical scope it claims has been settled.

The activities shown in grey are academic deliverables — the manuscript itself, the defense, and the associated forms — which accompany the study but produce no software increment and therefore correspond to no phase of the development model. The proposal defense closes the term and is marked as a milestone.

Figure 18

Gantt Chart of Activities for Summer 2026, A.Y. 2025-2026

Legend:



Details			JUN				JUL				AUG			
#	Phase of the Development Model	Activity / Deliverable	W1	W2	W3	W4	W1	W2	W3	W4	W1	W2	W3	W4
ITERATION 0 — PROPOSAL INCREMENT														
1	**1. Planning and Requirements Elicitation**	Research title and themes												
2		Signed schedule of deliverables												
3		Chapter 2: Review of Related Literature and Bibliography												
4		Chapter 3: Technical Background												
5		Research objectives and statement of the problem												
6	**2. Analysis and Design**	Chapter 4: Design and Methodology												
7		Software Requirements Specification and appendices												
ACADEMIC DELIVERABLES														
8	**Documentation**	Chapter 1: Rationale, significance, scope and limitation, abstract												
9		Proposal document draft, Chapters 1–4 with references												
10		Agreement letter and initial revisions												

Details			JUN				JUL				AUG			
#	Phase of the Development Model	Activity / Deliverable	W1	W2	W3	W4	W1	W2	W3	W4	W1	W2	W3	W4
11	**Defense**	Submission of three copies of the thesis proposal												
12		Thesis proposal defense									★			
13		Submission of Rubrics 1, 2.1 and 2.2												
14		Submission of revised document and compliance form												

4.9.1.2 Implementation Phase

The implementation term is organized into three iterations, each of which is a complete pass through Phases 1 to 5 of the development model described in Section 4.6.2, followed by a single entry into Phase 6. Iteration 1 produces the recognition increment: the vocabulary and recognition requirements are defined, the training pipeline and ability-attribution scheme are designed, the classifier is trained and the ConceptNet-derived ability table is resolved offline, the increment is verified through per-class precision and recall against a held-out split and against the independent TU-Berlin test set, and the resulting figures inform the calibration of the confidence and margin thresholds. Iteration 2 produces the playable increment: the drawing canvas, the procedural stroke animation, the visual assets, the levels, and the telemetry subsystem. It runs concurrently with Iteration 1, because the recognition track and the game runtime share no runtime dependency until integration and can therefore proceed in parallel. Iteration 3 composes the two: the model is exported, the recognition backend is served locally, the game and the backend are integrated across the loopback interface, and the composed system is verified through integration and regression tests before its end-to-end response time is measured. The finished-game milestone closes Iteration 3.

Phase 6, Deployment, is entered once, after the final increment. It comprises the export of the composed system for its target platforms, user testing with a minimum of thirty participants, the administration of the evaluation instruments, and the analysis of the accompanying telemetry. The remaining activities in *Figure 19*, the writing of Chapters 5 and 6, the submission to conference, the

thesis defense, and the preparation of the hardbound manuscript, are academic deliverables rather than phases of the development model, and are shown in grey to mark the distinction. *Figure 26* also makes the project's critical path visible: everything after Iteration 3 depends upon a working integration, and the user-testing window cannot open until the finished-game milestone is reached, which is why that milestone is marked rather than merely implied by the end of a bar.

Figure 19

Gantt Chart of Activities for First Semester, A.Y. 2026-2027

Legend:

Phase 1 — Planning and Requirements Elicitation
Phase 2 — Analysis and Design
Phase 3 — Implementation
Phase 4 — Testing
Phase 5 — Evaluation and Refinement
Phase 6 — Deployment
Academic deliverable
★ Milestone

Details			JUL				AUG				SEP				OCT				NOV				DEC			
#	Phase of the Development Model	Activity / Deliverable	1	2	3	4	1	2	3	4	1	2	3	4	1	2	3	4	1	2	3	4	1	2	3	4
ITERATION 1 — RECOGNITION INCREMENT (the classifier and the ability-attribution table)																										
1	**1. Planning and Requirements Elicitation**	Define the fifty-class vocabulary, target abilities, and recognition requirements for the increment																								
2	**2. Analysis and Design**	Design the training pipeline, the preprocessing contract, and the ability-attribution scheme																								
3	**3. Implementation**	Dataset acquisition; preprocessing and 80/10/10 split; CNN training; ConceptNet resolved offline into the static ability table																								
4	**4. Testing**	Unit tests on preprocessing and inference; per-class precision and recall; confusion matrix on the held-out split and on TU-Berlin																								

Details			JUL				AUG				SEP				OCT				NOV				DEC			
#	Phase of the Development Model	Activity / Deliverable	1	2	3	4	1	2	3	4	1	2	3	4	1	2	3	4	1	2	3	4	1	2	3	4
5	**5. Evaluation and Refinement**	Review of per-class recall against the accuracy floor; calibration of the confidence and margin thresholds; refinement feeding Iteration 3																								
ITERATION 2 — PLAYABLE INCREMENT (the game runtime, run concurrently with Iteration 1)																										
6	**1. Planning and Requirements Elicitation**	Define narrative, mechanics, level scope, and telemetry requirements for the increment																								
7	**2. Analysis and Design**	Design the drawing canvas, the entity model, the level structure, and the telemetry schema																								
8	**3. Implementation**	Stroke-capture canvas on SubViewport and Line2D; RuntimeRig2D procedural stroke animation; visual assets; environment and levels in Godot; telemetry subsystem																								
9	**4. Testing**	Unit tests on canvas capture, rig construction, and telemetry writes; regression against completed increments																								
10	**5. Evaluation and Refinement**	Playtest observation of the drawing loop and animation legibility; refinement feeding Iteration 3																								

Details			JUL				AUG				SEP				OCT				NOV				DEC			
#	Phase of the Development Model	Activity / Deliverable	1	2	3	4	1	2	3	4	1	2	3	4	1	2	3	4	1	2	3	4	1	2	3	4
ITERATION 3 — INTEGRATED INCREMENT (composition of the two tracks across the loopback interface)																										
11	**1. Planning and Requirements Elicitation**	Define the integration contract and the end-to-end response-time requirement																								
12	**2. Analysis and Design**	Design the request–response cycle and the backend supervision and health-check sequence																								
13	**3. Implementation**	Model export to ONNX; recognition backend served through FastAPI and ONNX Runtime; game and backend integrated over the loopback interface																								
14	**4. Testing**	Integration tests across the loopback interface; regression tests confirming earlier increments remain intact; system validation and defect correction																								
15	**5. Evaluation and Refinement**	End-to-end response-time measurement against the responsiveness requirement; final refinement of the composed system																								
★	**Milestone**	Finished game — all planned increments complete											★													

Details			JUL				AUG				SEP				OCT				NOV				DEC			
#	Phase of the Development Model	Activity / Deliverable	1	2	3	4	1	2	3	4	1	2	3	4	1	2	3	4	1	2	3	4	1	2	3	4
PHASE 6 — DEPLOYMENT																										
16	**6. Deployment**	Export of the composed system for Windows and macOS																								
17		User testing with a minimum of thirty participants; administration of the evaluation instruments																								
18		Telemetry analysis and consolidation of evaluation results																								
ITERATION 4 — POST-TESTING REVISIONS																										
19	**1. Planning and Requirements Elicitation**	Consolidation and prioritisation of defects, usability findings, and instrument results from the evaluation																								
	2. Analysis and Design	Revision of game systems, level content, and dialogue nodes addressing the prioritised findings																								
	3. Implementation	Retuning of recognition thresholds and, where per-class recall falls below the accepted floor, retraining and re-export of the model artifact set																								
	4. Testing	Regression testing of the revised build against the original test cases																								

Details			JUL				AUG				SEP				OCT				NOV				DEC			
#	Phase of the Development Model	Activity / Deliverable	1	2	3	4	1	2	3	4	1	2	3	4	1	2	3	4	1	2	3	4	1	2	3	4
	5. Evaluation and Refinement	Confirmatory re-testing with game development adviser																								
	6. Deployment	Re-export of the composed system and consolidation of revised results																								
ACADEMIC DELIVERABLES (documentation, defense, and submission)																										
20	**Documentation**	Chapter 5: Discussion of Results																								
21		Chapter 6: Summary, Conclusion and Recommendations																								
22		Document revisions																								
★		Submission to conference																★								
22	**Defense**	Defense preparation																								
★		Thesis defense presentation																			★					

Details			JUL				AUG				SEP				OCT				NOV				DEC			
#	Phase of the Development Model	Activity / Deliverable	1	2	3	4	1	2	3	4	1	2	3	4	1	2	3	4	1	2	3	4	1	2	3	4
23	**Final submission**	Final revisions and manuscript																								
24		Hardbound preparation and printing																								
★		Submission of hardbound manuscript																						★		

4.9.2 Responsibilities

The research team consists of four members. Roles are assigned according to the technical and research demands of the study, so that each member owns a distinct component of the system while contributing to the review of related literature and to the thesis documentation. Because the machine learning backend and the game runtime are developed in parallel, the lead developer holds responsibility for both tracks, supported by a second researcher-developer who assists across the two and independently owns the telemetry subsystem. Note that all four members serve as researchers and contribute to the review of related literature and to the thesis documentation. Table 11 presents the roles and responsibilities of each member.

Table 11

Table of Roles and Responsibilities

Member	Roles	Assignments
Rusk Rye Ioan C. Alarde	Researcher and Lead Developer	Learn the fundamental concepts of Convolutional Neural Networks and how they are implemented for sketch classification
		Prepare the dataset and train the CNN model, and afterwards test and evaluate the model
		Develop the game environment, levels, and physics-based movement system in Godot
		Develop the sketch-to-vector and animation pipeline, and integrate the machine learning backend with the game runtime

Table 11 (Continued)
Table of Roles and Responsibilities

Member	Roles	Assignments
Erika Bianca E. Api	Researcher, Developer, Game Designer	Design and produce the visual assets, sprites, and user interface elements of the game
		Establish the art direction of the game consistent with its narrative
		Conduct the review of related literature and contribute to the thesis documentation
		Assist in the conduct of user testing and in the collection of respondent data
Kent Jester P. Gatchalian	Researcher and Developer	Learn the fundamental concepts of Convolutional Neural Networks and how they are implemented for sketch classification
		Pre-process the data to be used in the training and testing of the model, and assist in model evaluation
		Assist in the development of the game levels and implement the in-game telemetry system
		Conduct research on the technical background of the study and contribute to the thesis documentation
Lady Kaina V. Gutierrez	Project Leader, Researcher and Game Designer	Coordinate the overall project, including task delegation, scheduling, and monitoring of deliverables
		Lead the research documentation and design the user testing protocol and acquire questionnaire, and analyze the evaluation and telemetry results

4.9.3 Budget and Cost Management

The costs incurred in the conduct of this study are grouped into six categories: hardware and equipment, software licences, datasets and knowledge base, compute and utilities, evaluation, and documentation. A contingency fund equivalent to ten percent of the subtotal is allocated to accommodate expenses not anticipated within these categories. Table 12 presents the detailed budget.

Table 12

Table of Expenses

Description	Unit Cost	Qty	Unit	Total
Hardware and Equipment				**₱102,000**
Workstation	₱45,000.00	2	units	₱90,000
Printer (Epson EcoTank L5290)	₱12,00000	1	unit	₱12,000
Software Licences				**₱1,100**
Aseprite (raster art and animation editor)	₱1,100.00	1	licence	₱1,100
Godot 4 (game engine, MIT licence)	₱0.00	1	licence	₱0
Python, PyTorch, ONNX Runtime, FastAPI	₱0.00	1	lot	₱0
Datasets and Knowledge Base				**₱0**
Quick, Draw! (CC BY 4.0)	₱0.00	1	corpus	₱0
TU-Berlin Sketch Dataset	₱0.00	1	corpus	₱0
ConceptNet 5.7.0 assertions dump	₱0.00	1	dump	₱0
Compute and Utilities				**₱24,000.00**
Electricity	₱2,000.00	6	months	₱12,000.00
Internet connectivity	₱2,000.00	6	months	₱12,000.00

Table 12 (Continuation)

Table of Expenses

Description	Unit Cost	Qty	Unit	Total
Evaluation				**₱3,000.00**
Respondent tokens (min. 30 participants)	₱100.00	30	pax	₱3,000.00
Venue / laboratory use (university-provided)	₱0.00	1	lot	₱0.00
Documentation				**₱2,150.00**
Bond paper (letter, 80 gsm)	₱350.00	3	reams	₱1,050.00
Hardbound copies	₱220.00	5	pcs	₱1,100.00
Subtotal				**₱132,250.00**
Contingency (10%)				**₱13,225.00**
Total				**₱145,475.00**

4.9.4 Budget Details

Table 12 presents the estimated cost of reproducing the study, organized into six categories. Hardware and equipment account for the largest share at ₱102,000, comprising two workstations and a printer. A reproducing team requires at least one workstation with a quad-core x86-64 processor or Apple Silicon equivalent, 16 GB of memory, and 50 GB of free storage, the last being driven chiefly by the uncompressed ConceptNet assertions dump and the fifty-class Quick, Draw! corpus. A dedicated graphics card is not required. Model training in this study is performed locally on the reference machine described in Section 4.1.1, using the Metal Performance Shaders backend for acceleration; a reproducing team working on x86-64 hardware would need a capable card with at least 4 GB of video memory. Software licences total ₱1,100, the

cost of a single raster art editor; the game engine, the machine-learning stack, and the serving framework are open-source and carry no licence fee. The datasets and knowledge base — both sketch corpora and the ConceptNet assertions dump — are freely available and cost nothing to obtain. Compute and utilities total ₱24,000 across the six-month term, covering cloud GPU access for model training together with electricity and internet connectivity. Evaluation costs ₱3,000 in respondent tokens for a minimum of thirty participants, the venue being university-provided. Documentation costs ₱2,150 for paper and hardbound copies. With a ten per cent contingency applied to the subtotal of ₱132,250, the estimated total is ₱145,475.

4.10 Verification, Validation, and Testing

4.10.1 Research Methodology of the Study

Researchers draw upon several scientific methodologies, and the activities required to address even a single research question may span more than one. This study is principally a Build Method study: its purpose is to construct an artifact: a functional game system in order to demonstrate that a player's freehand drawing can be recognized, attributed a documented real-world ability, and animated as the player's own strokes. Accordingly, the study reuses established components rather than reimplementing them (Godot, PyTorch, ONNX Runtime, FastAPI, and ConceptNet; Speer et al., 2017) and treats testing as a continuous activity rather than a terminal phase.

Two further methodologies support the work. The Experimental Method governs evaluation, which proceeds from an exploratory phase, in which initial training runs and response-time measurements reveal what questions ought to be asked, to an evaluation phase that answers them. The Model Method supplies the abstract representations of the system used to reason about its structure, namely the four-stage drawing-to-ability

interaction, the separation of the game runtime from the co-located recognition component, the request–response cycle, and the training pipeline.

The evaluation is designed to answer the following questions:

(a) how accurately does the model classify player-drawn sketches, and which classes does it confuse;

(b) is end-to-end response time low enough to sustain uninterrupted gameplay;

(c) do the attributed abilities correspond to players' expectations;

(d) does procedural stroke animation produce believable motion on drawings the researchers did not author;

(e) does the drawing canvas adequately support the player's creative expression; and

(f) can players complete the levels using the drawing mechanic.

The sections that follow describe how the system is verified against its specification, validated against the study's objectives and its users' needs, and tested for defects, together with the quantitative and qualitative measures planned to address each specific objective.

4.10.2 Verification

Verification determines whether the system is being built correctly, whether the implementation conforms to its specification and design. It is internal and technical, conducted by the researchers throughout the iterative cycle.

Code review. Each module produced within an iteration is reviewed by at least one team member other than its author, checking

conformance to coding conventions, correctness of logic, and adherence to the established architecture.

Manifest consistency. The entities.json manifest is the single source of truth for the class vocabulary. Verification confirms that the source labels exported to labels.json appear in prediction order, that the label-to-entity mapping in model_metadata.json is complete across all fifty classes, and that the labels validated at backend startup match the manifest. A mismatch between the model's output indices and the manifest would silently attribute the wrong ability to a correctly recognized drawing, and is therefore verified explicitly rather than assumed.

Unit testing. Components are tested in isolation against their specifications.

- *Preprocessing.* The routine is verified to invert the capture, crop to the ink bounding box, scale to 28 × 28 while preserving proportion, and normalize into the expected numeric range, producing a well-formed 1 × 1 × 28 × 28 tensor. Because a sketch is largely empty space held together by thin lines (Eitz, Hays, & Alexa, 2012), the crop and resize steps are tested against edge cases: drawings placed near the canvas border, very small drawings, and drawings consisting of a single stroke.

- *Ability attribution.* The static lookup table is verified to return the correct predefined ability for each of the fifty classes, and to raise an explicit error rather than a default value for any label absent from the table.

- *Rejection logic.* The thresholds are verified to behave as specified: a prediction with confidence below 0.60, or with a margin below 0.15, must trigger the redraw prompt and must not spawn an actor.

- *Stroke capture.* get_strokes() is verified to return polylines faithfully

corresponding to the Line2D points the player drew, since these strokes constitute the artwork that will be animated.

- *Runtime rig construction.* RuntimeRig2D is verified to construct an active ragdoll from real stroke geometry — segmenting the strokes, spawning a rigid body per segment, linking adjacent segments with joints, and identifying a body cluster with limbs attached at contact joints — for each rig_profile in the vocabulary. Joint stability is verified separately: for every enabled entity, the constructed rig is verified to reach and hold its gait targets without divergent oscillation, and a drawing that cannot be articulated is verified to degrade to a single intact body rather than to an unstable rig.

Model verification. The training procedure is verified to behave as designed. Training and validation accuracy are tracked across epochs to confirm convergence and to detect overfitting. The 80/10/10 partition is verified to follow a seeded shuffle so that the split is reproducible, and the partitions are audited to confirm that no data leakage occurs between them. Random affine augmentation is verified to be applied to the training split only, and never to validation or test data.

Integration verification. Because the recognition backend and the game runtime operate as separate processes within a monolithic, single-host system, their integration is verified through the local loopback interface. The researchers confirm that the canvas capture is correctly rasterized, Base64-encoded (Josefsson, 2006), and transmitted to the /predict endpoint; that the backend returns a well-formed response containing the predicted label, confidence value, and margin; and that the client correctly parses the response and either transforms the player, creates the corresponding utility object, or displays a redraw prompt. Loopback host binding, backend availability, request-size limits, malformed-response handling, and timeout behavior are also verified as

part of this process.

Following system integration, the complete prototype undergoes internal verification against its functional and non-functional requirements before external participants are involved. The researchers confirm that drawings are captured, recognized, mapped to the appropriate abilities, instantiated as the correct entities, and animated according to the system specifications. They also verify that the confidence and margin thresholds properly trigger the redraw prompt for uncertain predictions and that the levels can be completed using the implemented drawing mechanic.

4.10.3 Validation

Validation determines whether the *correct* system is being built, whether the system as constructed genuinely addresses the stated problem and satisfies the needs of its intended users. Whereas verification is internal and specification-oriented, validation is external and requires reference to the study's objectives and to the judgment of actual users. A system may satisfy every specification and nonetheless fail to constitute a playable game; validation guards against this outcome.

Four of these measures carry a stated criterion against which the system passes or fails; the remainder are reported descriptively. The criteria are a mean System Usability Scale score above 68, a median end-to-end response time at or below 250 milliseconds with a ninety-fifth percentile at or below 500 milliseconds, a frame rate not falling below fifty-five during a submission window, and a held-out per-class recall of at least 0.70 for any class on which a level-critical obstacle depends. No criterion is set for the remaining measures. Where a published benchmark exists for an instrument it is named and used; where none exists, scores are read against the scale midpoint and against the values reported in the instrument's source, and are interpreted alongside the telemetry rather than on their own.

Evaluation Aspects. The evaluation is organised around five aspects, so that algorithmic performance, practical usability, and effectiveness are each addressed by a named measure rather than treated together. Each aspect states what is measured, by what instrument, and how it is reported. Table 13 summarizes these aspects.

Table 13

The Five Evaluation Aspects, Their Measures, and Their Reporting

Aspects	Measure	Reported as
CNN Performance	Per-class precision and recall; confusion matrix; overall accuracy	Per class for all fifty with support counts; accuracy alongside, never instead
Recognition Speed	End-to-end response time from capture to entity appearance	Median and 95th percentile, in milliseconds
Gameplay Performance	Frame rate during submission; CPU utilisation; GPU utilisation; resident memory	Median and peak per session, on the reference machine
User Experience	SUS; UEQ-S; IEQ-SF; GEQ Core components; Godspeed Animacy; QUESI Familiarity	Means with 95% confidence intervals against published benchmarks
Gameplay Effective	Completion rate; time on level; redraw rate	Per session, reported within route

CNN Performance. Recognition performance is assessed by three metrics (accuracy, precision, and recall) defined in Equations 5 through 7, together with the confusion matrix of Equation 9. Precision and recall are reported per class for all fifty classes, and the confusion matrix shows which classes are mistaken for which. Overall accuracy is reported alongside these figures and never in place of them, because a single aggregate can appear healthy while individual classes fail; the three geometric primitives in particular are reported individually rather than absorbed into a fifty-class average. Overall accuracy is reported alongside these figures and never in place of them, because a single aggregate can appear healthy while individual classes fail; the three geometric primitives in particular are reported individually rather than absorbed into a fiftyclass average. Every figure is stated together with the sampling strategy and the per-class cap that produced it, and each per-class figure carries its support count, so that a recall computed from a handful of observations is not read as one computed from many. Figures are obtained from three sources, reported separately and never pooled: the held-out test split of the training corpus; the supplementary TU-Berlin set, with its forty-class coverage stated and the ten uncovered classes named individually; and live play, restricted to those obstacles that solicit a class by name and therefore record an intended label, with the number of classes so covered and the support count for each stated alongside.

Recognition Speed. Response time is measured end to end, as elapsed wall-clock time: from the capture of the drawing, through encoding, the hand-off to the recognition component over the loopback interface, preprocessing, inference, and the response, to the appearance of the spawned entity in the world. The request is dispatched asynchronously and its response handled in a callback, so the game runtime continues to render throughout this interval and no frame waits on the recognition component; the measured span is therefore the delay the

player perceives between finishing a drawing and seeing it act, not a period during which the game is suspended. It is reported as a median and a ninety-fifth percentile across a full evaluation session on the reference machine. The design targets, subject to empirical confirmation, are a median end-to-end response of 250 milliseconds or less and a ninety-fifth percentile of 500 milliseconds or less. Model inference time in isolation is not presented as the latency figure, because it is not the quantity the player experiences; where inference time is reported at all, it is reported as one component of the end-to-end figure and labelled as such.

Gameplay Performance. Gameplay performance is measured while the game runs, on the reference machine described in Section 4.1.1 (a MacBook Pro with an Apple M1 Max processor, 64 GB of unified memory, and 2 TB of storage) and separately on the Windows verification build. Four quantities are recorded.

Table 14

Gameplay Performance Measures and Their Instrumentation

Quantity	Definition	Instrumentation and Interval
Frame rate	Frames rendered per second, reported for the window spanning each submission — the interval during which recognition is in flight	Engine frame timer, every frame, aggregated per submission. Floor of 55 fps
CPU Utilization	Combined utilisation of the game process and the recognition process, as a percentage of one core-equivalent	Process-level sampling at 1 Hz for the duration of the session

Table 14 (Continued)
Gameplay Performance Measures and Their Instrumentation

Quantity	Definition	Instrumentation and Interval
GPU Utilization	Utilisation of the graphics processor while rendering	Platform GPU metrics sampling at 1 Hz
Memory Usage	Memory pool size for resident memory of the game process and the recognition backend, recorded separately	Process-level sampling at 1 Hz

Each quantity is reported as a median and a peak per session, with the two processes distinguished, so that the cost of recognition can be separated from the cost of rendering. The frame-rate measure carries a second purpose beyond describing performance: because recognition is dispatched asynchronously and no frame waits on its response, a frame rate that holds above the floor across the submission window is the evidence that this separation is in fact achieved. A frame rate that collapses during submissions would indicate that the recognition call is blocking the game runtime, whatever the code intends. Where a platform exposes no GPU counter, that is stated rather than estimated, and the measure is reported only for the platform that does. These figures characterise the system's demand on ordinary consumer hardware and are reported descriptively; no threshold is set for them beyond the frame-rate floor already specified in Section 4.10.2.

User Experience. User experience is measured with published, validated instruments, none of which was developed by the researchers. Usability is measured with the System Usability Scale (Brooke, 1996), ten items yielding a single score from 0 to 100, read against the conventional

above-average threshold of 68 (Sauro, 2011). Overall user experience is measured with the short version of the User Experience Questionnaire, the UEQ-S (Schrepp, Hinderks, & Thomaschewski, 2017), eight semantic-differential items on a seven-point scale yielding a pragmatic quality score and a hedonic quality score. Immersion is measured with the Immersion Experience Questionnaire — Short Form (Cutting, Aung, Perrett, & Cairns, 2025) and enjoyment with the Competence, Sensory and Imaginative Immersion, and Positive Affect components of the Game Experience Questionnaire Core Module (IJsselsteijn, de Kort, & Poels, 2013). Ability-expectation alignment is measured with the Perceived Familiarity subscale of the QUESI (Hurtienne & Naumann, 2010), and the believability of the animated drawing with the Animacy subscale of the Godspeed series (Bartneck, Kulić, Croft, & Zoghbi, 2009). All are reported as means with ninety-five per cent confidence intervals; none is subjected to significance testing between subgroups. Three carry a published interpretive reference and three do not. The System Usability Scale is read against the conventional above-average threshold of 68. UEQ-S scale means are read against the standard interpretation given in the UEQ Handbook, in which a value above 0.8 on the −3 to +3 range represents a positive evaluation, a value below −0.8 a negative one, and the interval between them a neutral evaluation, with pragmatic and hedonic quality read separately. QUESI Perceived Familiarity is read against the benchmark values reported by Naumann and Hurtienne (2010). For the IEQ-SF, the GEQ components, and the Godspeed Animacy subscale no population benchmark is published; these are read against the scale midpoint and against the values reported in their source papers, and the absence of a benchmark is stated rather than implied.

Two properties of the UEQ-S are disclosed. Its authors intend the short form for scenarios in which a full twenty-six-item UEQ cannot be administered, one of which is the inclusion of a user-experience measure

within an already lengthy questionnaire battery; that is the scenario here, the UEQ-S being the sixth instrument in the battery, and the full UEQ is therefore not administered. Its authors further caution that the overall mean of the eight items should not be read as a single user-experience indicator, because the relative importance of pragmatic and hedonic quality differs by product. The pragmatic and hedonic scores are accordingly reported and interpreted separately, and no combined UEQ-S score is reported.

Game Effectiveness. Effectiveness is measured behaviourally rather than by self-report. In-game telemetry records, for every session, the completion rate for each level attempted, the time spent on each level, and the redraw rate — the proportion of submitted drawings declined by the confidence and margin gate. All three are recorded for every session and reported alongside the route the participant took rather than averaged across routes, because the three routes differ by design in the amount of content they contain and in the classes they require. Completion is interpreted together with the per-class recall of the classes on that route, so that a level left incomplete can be attributed to a usability obstacle, to a low-recall class, or to the difficulty of the puzzle itself, rather than left unexplained.

Validation Activities Not Requiring Participants

Objective traceability. Each specific objective is traced to a corresponding implemented and demonstrable feature, ensuring that no objective remains unrealized and that no significant component lies outside the study's stated aims.

Ability-attribution validity. Each ability is validated by tracing it to its source. Each ability must correspond to a documented assertion under one of the six affordance-bearing relations listed in Table 9, or to an assertion on a named parent

concept reached through a guarded IsA traversal, with the mode of attribution recorded in Appendix C. Two classes — penguin and mushroom — carry no usable assertion at any depth, and their abilities are hand-authored and marked as such. This distinction is recorded rather than concealed: the study does not claim a single uniform derivation across all fifty classes, and the provenance of every ability, whether knowledge-graph-derived or hand-authored, is documented in the attribution table.

Player-expectation alignment. A drawing may be classified correctly and still be granted an ability that players find unintuitive. This is measured with the validated Perceived Familiarity subscale of the QUESI (Hurtienne & Naumann, 2010), which captures whether the ability behaved the way the player's prior knowledge of that entity would predict. Because QUESI Perceived Familiarity is the closest validated proxy for this novel construct rather than a purpose-built measure of it, it is interpreted alongside the per-class recall telemetry. This measure validates the attribution design rather than the classifier, and is especially pertinent for the primitive classes, whose mappings rely on structural ConceptNet relations rather than direct agentive CapableOf relations.

Animation believability. Because the artwork animated at runtime is the player's own drawing rather than an artwork prepared in advance, the resulting motion cannot be validated against a reference animation. It is validated instead by participant judgment using the six-item Animacy subscale of the Godspeed series (Bartneck et al., 2009), which rates whether the drawing read as alive, lively, organic, lifelike, interactive, and responsive rather than dead and mechanical. Because the Godspeed animacy and anthropomorphism indices correlate strongly ($r \approx .89$; Ho & MacDorman, 2010), the Animacy score is reported as a descriptive

indicator of believability. Observation additionally records the conditions under which procedural stroke animation degrades, notably disconnected strokes, unusual proportions, and drawings that do not conform to the expectations of the animation rules.

User Acceptance Testing. The system is validated through supervised play sessions with a minimum of thirty (30) participants drawn from the target audience. Participants play a defined set of OBRA levels under observation, and are observed to determine whether they can complete those levels using the drawing mechanic without external instruction beyond that provided in-game. While they play, the system automatically records the in-game telemetry described under the Gameplay-Evaluation Instruments in Section 4.2.1B, including level-completion rates, time-on-level, recognition results, confidence and margin values, redraw behaviour, end-to-end recognition response time, and the gameplay-performance measures of frame rate, processor and graphics utilisation, and resident memory. Where the intended class of a drawing is recorded, the gameplay data are also used to calculate per-class precision and recall and to construct a confusion matrix

Immediately after completing the assigned gameplay session, participants answer the System Usability Scale, the short version of the User Experience Questionnaire, the Immersion Experience Questionnaire — Short Form, the selected components of the Game Experience Questionnaire, the Godspeed Animacy subscale, and the QUESI Perceived Familiarity subscale. Participation is voluntary and based on informed consent, and every participant may withdraw at any point without penalty and without giving a reason. Each participant’s route arises from their own dialogue choices, and completion and time-on level are therefore reported within the route actually taken rather than pooled

across routes.

4.10.4 Testing

Testing refers to the evaluation of the OBRA system and subjecting the software product to various procedures to determine any issues and establish whether its components and the overall functionality perform as designed. This is also conducted continuously during development and at the completion of each iteration. The following testing activities will be conducted throughout the development of OBRA:

- **Unit testing** examines individual functions and modules in isolation.
- **Integration testing** examines communication between the game runtime and the recognition backend through the local loopback interface within the single-host system.
- **System testing** exercises the complete interaction end to end, from stroke capture through recognition, attribution, actor spawning, and procedural animation, for both the playable and obstacle spawn modes.
- **Performance testing** measures end-to-end response time, frame rate, and load time. Because the game runtime and recognition component operate within a monolithic, single-host system, response time is measured from the completion of the drawing to the appearance of the spawned actor, rather than from the model's inference time alone.
- **Compatibility testing** confirms that the exported builds operate correctly on both Windows and macOS.
- **Usability testing** is conducted during the thirty-participant play session.
- **Regression testing** is conducted at the close of each iteration to confirm that previously implemented functionality remains intact following the addition of new increments.

BIBLIOGRAPHY

Ali, S., Aslam, N., Kim, D., Abbas, A., Tufail, S., & Azhar, B. (2023). Context awareness based Sketch-DeepNet architecture for hand-drawn sketches classification and recognition in AIoT. *PeerJ Computer Science, 9*, Article e1186. https://doi.org/10.7717/peerj-cs.1186

Alghofaili, R., Sendhilnathan, N., Zhang, T., Grossman, T., Glueck, M., Jonker, T. R., & Lafreniere, B. (2024). Interactive mediation techniques for error-aware gesture input systems. In *Proceedings of Graphics Interface 2024* (pp. 1–12). ACM. https://doi.org/10.1145/3670947.3670964

Almario, V. S. (Ed.). (2010). *UP Diksiyonaryong Filipino* (2nd ed.). UP Sentro ng Wikang Filipino

Amershi, S., Begel, A., Bird, C., DeLine, R., Gall, H., Kamar, E., Nagappan, N., Nushi, B., & Zimmermann, T. (2019). Software engineering for machine learning: A case study. *2019 IEEE/ACM 41St International Conference on Software Engineering: Software Engineering in Practice (ICSE-SEIP)*. https://doi.org/10.1109/icse-seip.2019.00042

Amjoud, A. B., & Amrouch, M. (2023). Object detection using deep learning, CNNs and vision transformers: A review. *IEEE Access, 11*, 35479–35516. https://doi.org/10.1109/ACCESS.2023.3266093

Bartneck, C., Kulić, D., Croft, E., & Zoghbi, S. (2009). Measurement instruments for the anthropomorphism, animacy, likeability, perceived intelligence, and perceived safety of robots. International Journal of Social Robotics, 1(1), 71–81.

Bass, L., Clements, P., & Kazman, R. (2022). Software architecture in practice (4th ed.). Addison-Wesley.

Bernal, G., Bonilla, J., & Bellido, C. (1995). Ecological validity and cultural sensitivity for outcome research: Issues for the cultural adaptation and development of psychosocial treatments with Hispanics. Journal of Abnormal Child Psychology, 23(1), 67–82. https://doi.org/10.1007/BF01447045

Bhunia, A. K., Das, A., Muhammad, U. R., Yang, Y., Hospedales, T. M., Xiang, T., Gryaditskaya, Y., & Song, Y.-Z. (2020). Pixelor. *ACM Transactions on Graphics, 39*(6), 1–15. https://doi.org/10.1145/3414685.3417840

Bray, T. (Ed.). (2017). *The JavaScript Object Notation (JSON) Data Interchange Format* (RFC 8259). Internet Engineering Task Force. https://doi.org/10.17487/RFC8259

Breck, E., Cai, S., Nielsen, E., Salib, M., & Sculley, D. (2017). *The ML test score: A rubric for ML production readiness and technical debt reduction*. Research.Google. https://research.google/pubs/pub46555/

Brooke, J. (1996). SUS: A quick and dirty usability scale. In P. W. Jordan, B. Thomas, B. A. Weerdmeester, & A. L. McClelland (Eds.), *Usability evaluation in industry* (pp. 189–194). Taylor & Francis.

Bujang, M. A., Omar, E. D., & Baharum, N. A. (2018). A review on sample size determination for Cronbach's alpha test: A simple guide for researchers. The Malaysian Journal of Medical Sciences, 25(6), 85–99.

Cerda, C. M. C. (2021). Towards a "Filipino" video game: Teaching Filipino culture and identity for video game development. *Transactions of the Digital Games Research Association, 5*(3), 43–64. https://doi.org/10.26503/todigra.v5i3.122

Clover Studio. (2006). *Ōkami* [Video game]. Capcom.

Cormen, T. H., Leiserson, C. E., Rivest, R. L., & Stein, C. (2022). *Introduction to Algorithms, fourth edition*. MIT Press.

Cronbach, L. J. (1951). Coefficient alpha and the internal structure of tests. *Psychometrika, 16*(3), 297–334. https://doi.org/10.1007/BF02310555

Cutting, J., Aung, M., Perrett, F., & Cairns, P. A. (2025). The development and validation of the Immersion Experience Questionnaire Short Form (IEQ-SF). *International Journal of Human-Computer Studies, 203*, Article 103580. https://doi.org/10.1016/j.ijhcs.2025.103580

da Silva, G. F., Brasileiro, F., Lopes, R., Morais, F., Carvalho, M., & Turull, D. (2020). QoS-driven scheduling in the cloud. *Journal of Internet Services and Applications*, *11*(1). https://doi.org/10.1186/s13174-020-00129-0

Desurvire, H., Caplan, M., & Toth, J. A. (2004). Using heuristics to evaluate the playability of games. In CHI '04 Extended Abstracts on Human Factors in Computing Systems (pp. 1509–1512). Association for Computing Machinery. https://doi.org/10.1145/985921.986102

Desurvire, H., & Wiberg, C. (2009). Game usability heuristics (PLAY) for evaluating and designing better games: The next iteration. In A. A. Ozok & P. Zaphiris (Eds.), Online Communities and Social Computing (Lecture Notes in Computer Science, Vol. 5621, pp. 557–566). Springer.

Desurvire, H., & Wixon, D. (2013). Game principles: Change, choice & creation — Making better games. In CHI ’13 Extended Abstracts on Human Factors in Computing Systems. Association for Computing Machinery.

Dosovitskiy, A., Beyer, L., Kolesnikov, A., Weissenborn, D., Zhai, X., Unterthiner, T., Dehghani, M., Minderer, M., Heigold, G., Gelly, S., Uszkoreit, J., & Houlsby, N. (2021). An image is worth 16x16 words: Transformers for

image recognition at scale. *International Conference on Learning Representations (ICLR)*. https://openreview.net/forum?id=YicbFdNTTy

Duan, H., Huang, Y., Zhao, Y., Huang, Z., & Cai, W. (2022). User-generated content and editors in video games: Survey and vision. In *2022 IEEE Conference on Games (CoG)* (pp. 536–543). IEEE. https://doi.org/10.1109/CoG51982.2022.9893717

Eitz, M., Hays, J., & Alexa, M. (2012). How do humans sketch objects? *ACM Transactions on Graphics, 31*(4), Article 44. https://doi.org/10.1145/2185520.2185540

FastAPI. (n.d.). *FastAPI documentation*. https://fastapi.tiangolo.com/

Fernandez-Fernandez, R., Victores, J., Estevez, D., & Balaguer, C. (2019). *Quick, Stat!: A statistical analysis of the Quick, Draw! dataset*. arXiv. https://doi.org/10.48550/arXiv.1907.06417

Fielding, R. T. (2000). *Architectural styles and the design of network-based software architectures* [Doctoral dissertation, University of California, Irvine]. https://www.ics.uci.edu/~fielding/pubs/dissertation/top.htm

5th Cell. (2009). *Scribblenauts* [Video game]. Warner Bros. Interactive Entertainment.

Geijtenbeek, T., & Pronost, N. (2012). Interactive character animation using simulated physics: A state-of-the-art review. Computer Graphics Forum, 31(8), 2492–2515. https://doi.org/10.1111/j.1467-8659.2012.03189.x

Godot Engine contributors. (n.d.-a). *2D sprite animation*. Godot Engine documentation. https://docs.godotengine.org/en/stable/tutorials/2d/2d_sprite_animation.html

Godot Engine contributors. (n.d.-b). *Godot Engine documentation*. https://docs.godotengine.org/

Godot Engine contributors. (n.d.-c). *Nodes and scenes*. Godot Engine documentation. https://docs.godotengine.org/en/stable/getting_started/step_by_step/nodes_and_scenes.html

Godot Engine contributors. (n.d.-d). *Tween*. Godot Engine documentation. https://docs.godotengine.org/en/stable/classes/class_tween.html

Goodfellow, I., Bengio, Y., & Courville, A. (2016). Deep learning. In *Deeplearningbook.org*. MIT Press. https://www.deeplearningbook.org/

Google Creative Lab. (n.d.). *The Quick, Draw! dataset*. GitHub. https://github.com/googlecreativelab/quickdraw-dataset

Google Creative Lab. (2016). *Quick, Draw!* [Video game]. https://quickdraw.withgoogle.com/

Ha, D., & Eck, D. (2017). *A neural representation of sketch drawings*. arXiv. https://doi.org/10.48550/arXiv.1704.03477

Han, K., Wang, Y., Chen, H., Chen, X., Guo, J., Liu, Z., Tang, Y., Xiao, A., Xu, C., Xu, Y., Yang, Z., Zhang, Y., & Tao, D. (2023). A survey on vision transformer. *IEEE Transactions on Pattern Analysis and Machine Intelligence, 45*(1), 87–110. https://doi.org/10.1109/TPAMI.2022.3152247

He, K., Zhang, X., Ren, S., & Sun, J. (2016). Deep residual learning for image recognition. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)* (pp. 770–778). https://doi.org/10.1109/CVPR.2016.90

Henderson, J., Jonker, T. R., Lank, E., Wigdor, D., & Lafreniere, B. (2022). Investigating cross-modal approaches for evaluating error acceptability of a recognition-based input technique. *Proceedings of the ACM on Interactive, Mobile, Wearable and Ubiquitous Technologies, 6*(1), Article 15. https://doi.org/10.1145/3517262

Hertzog, M. A. (2008). Considerations in determining sample size for pilot studies. Research in Nursing & Health, 31(2), 180–191. https://doi.org/10.1002/nur.20247

Hill, R. (1998). What sample size is “enough” in internet survey research? Interpersonal Computing and Technology, 6(3–4).

Ho, C.-C., & MacDorman, K. F. (2010). Revisiting the uncanny valley theory: Developing and validating an alternative to the Godspeed indices. Computers in Human Behavior, 26(6), 1508–1518.

Hurtienne, J., & Naumann, A. (2010). QUESI — A questionnaire for measuring the subjective consequences of intuitive use. In *Interdisciplinary College 2010: Focus theme — Play, act and learn* (p. 536).

Hwang, J. D., Bhagavatula, C., Le Bras, R., Da, J., Sakaguchi, K., Bosselut, A., & Choi, Y. (2021). (Comet-)Atomic 2020: On symbolic and neural commonsense knowledge graphs. *Proceedings of the AAAI Conference on Artificial Intelligence, 35*(7), 6384–6392. https://doi.org/10.1609/aaai.v35i7.16792

IJsselsteijn, W. A., de Kort, Y. A. W., & Poels, K. (2013). *The Game Experience Questionnaire*. Technische Universiteit Eindhoven. https://research.tue.nl/en/publications/the-game-experience-questionnaire/

Ilievski, F., Szekely, P., & Zhang, B. (2021). CSKG: The CommonSense Knowledge Graph. In *The Semantic Web (ESWC 2021)* (pp. 680–696). Springer. https://doi.org/10.1007/978-3-030-77385-4_41

Isaac, S., & Michael, W. B. (1995). Handbook in research and evaluation (3rd ed.). Educational and Industrial Testing Services.

Jennett, C., Cox, A. L., Cairns, P., Dhoparee, S., Epps, A., Tijs, T., & Walton, A. (2008). Measuring and defining the experience of immersion in games. *International Journal of Human-Computer Studies, 66*(9), 641–661. https://doi.org/10.1016/j.ijhcs.2008.04.004

Johanson, G. A., & Brooks, G. P. (2010). Initial scale development: Sample size for pilot studies. Educational and Psychological Measurement, 70(3), 394–400. https://doi.org/10.1177/0013164409355692

John, T. (2009, June 18). *How innovative Scribblenauts recognizes 10,000 different words*. WIRED. https://www.wired.com/2009/06/scribblenauts/

Josefsson, S. (2006). *The Base16, Base32, and Base64 data encodings* (RFC 4648). Internet Engineering Task Force. https://datatracker.ietf.org/doc/html/rfc4648

Keebler, J. R., Shelstad, W. J., Smith, D. C., Chaparro, B. S., & Phan, M. H. (2020). Validation of the GUESS-18: A short version of the Game User Experience Satisfaction Scale (GUESS). *Journal of Usability Studies, 16*(1), 49–62.

Kocielnik, R., Amershi, S., & Bennett, P. N. (2019). Will you accept an imperfect AI? Exploring designs for adjusting end-user expectations of AI systems. In *Proceedings of the 2019 CHI Conference on Human Factors in Computing Systems* (Paper 411). ACM. https://doi.org/10.1145/3290605.3300641

Krizhevsky, A., Sutskever, I., & Hinton, G. E. (2012). ImageNet classification with deep convolutional neural networks. *Advances in Neural Information Processing Systems, 25*, 1097–1105.

Laugwitz, B., Held, T., & Schrepp, M. (2008). Construction and evaluation of a user experience questionnaire. In A. Holzinger (Ed.), HCI and usability for education and work (USAB 2008), Lecture Notes in Computer Science (Vol. 5298, pp. 63–76). Springer. https://doi.org/10.1007/978-3-540-89350-9_6

Law, E. L.-C., Brühlmann, F., & Mekler, E. D. (2018). Systematic review and validation of the Game Experience Questionnaire (GEQ) – Implications for citation and reporting practice. In Proceedings of the 2018 Annual Symposium on Computer-Human Interaction in Play (CHI PLAY ’18) (pp. 257–270). ACM. https://doi.org/10.1145/3242671.3242683

LeCun, Y., Bengio, Y., & Hinton, G. (2015). Deep learning. *Nature, 521*(7553), 436–444. https://doi.org/10.1038/nature14539

Lee, S. H., Lee, S., & Song, B. C. (2021). *Vision transformer for small-size datasets*. arXiv. https://doi.org/10.48550/arXiv.2112.13492

Lewis, J. R., & Sauro, J. (2009). The factor structure of the System Usability Scale. In M. Kurosu (Ed.), Human Centered Design (HCII 2009). Lecture Notes in Computer Science (Vol. 5619, pp. 94–103). Springer. https://doi.org/10.1007/978-3-642-02806-9_12

Liu, C. L., & Layland, J. W. (1973). Scheduling algorithms for multiprogramming in a hard-real-time environment. *Journal of the ACM*, *20*(1), 46–61. https://doi.org/10.1145/321738.321743

Liu, Z., Mao, H., Wu, C.-Y., Feichtenhofer, C., Darrell, T., & Xie, S. (2022). A ConvNet for the 2020s. In *2022 IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)* (pp. 11966–11976). https://doi.org/10.1109/CVPR52688.2022.01167

Maurício, J., Domingues, I., & Bernardino, J. (2023). Comparing vision transformers and convolutional neural networks for image classification: A

literature review. *Applied Sciences, 13*(9), Article 5521. https://doi.org/10.3390/app13095521

Miller, R. B. (1968). Response time in man-computer conversational transactions. In *Proceedings of the AFIPS Fall Joint Computer Conference* (Vol. 33, pp. 267–277). https://doi.org/10.1145/1476589.1476628

Mnih, V., Kavukcuoglu, K., Silver, D., Rusu, A. A., Veness, J., Bellemare, M. G., Graves, A., Riedmiller, M., Fidjeland, A. K., Ostrovski, G., Petersen, S., Beattie, C., Sadik, A., Antonoglou, I., King, H., Kumaran, D., Wierstra, D., Legg, S., & Hassabis, D. (2015). Human-level control through deep reinforcement learning. Nature, 518(7540), 529–533. https://doi.org/10.1038/nature14236

Naumann, A., & Hurtienne, J. (2010). Benchmarks for intuitive interaction with mobile devices. In Proceedings of MobileHCI 2010 (pp. 401–402). ACM.

Nguyen, T.-P., Razniewski, S., & Weikum, G. (2021). Advanced semantics for commonsense knowledge extraction. In *Proceedings of the Web Conference 2021* (pp. 2636–2647). https://doi.org/10.1145/3442381.3449827

Nielsen, J. (1993). *Usability engineering*. Academic Press.

Nielsen, J. (1994). Heuristic evaluation. In J. Nielsen & R. L. Mack (Eds.), *Usability inspection methods* (pp. 25–62). John Wiley & Sons.

Nielsen, J., & Molich, R. (1990). Heuristic evaluation of user interfaces. In Proceedings of the SIGCHI Conference on Human Factors in Computing Systems (pp. 249–256). Association for Computing Machinery.

Nintendo EAD. (2013). Super Mario 3D World [Video game]. Nintendo.

Olsen, L., Samavati, F. F., Sousa, M. C., & Jorge, J. A. (2009). Sketch-based modeling: A survey. *Computers & Graphics, 33*(1), 85–103. https://doi.org/10.1016/j.cag.2008.09.013

ONNX Runtime. (n.d.). *ONNX Runtime documentation*. https://onnxruntime.ai/docs/

Paszke, A., Gross, S., Massa, F., Lerer, A., Bradbury, J., Chanan, G., Killeen, T., Lin, Z., Gimelshein, N., Antiga, L., Desmaison, A., Köpf, A., Yang, E., DeVito, Z., Raison, M., Tejani, A., Chilamkurthy, S., Steiner, B., Fang, L., . . . Chintala, S. (2019). PyTorch: An Imperative Style, High-Performance Deep Learning Library. *arXiv (Cornell University)*, *32*, 8026–8037. https://doi.org/10.48550/arxiv.1912.01703

Picca, D., Jaccard, D., & Eberlé, G. (2015). Natural language processing in serious games: A state of the art. *International Journal of Serious Games, 2*(3). https://doi.org/10.17083/ijsg.v2i3.87

Pressman, R. S. (2010). *Software engineering: A practitioner's approach* (7th ed.). McGraw-Hill Science, Engineering & Mathematics.

Python Software Foundation. (n.d.). *Python documentation*. https://docs.python.org/

PyTorch Contributors. (n.d.). *PyTorch documentation*. https://docs.pytorch.org/

PyTorch Contributors. (2023). *Introduction to ONNX*. Pytorch.Org. https://docs.pytorch.org/tutorials/beginner/onnx/intro_onnx.html?utm_source=chatgpt.com

Ramírez, S. (n.d.). *FastAPI*. Retrieved August 4, 2026, from https://fastapi.tiangolo.com/

Real Academia Española. (2014). *Diccionario de la lengua española* (23rd ed.).

Sap, M., Le Bras, R., Allaway, E., Bhagavatula, C., Lourie, N., Rashkin, H., Roof, B., Smith, N. A., & Choi, Y. (2019). ATOMIC: An atlas of machine commonsense for if-then reasoning. *Proceedings of the AAAI Conference on Artificial Intelligence, 33*(1), 3027–3035. https://doi.org/10.1609/aaai.v33i01.33013027

Sauro, J. (2011). *A practical guide to the System Usability Scale: Background, benchmarks & best practices*. Measuring Usability LLC

Schrepp, M. (2023). User experience questionnaire handbook (Version 11). https://www.ueq-online.org/Material/Handbook.pdf

Schrepp, M., Hinderks, A., & Thomaschewski, J. (2017). Design and evaluation of a short version of the User Experience Questionnaire (UEQ-S). International Journal of Interactive Multimedia and Artificial Intelligence, 4(6), 103–108. https://doi.org/10.9781/ijimai.2017.09.001

Sculley, D., Holt, G., Golovin, D., Davydov, E., Phillips, T., Ebner, D., Chaudhary, V., Young, M., Crespo, J.-F., & Dennison, D. (2015). Hidden technical debt in machine learning systems. *Advances in Neural Information Processing Systems*, *28*.https://papers.nips.cc/paper/5656-hidden-technical-debt-in-machine-learning-systems

Shneiderman, B. (2007). Creativity support tools: Accelerating discovery and innovation. *Communications of the ACM, 50*(12), 20–32. https://doi.org/10.1145/1323688.1323689

Smith, H. J., Zheng, Q., Li, Y., Jain, S., & Hodgins, J. K. (2023). *A method for animating children's drawings of the human figure*. arXiv. https://doi.org/10.48550/arXiv.2303.12741

Sokolova, M., & Lapalme, G. (2009). A systematic analysis of performance measures for classification tasks. Information Processing & Management, 45(4), 427–437. https://doi.org/10.1016/j.ipm.2009.03.002

Speer, R., Chin, J., & Havasi, C. (2017). ConceptNet 5.5: An open multilingual graph of general knowledge. *Proceedings of the Thirty-First AAAI Conference on Artificial Intelligence*, 4444–4451.

Stroessner, S. J. (2020). On the social perception of robots: measurement, moderation, and implications. Living with Robots, 21–47. https://doi.org/10.1016/b978-0-12-815367-3.00002-5

Summerville, A., Snodgrass, S., Guzdial, M., Holmgård, C., Hoover, A. K., Isaksen, A., Nealen, A., & Togelius, J. (2018). Procedural content generation via machine learning (PCGML). *IEEE Transactions on Games, 10*(3), 257–270. https://doi.org/10.1109/TG.2018.2846639

Tullis, T. S., & Stetson, J. N. (2004). A comparison of questionnaires for assessing website usability. Proceedings of the Usability Professionals Association Conference.

Xu, P., Hospedales, T. M., Yin, Q., Song, Y.-Z., Xiang, T., & Wang, L. (2023). Deep learning for free-hand sketch: A survey. *IEEE Transactions on Pattern Analysis and Machine Intelligence, 45*(1), 285–312. https://doi.org/10.1109/TPAMI.2022.3148853

Xu, P., Joshi, C. K., & Bresson, X. (2022). Multigraph transformer for free-hand sketch recognition. *IEEE Transactions on Neural Networks and Learning Systems, 33*(10), 5150–5161. https://doi.org/10.1109/TNNLS.2021.3069230

Yannakakis, G. N., & Togelius, J. (2018). *Artificial intelligence and games*. Springer. https://doi.org/10.1007/978-3-319-63519-4

Yin, K., Loken, K., & van de Panne, M. (2007). SIMBICON: Simple biped locomotion control. ACM Transactions on Graphics, 26(3), Article 105. https://doi.org/10.1145/1276377.1276509

Yonatan Geifman, & Ran El-Yaniv. (2017). Selective classification for deep neural networks. *arXiv (Cornell University)*.

https://doi.org/10.48550/arxiv.1705.08500

Yu, Q., Yang, Y., Song, Y., Xiang, T., & Hospedales, T. (2015). Sketch-a-Net that Beats Humans. *arXiv (Cornell University)*. https://doi.org/10.48550/arxiv.1501.07873

Zhang, H., Khashabi, D., Song, Y., & Roth, D. (2020). TransOMCS: From linguistic graphs to commonsense knowledge. *Proceedings of the Twenty-Ninth International Joint Conference on Artificial Intelligence*, 4004–4010.

Zhu, X., Yang, X., Zheng, S., Zhang, Z., Gao, F., Huang, J., & Chen, J. (2025). *Vector sketch animation generation with differentiable motion trajectories*. arXiv. https://doi.org/10.48550/arXiv.2509.25857

APPENDICES

Appendix A: Informed Consent Form

UNIVERSITY OF SAN CARLOS

Department of Computer, Information Sciences and Mathematics

Talamban Campus, Cebu City, Philippines

INFORMED CONSENT

This informed consent form is for students and other members of the University of San Carlos campus community, eighteen (18) years of age or older, who are being invited to take part in a supervised session in which they play a prototype computer game and afterwards answer a set of short standardised questionnaires about their experience of it. They are invited to participate in this study titled “OBRA: Real-Time Drawing Recognition Using CNN and ConceptNet in a 2D Adventure Game.”

Name of Researcher(s)/ Proponent(s):	Lady Kaina V. Gutierrez, Rusk Rye Ioan C. Alarde, Erika Bianca E. Api, Kent Jester P. Gatchalian
Name of Organization/ Institution:	Department of Computer, Information Sciences and Mathematics, University of San Carlos
Title of the Study and Version number:	OBRA: Real-Time Drawing Recognition Using CNN and ConceptNet in a 2D Adventure Game — ver. 1/15 August 2026

This Informed Consent Form has two parts:

- Information Sheet (to share information about the study with you)
- Certificate of Consent (for signatures if you choose to participate)

PART I: INFORMATION SHEET

Introduction

We are Lady Kaina V. Gutierrez, Rusk Rye Ioan C. Alarde, Erika Bianca E. Api, and Kent Jester P. Gatchalian, fourth-year Computer Science students of the University of San Carlos. We are doing research on “OBRA: Real-Time Drawing Recognition Using CNN and ConceptNet in a 2D Adventure Game,” under the supervision of our faculty adviser, Christine F. Peña, MMath, of the Department of Computer, Information Sciences and Mathematics. We are going to give you information about the study and invite you to take part in this research. You do not have to decide today whether or not you will participate. Before you decide, you may talk to anyone you feel comfortable with about the research.

This consent form may contain words that you do not understand. Please ask us to stop as we go through the information and we will take time to explain. If you have questions later, you may ask them of any of the researchers named at the end of this form.

Purpose of the Research

OBRA is a single-player adventure game in which you draw by hand on a canvas inside the game. The game looks at what you drew, works out what it is, and turns it into a creature or an object that then has an ability of its own — a drawn bird can fly, a drawn ladder can be climbed, and so on. The abilities are not written by us one by one; they are taken from a public store of everyday common-sense knowledge about what things can do and what things are used for.

We want to find out two things. First, whether the game can recognise ordinary freehand drawings reliably enough for it to actually be playable by someone who is not an artist. Second, whether people find the resulting game easy to use, absorbing to play, and satisfying — and whether the ability the game grants matches what they expected the thing they drew to be able to do. These are questions that cannot be answered by testing the software on its own. They can only be answered by people who sit down and play it. That is why we are inviting you.

Type of Research Intervention

This research involves one supervised session in which you play a defined portion of a prototype computer game, followed by a set of six short written questionnaires about your experience of playing it. There is no experiment

performed on you, no medical or physical procedure of any kind, and no interview. While you play, the game itself records information about what happened in the game, as described under Procedures below.

Participant Selection

You are being invited to take part in this research because you are a currently enrolled student of the University of San Carlos, or a member of the same campus community, and you are eighteen (18) years of age or older. Participants are drawn at random from a pool of volunteers who answered a short screening questionnaire, and are grouped by how confident they said they were about drawing and how familiar they said they were with games. This grouping is deliberate: we need the game to be tested by people who feel hesitant about drawing as well as by people who feel confident, because a game that only works for confident drawers has not solved the problem we set out to solve.

You do not need any drawing skill to take part, and you do not need to have played games before. The game does not assume that you draw well. This study is not a test of your drawing ability, and nothing you draw will be judged as good or bad.

You are not eligible to take part if you are one of the researchers or the faculty adviser, if you have already seen the level design, the list of drawable things, or the table of abilities, if you have already taken part in a pilot session on the same version of the game, or if an uncorrected visual impairment or a motor impairment would prevent you from drawing on screen with a mouse or trackpad. This last exclusion is not a judgement about who the game is for. It is because this particular study measures the drawing mechanic itself rather than how accessible it is, and we have recorded that as a limitation of the study.

Voluntary Participation

Your participation in this study is entirely voluntary. It is your choice whether to participate or not. The choice that you make will have no bearing whatsoever on your grades, your academic standing, your relationship with the researchers, or your standing with the Department. The researchers are fellow students and have no influence over your marks. You may change your mind later and stop participating even if you agreed earlier, and you do not have to give a reason.

Procedures

If you agree to take part, the following will happen, in this order:

1. Presentation of the Informed Consent Form. A researcher will go through this form with you, answer any questions you have, and give you a copy to keep. You will be asked to sign the Certificate of Consent in Part II only if you decide to take part.
2. Short pre-session form. Before you play, you will answer a few background questions — your age band, your programme of study, how confident you feel about drawing, how familiar you are with games, and whether you have played drawing-based games before. This is asked before you play so that your answers are not coloured by the session. It does not ask for your name or any other identifying detail.
3. Gameplay. You will play a defined portion of OBRA on a laboratory computer for approximately forty-five (45) minutes, drawing on the in-game canvas to get past obstacles and move through the levels. A researcher will be present the whole time. You may ask for help, take a break, or stop at any point.
4. Collection of data. Two kinds of information are collected. The first is what you write on the questionnaires. The second is recorded automatically by the game while you play, under a randomly generated session code: which drawings you submitted and whether each was recognised or declined, how confident the recognition was, your progress through the levels, the time you spent, the dialogue choices you made, the items you collected, and how quickly the game responded to each drawing. Your drawings themselves are not kept — they exist only while you are playing and are discarded at the end of your session. Nothing recorded during the session is uploaded or sent anywhere: the game runs entirely on the laboratory computer, makes no internet connection while you play, and all data stays on that machine.
5. Questionnaires. Immediately after you finish playing, you will answer six short standardised questionnaires — fifty-four (54) items in total — about how easy the game was to use, how absorbed you felt while playing, what the overall experience was like, how lifelike the animated drawings appeared, and whether the abilities the game gave your drawings matched what you expected. All items are answered on simple rating scales. There are no right or wrong answers, and none of the questions is personal, sensitive, or medical. If you do not wish to answer a particular item, you may skip it and move on to the next.
6. Processing of data. Your questionnaire answers are encoded and combined with the answers of the other participants. Results are reported only as group averages and totals. Your session code is used

to link your questionnaire answers to your game record, and to nothing else.

7. Storage of data. All research data are stored in encrypted, password-protected files on computers belonging to the researchers, accessible only to the four researchers and the faculty adviser. Your signed consent form bears your name and is therefore stored separately, in a locked cabinet, and never alongside your session code. A logbook is kept of any transfer or sharing of the data.
8. Disposal of data. Three (3) years after the completion of the study, that is, by 31 December 2029, all electronic research data will be irreversibly deleted, including from the recycle bin, and all paper documents, including the signed consent forms, will be vertically shredded.

Duration

Your involvement in this research consists of a single session lasting approximately sixty (60) minutes in total: about forty-five (45) minutes of gameplay and about fifteen (15) minutes for the pre-session form and the questionnaires. There is no follow-up session, and you will not be contacted again after your session unless you ask us to be. Data collection for the study as a whole runs from 21 September 2026 to 26 September 2026. All soft and hard copies of the data collected will be irreversibly destroyed three (3) years after the completion of the study, by 31 December 2029, as described under Procedures and Confidentiality.

Risks (Including possible discrimination, burden, discomforts, and inconveniences)

There are five acknowledged categories of risk generally associated with participation in research studies: physical, psychological, social, economic, and legal. The researchers foresee minimal risk in this study, and none of it exceeds the risks of ordinary computer use. There are no foreseen physical, economic, or legal risks. Being discriminated against has not been seen as a possibility as a result of your participation.

Two minor discomforts are possible and are disclosed to you here:

- Mild frustration. The game will sometimes fail to recognise a drawing and will ask you to draw it again. This is expected, and how often it happens is one of the things we are measuring — it is a fact about the software, not about you. Redrawing costs you nothing inside the game, hints are

available, and no obstacle in the game can be passed only by drawing one specific thing.

- Fatigue or eye strain, as with any period of screen use. You may pause, rest, or stop whenever you wish.

You will not be deceived at any point in this study, and you will not be asked any personal, sensitive, or medical questions. If you experience any discomfort, you may skip any question, take a break, or withdraw from the study; and if the discomfort continues after the session you may contact the faculty adviser using the contact details given below.

Foreseeable circumstances and reasons under which participation in the study may be terminated

Your participation in the session may be terminated before it is complete in any of the following circumstances:

- you decide to stop, for any reason or for no reason at all;
- you appear to be in distress, discomfort, or fatigue, in which case the researcher present will pause the session and ask whether you wish to continue, reschedule, or stop;
- the game software fails in a way that cannot be recovered during the session, or the laboratory computer becomes unusable;
- the laboratory becomes unavailable, or the session is interrupted by circumstances outside the researchers' control, such as a power interruption or a suspension of classes.

If your session is terminated for any of these reasons, the partial record from that session is discarded and excluded from the analysis, unless you tell us that you are willing for it to be kept. You will not be asked to give a reason, and there is no penalty of any kind.

Benefits

There is no direct benefit to you from taking part, although many people find playing the game enjoyable. Your participation contributes to research on how automatic drawing recognition and everyday common-sense knowledge can be combined to make a game controlled by drawing rather than by buttons, and to the small body of locally produced work on Filipino-developed games. The findings will be of use to other researchers in drawing recognition and human–computer interaction, and to independent and small-scale game studios, particularly in the Philippines.

Reimbursements/Compensation

You will not be given any incentive to take part in this research. However, in recognition of your time, each participant will receive a token of one hundred pesos (₱100.00), and light refreshments will be served during the session. No travel costs are reimbursed, because the session takes place on campus at a time of your choosing. Because this study involves no physical or medical procedure and no foreseeable physical risk, no compensation or medical treatment for research-related injury is provided; should any untoward incident nevertheless occur during a session, the researchers will assist you in reaching the University clinic and will report the incident to the faculty adviser and to the University of San Carlos Research Ethics Committee.

Confidentiality

Your personal information is protected under the Data Privacy Act of 2012 (Republic Act No. 10173). This study is designed so that as little personal information as possible is collected in the first place, rather than collected and then stripped out afterwards.

- Your name, student number, e-mail address, and any other identifying detail are never entered into the game and are never written on any questionnaire.
- Instead of your name, a randomly generated session code identifies your record. Your game record and your questionnaire answers are linked to each other by that code and by nothing else.
- This signed consent form is the only document that bears your name. It is kept in a locked cabinet, separately from all research data, and your name is never written next to your session code. No document anywhere associates the two.
- All computer-based research files are encrypted or password-protected, and access is limited to the four student researchers named in this form and their faculty adviser. Any unforeseen or incidental data arising during the study is stored in a separate folder under the same protections. A logbook is kept of any event of data transfer or sharing.
- Results are reported in aggregate — as group averages and totals. No individual participant is identifiable in the thesis, in any presentation, or in any publication arising from it.
- Three (3) years after the completion of the study, by 31 December 2029, all electronic data will be hard-deleted, including from the recycle bin,

and all hard copies, including signed consent forms, will be vertically shredded.

There is one limit to confidentiality that we want you to be aware of. Sessions take place in a shared computer laboratory, so other people may see that you have taken part. We ask everyone present to respect one another's privacy, and we do not discuss any participant's performance with anyone; but we cannot prevent another person from noticing that you were there. If this is a concern for you, please tell the researcher and we will schedule your session at a quieter time.

Sharing the Result

Nothing you tell us in the questionnaires will be shared with anybody outside the research team, and nothing will be attributed to you by name. The knowledge we gain from this research will be shared with you and the other participants before it is made widely available: any participant who asks will be sent a short summary of the results. The findings will be presented in our undergraduate thesis defence and deposited in the University of San Carlos library. The researchers may also present the results at local or international conferences or publish them in research journals so that other interested people may learn from the research. In every case the data appear only in aggregate.

Right to Refuse or Withdraw

You do not have to take part in this research if you do not wish to do so. Your participation is entirely voluntary, and choosing to participate or not will not affect your grades, your academic standing, or your relationship with the Department in any way. You may stop the session at any time that you wish, and you may skip any questionnaire item you would rather not answer.

You may also withdraw after the session has ended. At the end of your session you will be given a slip of paper bearing your session code. Please keep it. Because your name is never recorded alongside your data, that code is the only way we can find your record. If you later decide you want your data destroyed, quote the code to any of the contacts named below on or before 26 September 2026, and your record will be deleted and excluded from the analysis, with no questions asked and no penalty of any kind. After that date the data will have been pooled for analysis and your individual record can no longer be singled out.

Who to Contact

You can ask us any more questions about any part of the research study, if you wish to. Do you have any questions?

If you wish to ask questions later or in the event of any research-related injury, you may contact any of the following:

Lady Kaina V. Gutierrez
Lead researcher — BS Computer Science, University of San Carlos
Email: 24100535@usc.edu.ph
Mobile Number: 09310743171

Christine F. Peña, MMath
Faculty adviser — Department of Computer, Information Sciences and Mathematics, University of San Carlos
Email: cfpena@usc.edu.ph

This study has undergone ethics review by the University of San Carlos Research Ethics Committee and may be reached regarding rights of the study participants, including grievances and complaints through the contact information below:

Ruby S. Suazo, Ph.D.
Panel Chair, USC Research Ethics Committee
University of San Carlos
Nasipit, Talamban, Cebu City
Email: rec@usc.edu.ph
Landline: (032) 401 2300 local 204

PART II: CERTIFICATE OF CONSENT

I have been invited to participate in research about a computer game controlled by freehand drawing, in which the drawings a player makes are automatically recognised and given abilities drawn from a store of everyday common-sense knowledge.

I have read the foregoing information, or it has been read to me. I have had the opportunity to ask questions about it and any questions I have been asked have been answered to my satisfaction. I consent voluntarily to be a participant in this study.

Respondent No. _____

Print Name of Participant _____

Signature of Participant _____

Phone Number _____

Email _____

Postal Address _____

Date _____

Statement by the researcher/person taking consent

I have accurately read out the information sheet to the potential participant, and to the best of my ability made sure that the participant understands that the following will be done:

1. The participant will play a defined portion of the OBRA prototype for approximately forty-five (45) minutes on a laboratory computer, under observation, and will afterwards answer six short standardised questionnaires totalling fifty-four (54) items.
2. While the participant plays, the game will automatically record gameplay information — drawings submitted and whether each was recognised,

progress, timings, dialogue choices, and response times — under a randomly generated session code and not under the participant's name. The participant's drawings themselves will not be kept beyond the session, and no data will leave the laboratory computer.

3. The participant's responses will be reported only in aggregate; the signed consent form will be stored separately from all research data; and all data and consent forms will be irreversibly destroyed three (3) years after the completion of the study.

I confirm that the participant was given an opportunity to ask questions about the study, and all the questions asked by the participant have been answered correctly and to the best of my ability. I confirm that the individual has not been coerced into giving consent, and the consent has been given freely and voluntarily.

A copy of this ICF has been provided to the participant.

Print Name of Researcher/person taking the consent

_____

Signature of Researcher/person taking the consent

_____

Date _____

ANNEX TO THE CERTIFICATE OF CONSENT

The two slips below are given to the participant at the end of the session. They exist because no participant's name is recorded alongside their data: the session code is the only means by which a participant can later identify their own record in order to have it destroyed.

PARTICIPANT SLIP — YOUR SESSION CODE

Please keep this slip. Your data is stored under the code below and under no other identifier. If you decide you want your record destroyed, quote this code to any of the contacts listed under Who to Contact in Part I of this form.

Session code	
Date of session	
Withdraw by	26 September 2026
Contact	Lady Kaina V. Gutierrez — 24100535@usc.edu.ph

WITHDRAWAL REQUEST

Complete this section only if you wish your data to be destroyed. Return it to any of the contacts named under Who to Contact in Part I, or send the same information by e-mail.

I wish to withdraw from the OBRA study. I ask that the data recorded under session code _____ be destroyed and excluded from the analysis. I understand that I do not need to give a reason.

Name (printed)	Signature and date

Appendix B: Gameplay Evaluation Instruments

This appendix reproduces the gameplay-evaluation instruments administered to participants immediately after each supervised play session, as specified in Chapter 4, Section 4.2.1B, and documents the permission or licence under which each is used. The battery comprises the System Usability Scale (administered in full), the short version of the User Experience Questionnaire (administered in full), the Immersion Experience Questionnaire — Short Form (administered in full), the Game Experience Questionnaire Core Module — Competence, Sensory and Imaginative Immersion, and Positive Affect — and two validated subscales borrowed from adjacent fields: the Animacy subscale of the Godspeed Questionnaire Series for the believability of the animated drawing, and the Familiarity subscale of the QUESI for the fit between the granted ability and player expectation. The GEQ's remaining components, its In-Game Module, and its Social Presence and Post-Game Modules are not administered, for the reasons given in Section 4.2.1B. A further instrument, the Principles of Game Playability (PLAY), is administered to expert reviewers rather than to participants and is reproduced in Section G. The informed consent form administered alongside these instruments is reproduced in Section H. A seventh participant instrument, the Cultural Content Questionnaire, is administered to a separate group of respondents (students of anthropology, sociology, and history) and is reproduced in Section I.

Appendix C Section B reproduces the permission and licensing evidence for every instrument in this appendix, including the correspondence with the

author of PLAY, which is administered to a panel of game development practitioners rather than to participants and is described in Chapter 4, Section 4.2.1C.

All instruments are administered in English and no instrument has been translated. No item wording has been altered, with one disclosed exception: the referent of the three QUESI Perceived Familiarity items is adapted to the ability granted to the player's drawing, as described and justified in Section F.

Permission and Licensing

None of the six participant instruments was developed by the researchers. Each is a published, validated instrument used with the permission of its authors or under the terms of the licence under which it was released. Permission was sought in writing from the corresponding authors of the four instruments not released under an unrestricted licence. The correspondence is reproduced in Section H.

Table B-1

Permission and Licensing Status of the Gameplay-Evaluation Instruments and Expert Review Instruments

Instrument	Basis of use	Terms	Date
System Usability Scale	Open release by author	Released for free use in usability assessment on the sole condition that the source is acknowledged (Brooke, 1996). No individual permission required.	n/a
UEQ-S	Open publication by authors	The questionnaire, its handbook and its analysis tool are published for download at ueq-online.org,	n/a

		where the authors state that all materials are free of charge.	
IEQ-SF	Written permission; CC BY	Author manuscript deposited under a Creative Commons Attribution licence. Written permission additionally granted by Dr. Joe Cutting, conditional on citation of the source paper.	4 Aug 2026
GEQ Core (3 of 7 components)	Written permission	Permission granted by Prof. dr. Wijnand A. IJsselsteijn to use the GEQ in the manner specified in the request, namely the three named Core components only.	4 Aug 2026
Godspeed Animacy subscale	Written permission; CC BY-NC	Source article published open access under a Creative Commons Attribution-NonCommercial licence. Written permission additionally granted by Assoc. Prof. Christoph Bartneck, conditional on due credit.	5 Aug 2026
QUESI Familiarity subscale	Adapted subscale; permission to use and modify requested	Hurtienne and Naumann (2010) expressly encourage the design and research community to use the questionnaire, naming game development. The item referent is adapted to the granted ability, as described in Section F, so permission to use and modify was requested. Delivered 4 Aug 2026; automatic reply indicated unavailability until 24 Aug 2026. The request is to be resent on that date.	Pending

PLAY (expert reviewers, not participants)	Written permission	Copyright held by the authors, not ACM. Granted by Dr. Heather Desurvire conditional on credit and documentation of changes. All fifty principles administered verbatim; no item altered.	11 Aug 2026

Two conditions attached to the grants above are satisfied by the study as designed. The permission for the IEQ-SF is conditional on citation of Cutting et al. (2025) in this thesis and in any associated publication, and its author further recommends that Cronbach's alpha be computed per subscale should the scale be translated; no translation is made, as all instruments are administered in their original English, and the recommendation therefore does not apply. The permission for the Godspeed Animacy subscale is conditional on due credit, and the non-commercial term of its licence is satisfied, this study being unfunded and conducted solely for an academic requirement.

The QUESI Familiarity subscale is adapted in one respect: the referent of each item is the ability granted to the player's drawing rather than the system as a whole. The adaptation is described and justified in Section F. Because the subscale is modified, permission to use and to modify it has been requested rather than relying on the authors' published invitation alone.

A. System Usability Scale (SUS)

Brooke, J. (1996). SUS: A quick and dirty usability scale. In P. W. Jordan, B. Thomas, B. A. Weerdmeester, & A. L. McClelland (Eds.), Usability evaluation in industry (pp. 189–194). Taylor & Francis.

The System Usability Scale (SUS) is a ten-item Likert scale that yields a single, global score of perceived usability. It is administered immediately after the respondent has finished using the system, before any debriefing or discussion takes place. Respondents should record their immediate reaction to each statement rather than deliberating at length, and every item should be marked — if a respondent feels an item does not apply, they should mark the center point of the scale.

#	Statement	1	2	3	4	5
1	I think that I would like to use this system frequently	☐	☐	☐	☐	☐
2	I found the system unnecessarily complex	☐	☐	☐	☐	☐
3	I thought the system was easy to use	☐	☐	☐	☐	☐
4	I think that I would need the support of a technical person to be able to use this system	☐	☐	☐	☐	☐
5	I found the various functions in this system were well integrated	☐	☐	☐	☐	☐
6	I thought there was too much inconsistency in this system	☐	☐	☐	☐	☐
7	I would imagine that most people would learn to use this system very quickly	☐	☐	☐	☐	☐
8	I found the system very cumbersome to use	☐	☐	☐	☐	☐
9	I felt very confident using the system	☐	☐	☐	☐	☐
10	I needed to learn a lot of things before I could get going with this system	☐	☐	☐	☐	☐

Scale: 1 = Strongly Disagree, 5 = Strongly Agree.

Scoring Guidelines

For odd-numbered items (1, 3, 5, 7, 9), the score contribution is the scale position minus 1. For even-numbered items (2, 4, 6, 8, 10), the score contribution is 5 minus the scale position. Sum the ten contributions (range 0–40) and multiply by 2.5 to obtain the overall SUS score, which ranges from 0 to 100. A score above 68 is conventionally interpreted as above average (Brooke, 1996; Lewis & Sauro, 2009).

Permission. The SUS was released by its author for unrestricted use in usability assessment, subject only to acknowledgment of the source (Brooke, 1996). No individual permission is required and none was sought.

B. User Experience Questionnaire — Short Version (UEQ-S)

Schrepp, M., Hinderks, A., & Thomaschewski, J. (2017). Design and evaluation of a short version of the User Experience Questionnaire (UEQ-S). International Journal of Interactive Multimedia and Artificial Intelligence, 4(6), 103–108.

Eight items, each a pair of opposed terms rated on a seven-point scale. The first four items form the Pragmatic Quality scale and the last four the Hedonic Quality scale. Responses are scored from −3 at the left-hand term to +3 at the right-hand term, and each scale is the mean of its four items. The two scales are reported separately; no combined score is computed. Administered post-session, in English, verbatim.

#	Statement								
1	Obstructive	☐	☐	☐	☐	☐	☐	☐	Supportive
2	Complicated	☐	☐	☐	☐	☐	☐	☐	Easy
3	Inefficient	☐	☐	☐	☐	☐	☐	☐	Efficient
4	Confusing	☐	☐	☐	☐	☐	☐	☐	Clear
5	Boring	☐	☐	☐	☐	☐	☐	☐	Exciting
6	Not Interesting	☐	☐	☐	☐	☐	☐	☐	Interesting
7	Conventional	☐	☐	☐	☐	☐	☐	☐	Inventive
8	Usual	☐	☐	☐	☐	☐	☐	☐	Leading Edge

Permission. The User Experience Questionnaire and its short version are distributed by their authors through the official UEQ website (ueq-online.org), which states that all materials required to work with the questionnaire are completely free of charge. The source paper is cited in this thesis and will be

cited in any associated publication. The eight items are administered in their original English, verbatim; no translation or adaptation is made.

C. Immersion Experience Questionnaire — Short Form (IEQ-SF)

Cutting, J., Aung, M., Perrett, F., & Cairns, P. A. (2025). The development and validation of the Immersion Experience Questionnaire Short Form (IEQ-SF). International Journal of Human-Computer Studies, 203, Article 103580. https://doi.org/10.1016/j.ijhcs.2025.103580

The IEQ-SF is an eleven-item short form of the thirty-one-item Immersion Experience Questionnaire (Jennett et al., 2008), organized around three subfactors: Involvement (items 1–4), Real-World Dissociation (items 5–8), and Challenge (items 9–11). Validation work reports that it functions very similarly to the full-length original while being substantially quicker to administer (Cutting et al., 2025). Respondents indicate their agreement with each statement below on a five-point scale from “Strongly Disagree” (1) to “Strongly Agree” (5).

#	Statement	1	2	3	4	5
1	I felt focused on the game.	☐	☐	☐	☐	☐
2	The game was something that I was experiencing, rather than just doing.	☐	☐	☐	☐	☐
3	I felt motivated when playing the game.	☐	☐	☐	☐	☐
4	I enjoyed playing the game.	☐	☐	☐	☐	☐
5	I felt consciously aware of being in the real world whilst playing. *	☐	☐	☐	☐	☐
6	I forgot about my everyday concerns.	☐	☐	☐	☐	☐
7	I felt that I was separated from the real-world environment.	☐	☐	☐	☐	☐
8	I found myself so involved that I was unaware I was using controls.	☐	☐	☐	☐	☐
9	I found the game challenging.	☐	☐	☐	☐	☐
10	I found the game easy. *	☐	☐	☐	☐	☐

#	Statement	1	2	3	4	5
11	I felt in suspense about whether or not I would do well in the game.	☐	☐	☐	☐	☐

Scale: 1 = Strongly Disagree, 5 = Strongly Agree. * Reverse-scored item (score = 6 − response) before averaging.

Permission. The accepted author manuscript is deposited in the University of York research repository under a Creative Commons Attribution licence. Written permission to use the instrument in this study was additionally granted by Dr. Joe Cutting on 4 August 2026, conditional on citation of the source paper in this thesis and in any associated publication. The scale is administered in its original English; no translation is made.

D. Game Experience Questionnaire (GEQ)

IJsselsteijn, W. A., de Kort, Y. A. W., & Poels, K. (2013). The Game Experience Questionnaire. Technische Universiteit Eindhoven.

The Core Module assesses game experience immediately after a play session. Because each component is scored as an independent average of its own items (IJsselsteijn et al., 2013), only the three components relevant to this study's objectives are administered here — Competence, Sensory and Imaginative Immersion, and Positive Affect (16 of the 33 Core items). Respondents indicate how they felt while playing for each statement, on a five-point scale from 'not at all' (0) to 'extremely' (4). Items are renumbered 1–16 for administration; the component each belongs to is shown for scoring. Following Law et al. (2018), the component scores are treated as descriptive indicators interpreted alongside the other instruments and the telemetry rather than on their own.

#	Statement	Component	0	1	2	3	4
1	I felt content	Positive Affect	☐	☐	☐	☐	☐
2	I thought it was fun	Positive Affect	☐	☐	☐	☐	☐
3	I felt happy	Positive Affect	☐	☐	☐	☐	☐
4	I felt good	Positive Affect	☐	☐	☐	☐	☐
5	I enjoyed it	Positive Affect	☐	☐	☐	☐	☐
6	I felt skilful	Competence	☐	☐	☐	☐	☐
7	I felt competent	Competence	☐	☐	☐	☐	☐
8	I was good at it	Competence	☐	☐	☐	☐	☐
9	I felt successful	Competence	☐	☐	☐	☐	☐
10	I was fast at reaching the game's targets	Competence	☐	☐	☐	☐	☐
11	I was interested in the game's story	Sensory & Imaginative Immersion	☐	☐	☐	☐	☐
12	It was aesthetically pleasing	Sensory & Imaginative Immersion	☐	☐	☐	☐	☐

#	Statement	Component	0	1	2	3	4
13	I felt imaginative	Sensory & Imaginative Immersion	☐	☐	☐	☐	☐
14	I felt that I could explore things	Sensory & Imaginative Immersion	☐	☐	☐	☐	☐
15	I found it impressive	Sensory & Imaginative Immersion	☐	☐	☐	☐	☐
16	It felt like a rich experience	Sensory & Imaginative Immersion	☐	☐	☐	☐	☐

Scale: 0 = Not at all, 1 = Slightly, 2 = Moderately, 3 = Fairly, 4 = Extremely. Scoring: each component score is the mean of its items — Positive Affect (1–5), Competence (6–10), Sensory & Imaginative Immersion (11–16). Report the three component means; do not compute a single GEQ total.

Permission. Written permission was granted by Prof. Dr. Wijnand A. IJsselsteijn on 4 August 2026 to use the GEQ in the manner set out in the request, namely the administration of the Competence, Sensory and Imaginative Immersion, and Positive Affect components of the Core Module only, with item wording and response format unchanged. The remaining Core components, the In-Game Module, and the Social Presence and Post-Game Modules are not administered.

E. Godspeed Animacy Subscale - Animation Believability

The Animacy subscale of the Godspeed Questionnaire Series measures the degree to which an artifact is perceived as alive and lively rather than inert and mechanical. It comprises six five-point semantic-differential items. In OBRA it is administered after the player has seen their drawing animated in play, to assess whether the stroke-animated drawing reads as a living, moving entity. The Animacy score is the mean of the six items. Because Ho and MacDorman (2010) report that the Godspeed animacy and anthropomorphism indices correlate strongly ($r \approx .89$), the score is treated as a descriptive indicator of believability rather than an independent construct. Mark, for each pair, the point that best matches your impression of your drawing as it moved (1 = the left word, 5 = the right word).

#	Statement	1	2	3	4	5	
1	Dead	☐	☐	☐	☐	☐	Alive
2	Stagnant	☐	☐	☐	☐	☐	Lively
3	Mechanical	☐	☐	☐	☐	☐	Organic
4	Artificial	☐	☐	☐	☐	☐	Lifelike
5	Inert	☐	☐	☐	☐	☐	Interactive
6	Apathetic	☐	☐	☐	☐	☐	Responsive

Five-point semantic differential (Bartneck et al., 2009). **Scoring:** Animacy score is the mean of the six items; reported as a descriptive indicator of believability.

Permission. The source article is published open access under a Creative Commons Attribution-NonCommercial licence. Written permission to use the subscale was additionally granted by Assoc. Prof. Christoph Bartneck on 5 August 2026, conditional on due credit. The non-commercial term is satisfied: this study is unfunded and is conducted solely to satisfy an undergraduate degree requirement. Only the Animacy subscale is administered; the remaining subscales of the series are not used.

F. QUESI – Perceived Familiarity Subscale (ability-expectation alignment)

The Questionnaire for Measuring the Subjective Consequences of Intuitive Use (QUESI) comprises fourteen items across five subscales; this study administers its three-item Perceived Familiarity subscale, whose items derive from the precondition of prior knowledge — whether a system behaves the way the user's existing world-knowledge predicts. This maps closely onto OBRA's claim that a ConceptNet-derived ability should match what a player already expects the drawn thing to do. Respondents rate agreement on a five-point scale from 'strongly disagree' (1) to 'strongly agree' (5), the subscale being adapted in one respect described below. The Perceived Familiarity score is the mean of the three items, interpreted as the validated proxy for ability-expectation alignment alongside the per-class recall telemetry.

#	Statement	1	2	3	4	5
1	I could use my drawing's ability in a way that seemed familiar to me.	☐	☐	☐	☐	☐
2	In using my drawing's ability, I could fall back on things I already knew about that kind of thing.	☐	☐	☐	☐	☐
3	The way my drawing's ability worked was familiar to me from what I already know.	☐	☐	☐	☐	☐

Scale: 1 = Strongly Disagree, 5 = Strongly Agree (Naumann & Hurtienne, 2010).

Scoring: Perceived Familiarity score is the mean of the three items; interpreted alongside per-class recall telemetry as the validated proxy for ability-expectation alignment.

Adaptation. The subscale is adapted rather than adopted verbatim. The referent of each of the three items has been changed from the system as a whole to the ability granted to the player's drawing. Nothing else is altered: no item is added, removed, or reordered, the five-point agreement format is unchanged, and the score remains the mean of the three items. The change was made because the

construct under measurement is the fit between a single attributed ability and the player's prior expectation, and a participant who has drawn one entity has no basis on which to judge the system in general. Adapting the referent of a validated subscale to the domain of the study, while leaving its structure and scoring intact, is standard practice, and the adaptation is judged not to alter the construct. Because the instrument is adapted, Cronbach's alpha for the three items as administered will be computed from the study's own responses and reported alongside the published figure.

Permission. Hurtienne and Naumann (2010) expressly encourage the design and research community to use the questionnaire in order to inform its norming, naming product, software, and game development as areas of particular interest. This study falls within that invitation to use. Because the referent of the items has been adapted as described above, permission to use and to modify the subscale has been sought rather than permission to use alone. A written permission request was nevertheless sent to Prof. Dr. Jörn Hurtienne on 4 August 2026 and was confirmed delivered; an automatic reply indicated that he would be unavailable by e-mail until 24 August 2026 and requested that important messages be resent after that date. The request is scheduled to be resent on 24 August 2026, and any response received will be appended to Section H.

G. Expert Design Review Form (PLAY)

Desurvire, H., & Wiberg, C. (2009). Game usability heuristics (PLAY) for evaluating and designing better games: The next iteration. In A. A. Ozok & P. Zaphiris (Eds.), Online Communities and Social Computing (Lecture Notes in Computer Science, Vol. 5621, pp. 557–566). Springer.

This appendix reproduces the form administered to game development practitioners, as specified in Section 4.2.1C. All fifty principles of Table 1 are administered in their published wording and order. Item codes correspond to the published numbering — I-A1 is Category I, Heuristic A, item 1. Each reviewer completes the form independently.

1. Purpose and Scope

This form collects expert judgement on the design of a game that is still in development. It is a formative expert design review, not a playability test of a finished product.

The instrument is the Principles of Game Playability (PLAY) of Desurvire and Wiberg (2009), the refined successor to the Heuristics to Evaluate Playability of Desurvire, Caplan and Toth (2004). PLAY was developed for use throughout the design process, particularly at the concept phase when changes are least costly, and its principles were validated against high- and low-ranked games. All fifty principles are administered in their published wording and order; nothing has been reworded, omitted, or added.

Written permission to use the instrument was granted by Dr. Heather Desurvire, conditional on credit and on documentation of any changes. Both conditions are met.

Your responses will identify design problems before implementation continues, and will be reported in the researchers' undergraduate thesis. Please answer honestly; a review that finds problems is far more useful than one that does not.

2. Materials Provided for Review

Tick each item you were given and actually consulted.

Reviewed	Material	Notes
	Game design plan (document)	
	System and gameplay flowchart	
	Video demonstration of the drawing-to-entity mechanic	
	Working prototype build	
	Concept art and visual direction samples	

3. Reviewer Information

Fields in red are to be completed by the reviewer.

Full name			
Studio, company, or affiliation			
Current role or job title			
Years of game development experience			
Titles shipped or projects worked on (optional)			
Did you play the prototype build?	Yes	No	Partially
	☐	☐	☐
Email address			
Date review completed			

4. Instructions

1. Review the materials once end to end before rating anything, then work through the principles in order.

2. For each principle, enter a rating code from the key below.

3. Where you enter P or N, assign a severity rating from 0 to 4 and describe the specific problem in the Comment column. A rating without a described problem cannot be acted on.

4. Where the materials do not let you judge a principle, enter CA. Please do not guess. Some principles concern qualities that emerge only over an extended play session, and CA is the correct answer for those.

5. Record each distinct problem in the Problem Log in Part 6.

6. Answer the open questions in Part 5 in your own words. These carry as much weight as the ratings.

7. Please work independently, without discussing the game or your ratings with the other reviewers or with the researchers until you have submitted this form.

Completing the form takes approximately sixty minutes.

Rating key

Rating	Code	Rating
0	**M**	Met — the design satisfies this principle.
1	**P**	Partially met — satisfied in part; the gap is described in the Comment column.
2	**N**	Not met — the design violates this principle.
3	**CA**	Cannot assess — the materials do not contain enough information to judge. This is an expected and acceptable answer.

Severity scale (Nielsen, 1994)

Rating	Label	Meaning
0	**Not a problem**	I do not agree that this is a playability problem at all.
1	**Cosmetic**	Need not be fixed unless extra time is available.
2	**Minor**	Fixing this should be given low priority.
3	**Major**	Important to fix; should be given high priority.
4	**Catastrophic**	Imperative to fix before the game is released to players.

Part 1 - Category I: Game Play

#	Principle (as published)	Rating	Sev.	Comment
A. Heuristic: Enduring Play				
I-A1	The players finds the game fun, with no repetitive or boring tasks			
I-A2	The players should not experience being penalized repetitively for the same failure.			
I-A3	The players should not lose any hard won possessions.			
I-A4	Gameplay is long and enduring and keeps the players' interest.			
I-A5	Any fatigue or boredom was minimized by varying activities and pacing during the game play.			
B. Heuristic: Challenge, Strategy and Pace				
I-B1	Challenge, strategy and pace are in balance.			
I-B2	The game is paced to apply pressure without frustrating the players. The difficulty level varies so the players experience greater challenges as they develop mastery.			
I-B3	Easy to learn, harder to master.			
I-B4	Challenges are positive game experiences, rather than negative experiences, resulting in wanting to play more, rather than quitting.			
I-B5	AI is balanced with the players' play.			
I-B6	The AI is tough enough that the players have to try different tactics against it.			
C. Heuristic: Consistency in Game World				
I-C1	The game world reacts to the player and remembers their passage through it.			
I-C2	Changes the player make in the game world are persistent and noticeable if			

#	Principle (as published)	Rating	Sev.	Comment
	they back-track to where they have been before.			
D. Heuristic: Goals				
I-D1	The game goals are clear. The game provides clear goals, presents overriding goals early as well as short term goals throughout game play.			
I-D2	The skills needed to attain goals are taught early enough to play or use later, or right before the new skill is needed.			
I-D3	The game gives rewards that immerse the player more deeply in the game by increasing their capabilities, capacity or for example, expanding their ability to customize.			
E. Heuristic: Variety of Players and Game Styles				
I-E1	The game supports a variety of game styles.			
I-E2	The game is balanced with multiple ways to win.			
I-E3	The first ten minutes of play and player actions are painfully obvious and should result in immediate and positive feedback for all types of players.			
I-E4	The game had different AI settings so that it was challenging to all levels of players, whether novice or expert players.			
F. Heuristic: Players Perception of Control				
I-F1	Players feel in control.			
I-F2	The player's have a sense of control and influence onto the game world.			

Part 2 - Category II: Coolness, Entertainment, Humor, Emotional Immersion

#	Principle (as published)	Rating	Sev.	Comment
A. Heuristic: Emotional Connection				
II-A1	There is an emotional connection between the player and the game world as well as with their "avatar."			
B. Heuristic: Coolness/Entertainment				
II-B1	The game offers something different in terms of attracting and retaining the players' interest.			
C. Heuristic: Humor				
II-C1	The game uses humor well.			
D. Heuristic: Immersion				
II-D1	The game utilizes visceral, audio and visual content to further the players' immersion in the game.			

Part 3 - Category III: Usability and Game Mechanics

#	Principle (as published)	Rating	Sev.	Comment
A. Heuristic: Documentation/Tutorial				
III-A1	Player does not need to read the manual or documentation to play.			
III-A2	Player does not need to access the tutorial in order to play.			
B. Heuristic: Status and Score				
III-B1	Game controls are consistent within the game and follow standard conventions.			
III-B2	Status score Indicators are seamless, obvious, available and do not interfere with game play.			
III-B3	Controls are intuitive, and mapped in a natural way; they are customizable and default to industry standard settings.			
III-B4	Consistency shortens the learning curve by following the trends set by the gaming industry to meet users'			

#	Principle (as published)	Rating	Sev.	Comment
	expectations. If no industry standard exists, perform usability/playability research to ascertain the best mapping for the majority of intended players.			
C. Heuristic: Game Provides Feedback				
III-C1	Game provides feedback and reacts in a consistent, immediate, challenging and exciting way to the players' actions.			
III-C2	Provide appropriate audio/visual/visceral feedback (music, sound effects, controller vibration).			
D. Heuristic: Terminology				
III-D1	The game goals are clear. The game provides clear goals, presents overriding goals early as well as short term goals throughout game play.			
III-D2	The skills needed to attain goals are taught early enough to play or use later, or right before the new skill is needed.			
III-D3	The game gives rewards that immerse the player more deeply in the game by increasing their capabilities, capacity or, for example, expanding their ability to customize.			
E. Heuristic: Burden On Player				
III-E1	The game does not put an unnecessary burden on the player.			
III-E2	Player is given controls that are basic enough to learn quickly, yet expandable for advanced options for advanced players.			
F. Heuristic: Screen Layout				
III-F1	Screen layout is efficient, integrated, and visually pleasing.			
III-F2	The player experiences the user interface as consistent (in controller, color, typographic, dialogue and user interface design).			

#	Principle (as published)	Rating	Sev.	Comment
III-F3	The players experience the user interface/HUD as a part of the game.			
III-F4	Art is recognizable to the player and speaks to its function.			
G. Heuristic: Navigation				
III-G1	Navigation is consistent, logical and minimalist.			
H. Heuristic: Error Prevention				
III-H1	Player error is avoided.			
III-H2	Player interruption is supported, so that players can easily turn the game on and off and be able to save the games in different states.			
III-H3	Upon turning on the game, the player has enough information to begin play.			
III-H4	Players should be given context sensitive help while playing so that they are not stuck and need to rely on a manual for help.			
III-H5	All levels of players are able to play and get involved quickly and easily with tutorials, and/or progressive or adjustable difficulty levels.			
I. Heuristic: Game Story Immersion				
III-I1	Game story encourages immersion (If game has story component).			

Part 4 - Problem Log

List each distinct problem you identified. Reference the item code where one applies; write "general" where none does. Add rows as needed.

#	Where (screen / artifact)	Problem observed	Heuristic	Severity
1				
2				
3				
4				
5				
6				
7				
8				
9				
10				
11				
12				

Part 5 - Overall Assessment

In two or three sentences, state your overall judgement of the design as presented, and the single change you would make first.

Part 6 - Confirmation

I confirm that I reviewed the materials described in Section 2, that I completed this form independently, and that the judgements recorded above are my own.

_____

Signature over printed name

_____

Date

References

Desurvire, H., Caplan, M., & Toth, J. A. (2004). Using heuristics to evaluate the playability of games. In *CHI '04 Extended Abstracts on Human Factors in Computing Systems* (pp. 1509–1512). Association for Computing Machinery. https://doi.org/10.1145/985921.986102

Desurvire, H., & Wiberg, C. (2009). Game usability heuristics (PLAY) for evaluating and designing better games: The next iteration. In A. A. Ozok & P. Zaphiris (Eds.), Online Communities and Social Computing (Lecture Notes in Computer Science, Vol. 5621, pp. 557–566). Springer.

Desurvire, H., & Wixon, D. (2013). Game principles: Change, choice & creation — Making better games. In CHI ’13 Extended Abstracts on Human Factors in Computing Systems. Association for Computing Machinery.

Nielsen, J., & Molich, R. (1990). Heuristic evaluation of user interfaces. In Proceedings of the SIGCHI Conference on Human Factors in Computing Systems (pp. 249–256). Association for Computing Machinery.

H. Cultural Content Questionnaire

This appendix reproduces the Cultural Content Questionnaire administered to students of anthropology, sociology, and history, as specified in Chapter 4, Section 4.2.1D. The instrument is researcher-developed. Its eight dimensions are taken from the Ecological Validity Model of Bernal, Bonilla and Bellido (1995) and are retained as published; the twenty-one rating items and six open-response questions are the research team's own composition. No item is reproduced from the source, and no permission is required for the use of a framework in this manner under Republic Act No. 8293, Section 175. The instrument is administered in English; no translation is made.

CULTURAL CONTENT QUESTIONNAIRE

For respondents from Anthropology, Sociology, and History

Project: OBRA — A Drawing-Based Adventure Game with Knowledge-Grounded Ability Attribution using CNN and ConceptNet

Adviser: Christine F. Peña, MMath

Purpose

Thank you for playing OBRA and for agreeing to answer these questions. OBRA is a single-player adventure game set in the Philippines, in which the player advances by drawing. Its five levels draw on real Philippine places, festivals, and figures.

The research team are Computer Science students. We are not trained in Philippine culture, and the cultural content of this game is our own work. We are asking you, as a student of anthropology, sociology, or history, whether we have handled that material accurately and respectfully.

There are no right or wrong answers, and we are not looking for reassurance. Where we have got something wrong, saying so is the most useful thing you can do. Your answers are anonymous and are reported only in aggregate.

The questionnaire is organised around the eight dimensions of the Ecological Validity Model (Bernal, Bonilla & Bellido, 1995), a published framework for judging whether work developed in one cultural setting has been adequately adapted to another. The dimensions are not the research team's own; the statements under them are.

The questionnaire takes about twenty minutes. You may skip any question, and you may stop at any time.

Part A - About You

Fields in red are for you to complete. No information that could identify you personally is collected.

Program and year level	*[e.g. BA Anthropology, 3rd year]*
Major or discipline	*[Anthropology / Sociology / History]*
Relevant courses taken	*[e.g. Philippine Culture and Society, Philippine History]*
Region(s) of the Philippines you have lived in or know well	*[Region(s)]*
Date played	*[DD Month YYYY]*
Approximate time spent playing	*[minutes]*

Some parts of the game draw on regions you may or may not know well. Please mark how familiar you are with each. It is entirely normal not to know some of these.

How familiar are you with each of the following?	Not at all	A little	Somew hat	Well	Very well
The Cordillera region and the Banaue Rice Terraces					
Sinulog or other Visayan street festivals					
Coastal and small-boat fishing communities					
Philippine folklore, including the Aswang					
The Bicol region and Mayon Volcano					

Part B - The Game as a Whole

For each statement, mark the number that best matches your view. 1 = Strongly Disagree, 2 = Disagree, 3 = Neither agree nor disagree, 4 = Agree, 5 = Strongly Agree. Mark CJ (cannot judge) if you do not feel you know enough to answer — this is a useful answer, not a failure to answer.

#	Statement	1	2	3	4	5	CJ
B1. Language — the words the game uses							
1	The Filipino words the game keeps untranslated (Lola, Lolo, apo, bahay kubo, bangka, banderitas) are used correctly.						
2	Leaving those words untranslated felt natural rather than decorative.						
B2. Persons — the people and their relationships							
3	The relationship between the grandchild and the grandmother reflects how Filipino families actually relate to one another.						
4	The way the grandfather appears as a guiding presence fits Filipino beliefs about family and the dead.						
B3. Metaphors — symbols and figures							
5	Using the Aswang to represent the grandmother's grief respects what the figure means in Philippine culture.						
6	The other symbols the game uses (hidden flowers for memory, Mayon for the story's climax) carry meanings that fit their cultural sources.						
B4. Content — customs, places, and everyday life							
7	The festival shown in the game accurately reflects a Philippine street festival.						
8	The everyday details the game shows — homes, boats, streets, landscapes — are accurate.						
B5. Concepts — the ideas the story is built on							
9	The game's treatment of grief and memory feels Filipino rather than borrowed from elsewhere.						
10	The idea of family obligation in the story matches how Filipinos actually understand it.						

#	Statement	1	2	3	4	5	CJ
B6. Goals — what the game asks the player to reach for							
11	The ending the game works toward is one a Filipino player would find meaningful.						
12	What the game asks the player to value is something a Filipino audience would recognise as worth reaching.						
B7. Methods — how the game delivers its content							
13	Drawing as the main way to play suits the Philippine setting and story.						
14	The dialogue choices offered fit the characters and the situations they are in.						
B8. Context — the wider setting							
15	The five settings together give a fair picture of the Philippines.						
16	The depiction of poorer neighbourhoods in the festival level is fair rather than stereotyping.						

Part C - The Five Levels

Same scale. Answer only for levels you reached.

#	Statement	1	2	3	4	5	CJ
Levels							
17	Bahay Kubo: combining a bahay kubo with the Banaue Rice Terraces makes sense as a single place.						
18	Pista: naming the festival after a real one while inventing its details is appropriate.						
19	Dagat: the handling of the bangka and of being at sea during a storm is believable.						
20	Dilim: the forest and its Aswang encounter treat Philippine folklore respectfully.						
21	Mayon: setting the story's climax on Mayon Volcano is appropriate given its real associations.						

Part D - In Your Own Words

These questions matter as much as the ratings above. Please answer in as much or as little detail as you wish.

D1	**Was there anything in the game that felt wrong, inaccurate, or disrespectful? Please describe what it was and where you saw it.**
D2	**Was there anything important about Philippine culture that the game left out, or that you expected to see and did not?**
D3	**Does the game feel like it understands Philippine culture, or like it only uses Philippine names, images, and places? Please explain your answer.**
D4	**The game is titled OBRA. Does that title fit the game, and does it feel rooted in Philippine artistic tradition?**

D5	**Would you be comfortable showing this game to someone from one of the regions it depicts? Why or why not?**
D6	**Is there anything else you would like the researchers to know?**

Part E - Overall

Please mark one.

	The cultural content may be used as it is.
	The cultural content may be used after the minor changes I noted above.
	The cultural content needs major changes before it is shown to other players.

Thank you. If you would like to add anything after submitting, or have questions about the study, you may contact the research team at 24100535@usc.edu.ph .

Reference

Bernal, G., Bonilla, J., & Bellido, C. (1995). Ecological validity and cultural sensitivity for outcome research: Issues for the cultural adaptation and development of psychosocial treatments with Hispanics. Journal of Abnormal Child Psychology, 23(1), 67–82. https://doi.org/10.1007/BF01447045

Appendix C: Transmittal Letter, Instrument Permission and Licensing Evidence, Call for Participants, and Screening Questionnaire

A. Transmittal Letters

August 12, 2026

Chestere Eube I. Baring, MBA
Game Development Adviser
Department of Computer, Information Sciences and Mathematics
University of San Carlos
Cebu City, Philippines

Dear Sir Baring:

Greetings!

We sincerely thank you for accepting our request to assist and guide us in our thesis project entitled **"OBRA: Real-Time Drawing Recognition Using CNN and ConceptNet in a 2D Adventure Game."**

OBRA is a 2D adventure game where players create freehand drawings that are recognized through a Convolutional Neural Network (CNN) and transformed into functional in-game entities with abilities attributed using ConceptNet. As part of our game development and evaluation, we intend to seek feedback from game developers regarding OBRA's design, mechanics, technical implementation, system integration, and overall gameplay experience.

In line with this, we are respectfully submitting our **Game Developer Review Form**, which will be administered to selected game developer reviewers. We would greatly appreciate your guidance in assessing whether the form adequately covers relevant aspects of the game and in identifying questions or areas that should be revised, added, or clarified.

We would also appreciate any recommendations you may have regarding our game-development research and evaluation process.

Thank you very much for your time, guidance, and willingness to assist our group.

Respectfully,
Submitted by:

RUSK RYE IOAN C. ALARDE	ERIKA BIANCA E. API	KENT JESTER P. GATCHALIAN	LADY KAINA V. GUTIERREZ

Noted by:
CHRISTINE F. PEÑA, MMath
Thesis Project Adviser

Approved by:
CHESTERE EUBE I. BARING, MBA
Game Development Adviser

August 25, 2026

ANGIE M. CENIZA-CANILLO, PhD
Chairperson, Department of Computer, Information Sciences and Mathematics
University of San Carlos – Talamban Campus, Cebu City

Thru: REV. FR. PETER FILLO, SVD
System Administrator, DCISM

Dear Dr. Ceniza-Canillo:

We are undergraduate students of the Bachelor of Science in Computer Science program of this Department, presently undertaking our thesis entitled **"OBRA: A Drawing-Based Adventure Game with Knowledge-Grounded Ability Attribution using CNN and ConceptNet,"** under the supervision of Christine F. Peña, MMath.

In connection with the evaluation phase of this study, we respectfully request permission to use two (2) DCISM computer laboratories on the dates below, subject to availability and to the Department's class schedule, and permission to install the software our prototype requires on the workstations assigned to us.

Laboratories: **LB400 and LB467**	Dates: **October 1–2, 2026**
Time: **1:00–3:30 PM**	Workstations per laboratory: **15**

Each session takes approximately forty-five (45) minutes: the participant plays a defined portion of the OBRA prototype on a laboratory computer under observation, then answers a battery of standardized questionnaires. At least thirty (30) participants are sought. Two laboratories are requested so that a participant answering the questionnaires is not seated beside another still playing, whose screen and reactions would influence the answers.

On the matter of installation: the prototype requires a Python environment with the recognition dependencies, the exported model files, and the game build to be present locally on each machine used. Nothing else is installed system-wide, and the software needs no network connection, connects to no external server, and writes no data outside the machine it runs on. We will install only on the workstations assigned to us, under whatever conditions the System Administrator may prescribe, and undertake to remove all components and restore the machines at the end of the testing period.

We further undertake to observe all laboratory rules and Departmental policies, to keep within the approved schedule without disrupting any class, to have a team member

present throughout every session, and to assume responsibility for any damage attributable to our use. The study is under review by the University of San Carlos Research Ethics Committee; no session will be conducted before clearance is granted.

The dates, slots, laboratories, and number of workstations above are indicative, and we are prepared to adjust any of them to whatever the Department finds workable. Questions may be directed to the undersigned at 24100535@usc.edu.ph. Thank you for your consideration.

Respectfully yours,

LADY KAINA V. GUTIERREZ
Project Leader

RUSK RYE JOAN C. ALARDE
Lead Developer

ERIKA BIANCA E. API
Developer and Game Designer

KENT JESTER P. GATCHALIAN
Researcher and Developer

Noted by:

CHRISTINE F. PEÑA, MMath
Thesis Adviser

Endorsed by:

REV. FR. PETER FILLO, SVD
System Administrator, DCISM

Endorsed by:

CHRISTINE D. BANDALAN, M.Eng
Laboratory Head, DCISM

Approved by:

ANGIE M. CENIZA-CANILLO, PhD
Chairperson, DCISM

UNIVERSITY OF SAN CARLOS
Talamban Campus, Nasipit, Talamban, Cebu City 6000, Philippines
School of Arts and Sciences
Department of Computer, Information Sciences and Mathematics

[Date]

TO WHOM IT MAY CONCERN:

Greetings!

We are undergraduate students of the Bachelor of Science in Computer Science program of the Department of Computer, Information Sciences and Mathematics, University of San Carlos, presently undertaking our thesis entitled **"OBRA: A Drawing-Based Adventure Game with Knowledge-Grounded Ability Attribution using CNN and ConceptNet,"** under the supervision of **Christine F. Peña, MMath.**

As part of the evaluation phase of our study, a **Cultural Content Evaluation** will be conducted among students of the Department of Anthropology, Sociology and History (DASH). DASH students were purposively identified as appropriate respondents because their academic background in Philippine society, culture, and history provides a relevant basis for assessing the accuracy, appropriateness, and respectful representation of the cultural elements incorporated in OBRA.

For this purpose, the research team developed the OBRA Cultural Content Questionnaire, a researcher-made instrument organized according to the eight dimensions of the Ecological Validity Model of Bernal, Bonilla, and Bellido (1995). The twenty-one (21) rating items and six (6) open-response questions were developed specifically to evaluate the cultural content presented in OBRA.

The evaluation will be conducted in two stages. First, fifteen (15) DASH students will participate in the pilot administration, which will primarily identify issues concerning questionnaire wording, comprehension, item performance, and completion time. Necessary revisions will then be made before the final administration. A separate group of thirty (30) DASH students will subsequently participate in the main administration of the finalized questionnaire. Respondents who participate in the pilot administration will not be included in the main administration, requiring a total of forty-five (45) respondents.

The Cultural Content Questionnaire will be administered entirely through an online form. Relevant game materials, including level descriptions, visual references, and supporting materials necessary to answer the questionnaire, will likewise be provided electronically. Completion is expected to take approximately twenty (20) minutes.

Participation will be entirely voluntary, and respondents may skip any question or withdraw from the study at any time without giving a reason and without consequence to their academic standing. Written informed consent will be obtained before participation, and all collected data will be handled in accordance with the **Data Privacy Act of 2012 (Republic Act No. 10173)**. No recruitment or data collection will begin until the required ethical clearance has been granted by the University of San Carlos Research Ethics Committee.

Questions regarding the study may be directed to the undersigned project leader at **24100535@usc.edu.ph**. Thank you very much for your time and consideration.

Thank you very much for your time and consideration.

Respectfully yours,

LADY KAINA V. GUTIERREZ
Project Leader

RUSK RYE IOAN C. ALARDE
Lead Developer

ERIKA BIANCA E. API
Developer and Game Designer

KENT JESTER P. GATCHALIAN
Researcher and Developer

Noted by:

CHRISTINE F. PEÑA, MMath
Thesis Adviser

UNIVERSITY OF SAN CARLOS
Talamban Campus, Nasipit, Talamban, Cebu City 6000, Philippines
School of Arts and Sciences
Department of Computer, Information Sciences and Mathematics

[Date]

TO WHOM IT MAY CONCERN:

Greetings!

We are undergraduate students of the Bachelor of Science in Computer Science program of the Department of Computer, Information Sciences and Mathematics, University of San Carlos, Talamban Campus, presently undertaking our thesis entitled **"OBRA: A Drawing-Based Adventure Game with Knowledge-Grounded Ability Attribution using CNN and ConceptNet,"** under the supervision of **Christine F. Peña, MMath**.

OBRA is a single-player two-dimensional adventure game in which players progress by creating freehand drawings on an in-game canvas. These drawings are recognized by the system and transformed into usable in-game entities with corresponding abilities.

In connection with the evaluation phase of our study, we respectfully invite eligible students and members of the University of San Carlos community to participate as **gameplay testers**. A minimum of thirty (30) participants will take part in supervised gameplay sessions to be conducted at the **DCISM Computer Laboratory, University of San Carlos – Talamban Campus**, during **October 2026**.

Each session will take approximately sixty (60) minutes, including around forty-five (45) minutes of supervised gameplay followed by a set of standardized questionnaires concerning the game's usability, immersion, and overall player experience. No particular level of drawing skill or gaming experience is required. Participants with varying levels of drawing confidence and gaming familiarity are encouraged to take part.

During gameplay, the system will record an anonymous session log containing gameplay and recognition information, such as drawing submissions, recognition outcomes, response times, and level progress. The drawings themselves will not be retained after the session. Names and signatures collected through this invitation and the informed consent process will be kept strictly confidential and stored separately from

all gameplay records and questionnaire responses. Research data will be identified only through a randomly generated session code, and no participant will be identified by name in the thesis, presentation, or any resulting publication.

Participation is entirely voluntary. Participants may skip any questionnaire item or withdraw from the study at any time without giving a reason and without any consequence to their grades or academic standing. Written informed consent will be obtained before the gameplay session begins. All research data will be handled in accordance with the Data Privacy Act of 2012 (Republic Act No. 10173) and will be securely disposed of after the retention period specified in the research protocol.

The protocol, informed consent form, and research instruments are subject to ethical review by the University of San Carlos Research Ethics Committee (rec@usc.edu.ph). No gameplay session or research data collection will be conducted before the required ethical clearance has been granted.

Questions regarding the study may be directed to the undersigned project leader at 24100535@usc.edu.ph.

Thank you for your time and willingness to support our research.

Respectfully yours,

LADY KAINA V. GUTIERREZ
Project Leader

RUSK RYE IOAN C. ALARDE
Lead Developer

ERIKA BIANCA E. API
Developer and Game Designer

KENT JESTER P. GATCHALIAN
Researcher and Developer

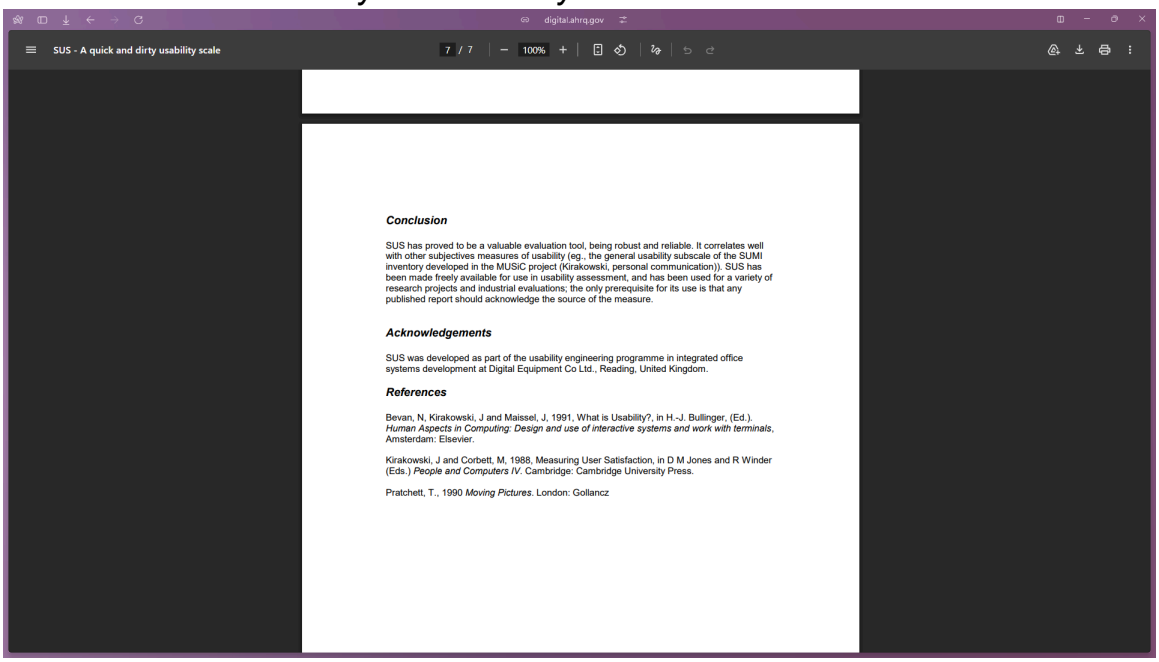
Noted by:

CHRISTINE F. PEÑA, MMath
Thesis Adviser

B. Instrument Permission and Licensing Evidence

Figure B-1

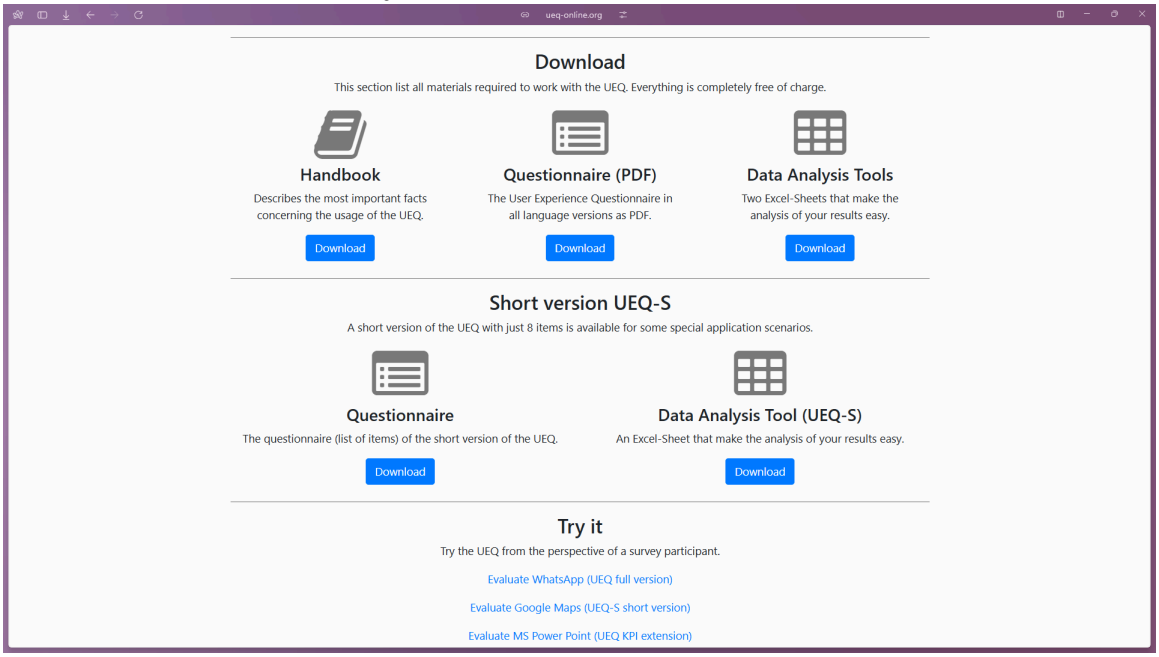
Grant of free use of the System Usability Scale



Note. Conclusion section of Brooke (1996), as hosted by the Agency for Healthcare Research and Quality at digital.ahrq.gov, stating that the scale is freely available for use in usability assessment subject only to acknowledgment of the source.

Figure B-2

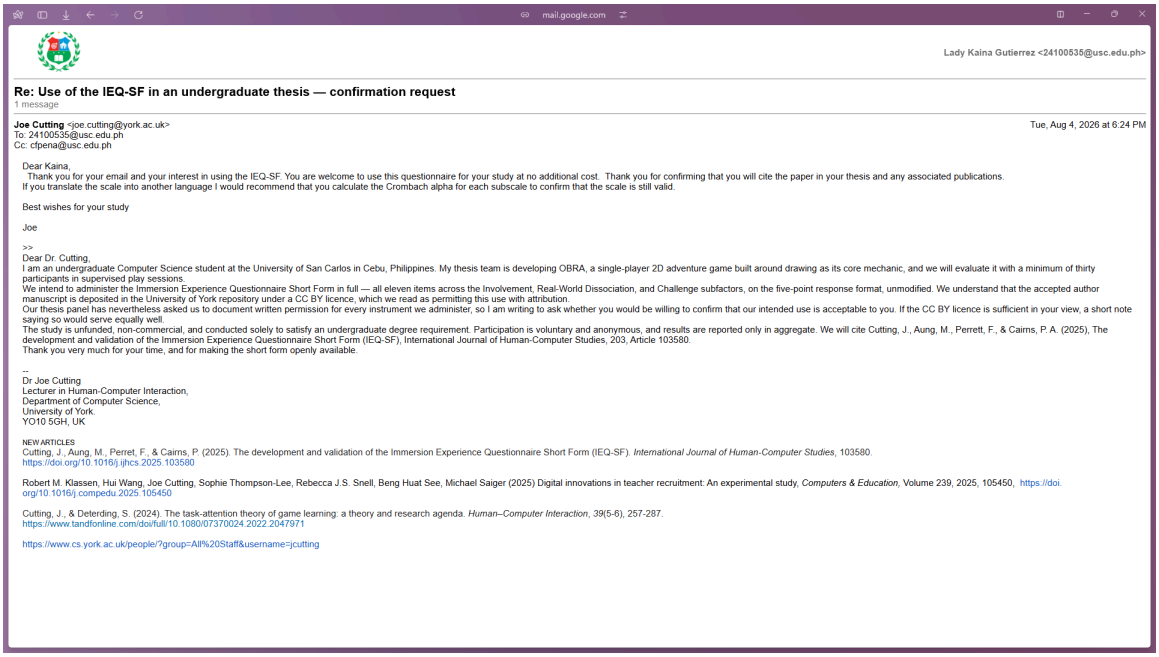
Statement of free availability of the UEQ-S



Note. Download section of the official User Experience Questionnaire website (ueq-online.org), stating that all evaluation materials, including the short version (UEQ-S) and data analysis tools, are completely free of charge subject to acknowledgment of the source.

Figure B-3

Request for permission to administer the Immersion Experience Questionnaire - Short Form



mail.google.com

Lady Kaina Gutierrez <24100535@usc.edu.ph>

Re: Use of the IEQ-SF in an undergraduate thesis — confirmation request

1 message

Joe Cutting <joe.cutting@york.ac.uk> Tue, Aug 4, 2026 at 6:24 PM
To: 24100535@usc.edu.ph
Cc: cfpena@usc.edu.ph

Dear Kaina,
Thank you for your email and your interest in using the IEQ-SF. You are welcome to use this questionnaire for your study at no additional cost. Thank you for confirming that you will cite the paper in your thesis and any associated publications.
If you translate the scale into another language I would recommend that you calculate the Crombach alpha for each subscale to confirm that the scale is still valid.

Best wishes for your study

Joe

>>
Dear Dr. Cutting,
I am an undergraduate Computer Science student at the University of San Carlos in Cebu, Philippines. My thesis team is developing OBRA, a single-player 2D adventure game built around drawing as its core mechanic, and we will evaluate it with a minimum of thirty participants in supervised play sessions.
We intend to administer the Immersion Experience Questionnaire Short Form in full — all eleven items across the Involvement, Real-World Dissociation, and Challenge subfactors, on the five-point response format, unmodified. We understand that the accepted author manuscript is deposited in the University of York repository under a CC BY licence, which we read as permitting this use with attribution.
Our thesis panel has nevertheless asked us to document written permission for every instrument we administer, so I am writing to ask whether you would be willing to confirm that our intended use is acceptable to you. If the CC BY licence is sufficient in your view, a short note saying so would serve equally well.
The study is unfunded, non-commercial, and conducted solely to satisfy an undergraduate degree requirement. Participation is voluntary and anonymous, and results are reported only in aggregate. We will cite Cutting, J., Aung, M., Perrett, F., & Cairns, P. A. (2025), The development and validation of the Immersion Experience Questionnaire Short Form (IEQ-SF), International Journal of Human-Computer Studies, 203, Article 103580.
Thank you very much for your time, and for making the short form openly available.

--
Dr Joe Cutting
Lecturer in Human-Computer Interaction,
Department of Computer Science,
University of York.
YO10 5GH, UK

NEW ARTICLES
Cutting, J., Aung, M., Perret, F., & Cairns, P. (2025). The development and validation of the Immersion Experience Questionnaire Short Form (IEQ-SF). *International Journal of Human-Computer Studies*, 103580. https://doi.org/10.1016/j.ijhcs.2025.103580

Robert M. Klassen, Hui Wang, Joe Cutting, Sophie Thompson-Lee, Rebecca J.S. Snell, Beng Huat See, Michael Saiger (2025) Digital innovations in teacher recruitment: An experimental study, *Computers & Education,* Volume 239, 2025, 105450, https://doi.org/10.1016/j.compedu.2025.105450

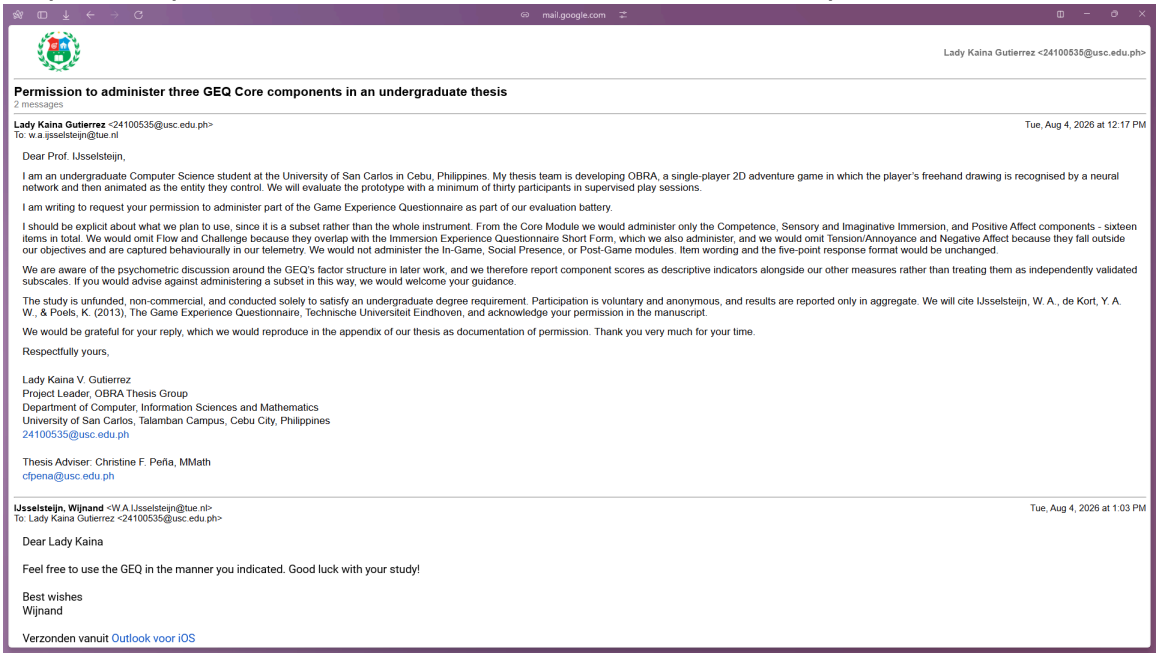
Cutting, J., & Deterding, S. (2024). The task-attention theory of game learning: a theory and research agenda. *Human–Computer Interaction*, *39*(5-6), 257-287. https://www.tandfonline.com/doi/full/10.1080/07370024.2022.2047971

https://www.cs.york.ac.uk/people/?group=All%20Staff&username=jcutting

Note. Received 4 August pd. Permission is conditional on citation of the source paper in this thesis and in any associated publication.

Figure B-4

Request for permission to administer three GEQ Core components



mail.google.com

Lady Kaina Gutierrez <24100535@usc.edu.ph>

Permission to administer three GEQ Core components in an undergraduate thesis

2 messages

Lady Kaina Gutierrez <24100535@usc.edu.ph> Tue, Aug 4, 2026 at 12:17 PM
To: w.a.ijsselsteijn@tue.nl

Dear Prof. IJsselsteijn,

I am an undergraduate Computer Science student at the University of San Carlos in Cebu, Philippines. My thesis team is developing OBRA, a single-player 2D adventure game in which the player's freehand drawing is recognised by a neural network and then animated as the entity they control. We will evaluate the prototype with a minimum of thirty participants in supervised play sessions.

I am writing to request your permission to administer part of the Game Experience Questionnaire as part of our evaluation battery.

I should be explicit about what we plan to use, since it is a subset rather than the whole instrument. From the Core Module we would administer only the Competence, Sensory and Imaginative Immersion, and Positive Affect components - sixteen items in total. We would omit Flow and Challenge because they overlap with the Immersion Experience Questionnaire Short Form, which we also administer, and we would omit Tension/Annoyance and Negative Affect because they fall outside our objectives and are captured behaviourally in our telemetry. We would not administer the In-Game, Social Presence, or Post-Game modules. Item wording and the five-point response format would be unchanged.

We are aware of the psychometric discussion around the GEQ's factor structure in later work, and we therefore report component scores as descriptive indicators alongside our other measures rather than treating them as independently validated subscales. If you would advise against administering a subset in this way, we would welcome your guidance.

The study is unfunded, non-commercial, and conducted solely to satisfy an undergraduate degree requirement. Participation is voluntary and anonymous, and results are reported only in aggregate. We will cite IJsselsteijn, W. A., de Kort, Y. A. W., & Poels, K. (2013), The Game Experience Questionnaire, Technische Universiteit Eindhoven, and acknowledge your permission in the manuscript.

We would be grateful for your reply, which we would reproduce in the appendix of our thesis as documentation of permission. Thank you very much for your time.

Respectfully yours,

Lady Kaina V. Gutierrez
Project Leader, OBRA Thesis Group
Department of Computer, Information Sciences and Mathematics
University of San Carlos, Talamban Campus, Cebu City, Philippines
24100535@usc.edu.ph

Thesis Adviser: Christine F. Peña, MMath
cfpena@usc.edu.ph

IJsselsteijn, Wijnand <W.A.IJsselsteijn@tue.nl> Tue, Aug 4, 2026 at 1:03 PM
To: Lady Kaina Gutierrez <24100535@usc.edu.ph>

Dear Lady Kaina

Feel free to use the GEQ in the manner you indicated. Good luck with your study!

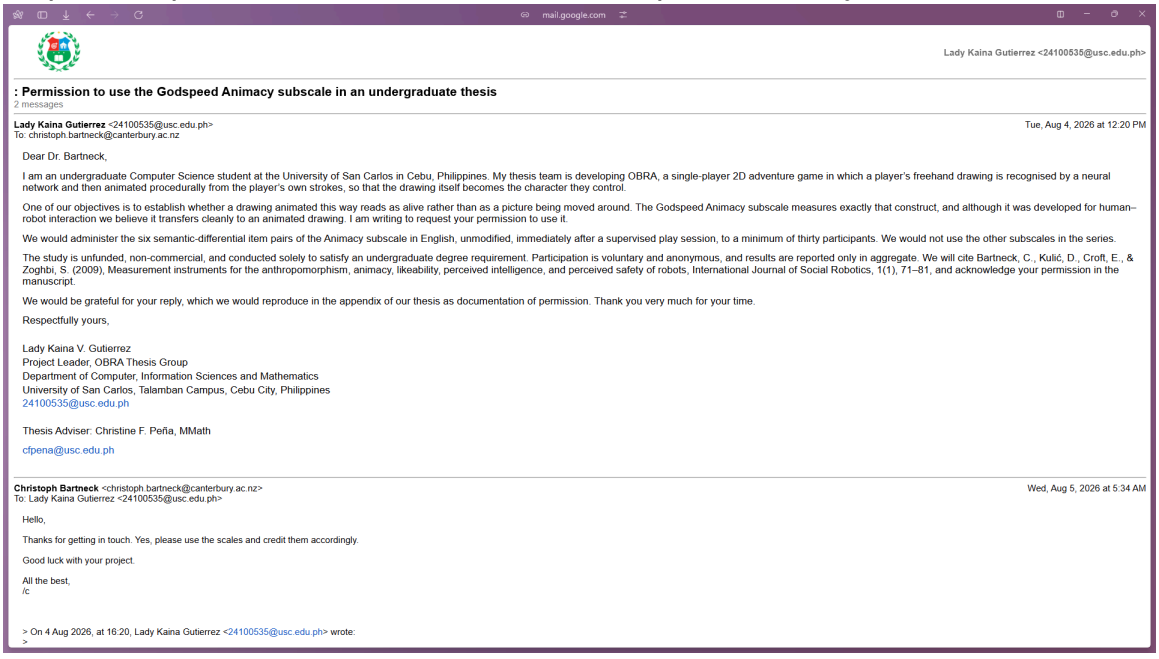
Best wishes
Wijnand

Verzonden vanuit Outlook voor iOS

Note. Received 4 August 2026. Permission is granted for use of the Game Experience Questionnaire in the manner specified in the request.

Figure B-5

Request for permission to administer the Godspeed Animacy subscale



mail.google.com

Lady Kaina Gutierrez <24100535@usc.edu.ph>

: Permission to use the Godspeed Animacy subscale in an undergraduate thesis

2 messages

Lady Kaina Gutierrez <24100535@usc.edu.ph> Tue, Aug 4, 2026 at 12:20 PM
To: christoph.bartneck@canterbury.ac.nz

Dear Dr. Bartneck,

I am an undergraduate Computer Science student at the University of San Carlos in Cebu, Philippines. My thesis team is developing OBRA, a single-player 2D adventure game in which a player's freehand drawing is recognised by a neural network and then animated procedurally from the player's own strokes, so that the drawing itself becomes the character they control.

One of our objectives is to establish whether a drawing animated this way reads as alive rather than as a picture being moved around. The Godspeed Animacy subscale measures exactly that construct, and although it was developed for human–robot interaction we believe it transfers cleanly to an animated drawing. I am writing to request your permission to use it.

We would administer the six semantic-differential item pairs of the Animacy subscale in English, unmodified, immediately after a supervised play session, to a minimum of thirty participants. We would not use the other subscales in the series.

The study is unfunded, non-commercial, and conducted solely to satisfy an undergraduate degree requirement. Participation is voluntary and anonymous, and results are reported only in aggregate. We will cite Bartneck, C., Kulić, D., Croft, E., & Zoghbi, S. (2009), Measurement instruments for the anthropomorphism, animacy, likeability, perceived intelligence, and perceived safety of robots, International Journal of Social Robotics, 1(1), 71–81, and acknowledge your permission in the manuscript.

We would be grateful for your reply, which we would reproduce in the appendix of our thesis as documentation of permission. Thank you very much for your time.

Respectfully yours,

Lady Kaina V. Gutierrez
Project Leader, OBRA Thesis Group
Department of Computer, Information Sciences and Mathematics
University of San Carlos, Talamban Campus, Cebu City, Philippines
24100535@usc.edu.ph

Thesis Adviser: Christine F. Peña, MMath

cfpena@usc.edu.ph

Christoph Bartneck <christoph.bartneck@canterbury.ac.nz> Wed, Aug 5, 2026 at 5:34 AM
To: Lady Kaina Gutierrez <24100535@usc.edu.ph>

Hello,

Thanks for getting in touch. Yes, please use the scales and credit them accordingly.

Good luck with your project.

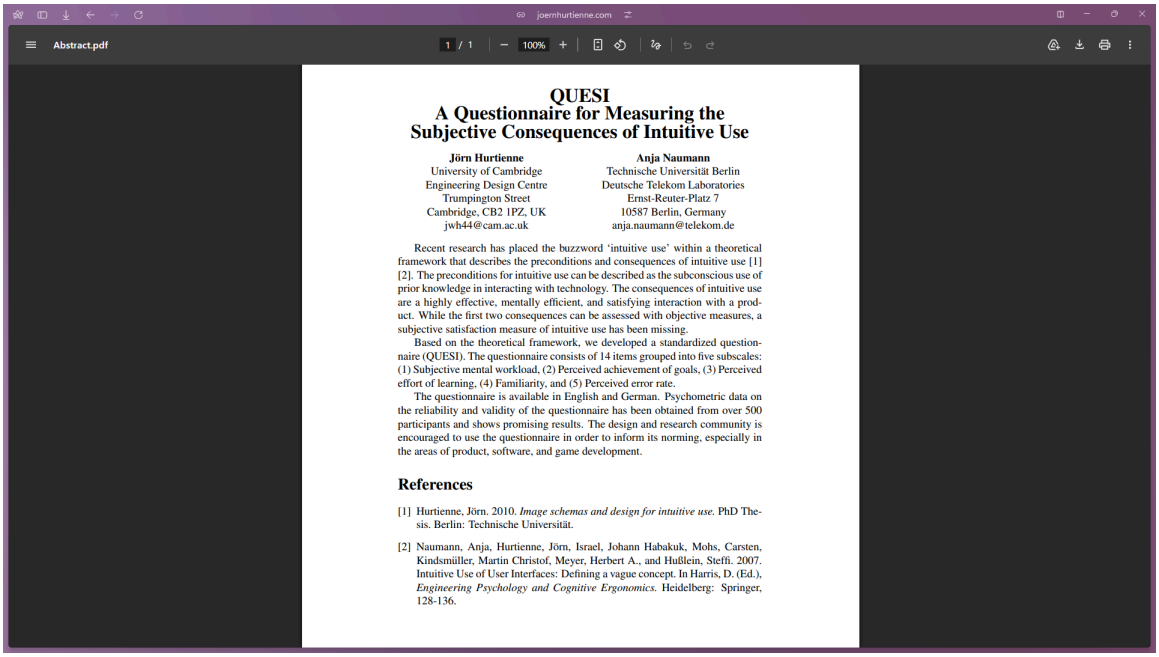
All the best,
/c

> On 4 Aug 2026, at 16:20, Lady Kaina Gutierrez <24100535@usc.edu.ph> wrote:
>

Note. Received 5 August 2026. Permission is conditional on due credit. The non-commercial term of the licence is satisfied, this study being unfunded and conducted solely for an academic requirement.

Figure B-6

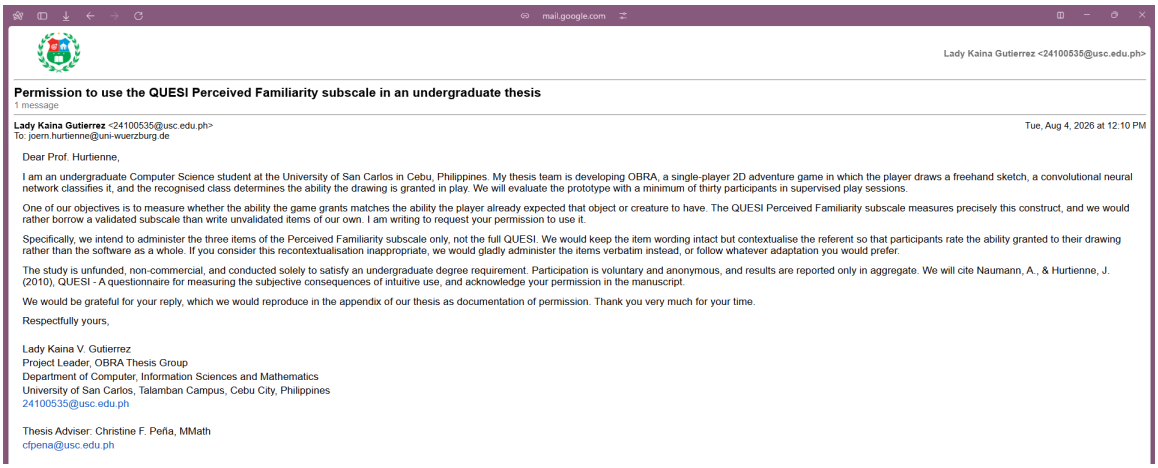
Published invitation to use the QUESI



Note. Passage in Hurtienne and Naumann (2010) in which the authors encourage the design and research community to use the questionnaire, naming game development among the areas of particular interest.

Figure B-7

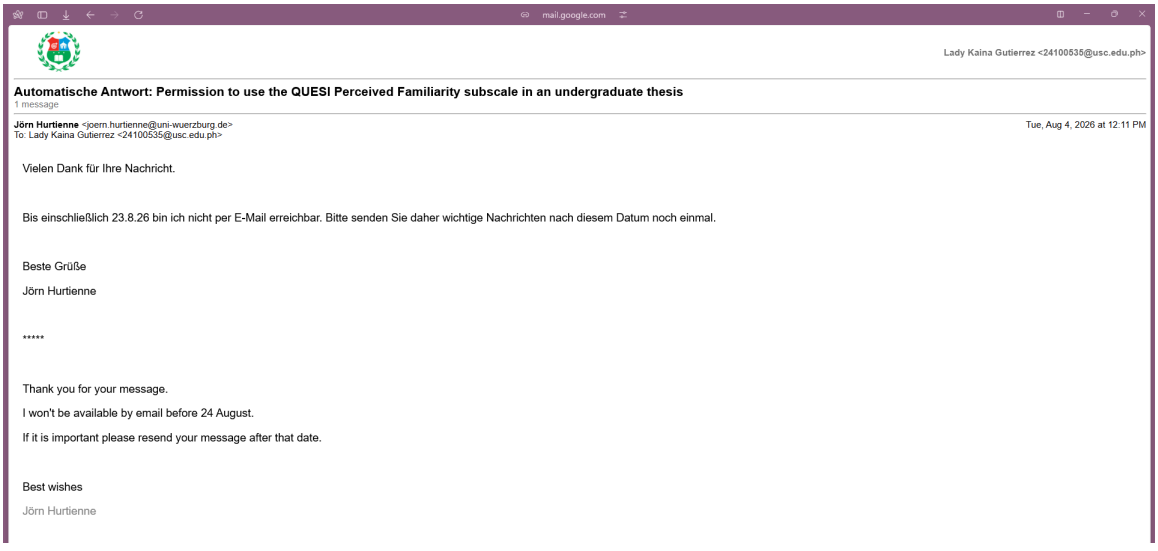
Request for permission to administer the QUESI Perceived Familiarity subscale, with delivery confirmation



Note. Message sent to Prof. Dr. Jörn Hurtienne, University of Würzburg, on 4 August 2026, and confirmed delivered.

Figure B-8

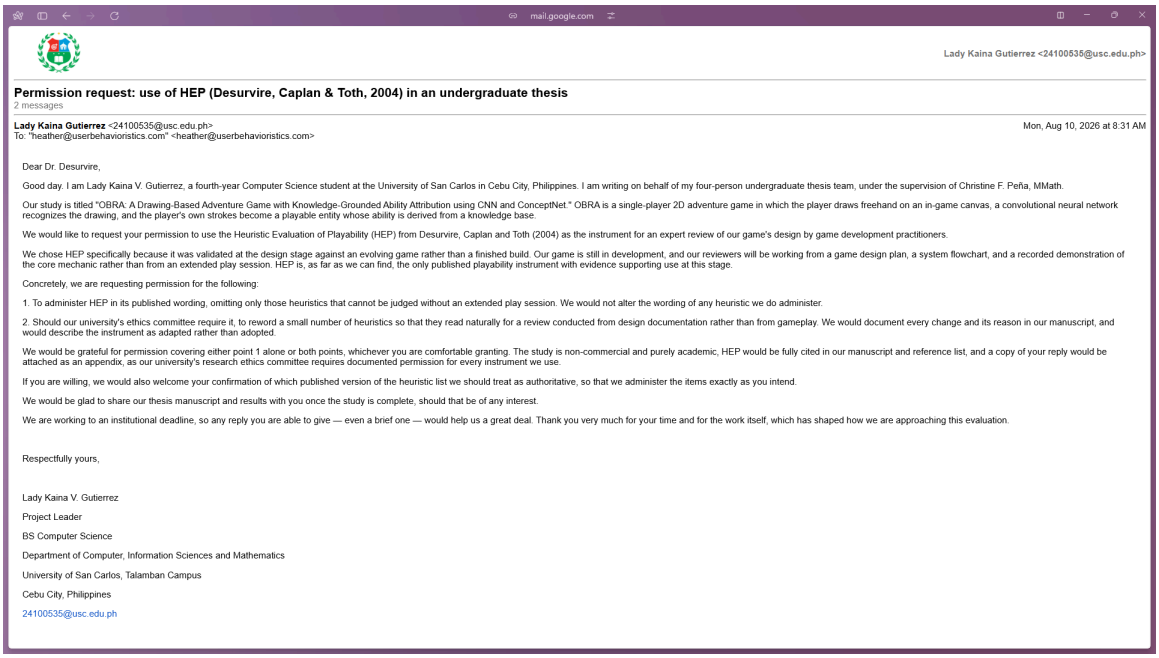
Automatic reply indicating unavailability of the addressee



Note. Received 4 August 2026, indicating unavailability until 24 August 2026 and requesting that important messages be resent after that date. The request is scheduled for resending on that date.

Figure B-9

Request for permission to administer the Heuristic Evaluation for Playability



mail.google.com

Lady Kaina Gutierrez <24100535@usc.edu.ph>

Permission request: use of HEP (Desurvire, Caplan & Toth, 2004) in an undergraduate thesis

2 messages

Lady Kaina Gutierrez <24100535@usc.edu.ph>
To: "heather@userbehavioristics.com" <heather@userbehavioristics.com>

Mon, Aug 10, 2026 at 8:31 AM

Dear Dr. Desurvire,

Good day. I am Lady Kaina V. Gutierrez, a fourth-year Computer Science student at the University of San Carlos in Cebu City, Philippines. I am writing on behalf of my four-person undergraduate thesis team, under the supervision of Christine F. Peña, MMath.

Our study is titled "OBRA: A Drawing-Based Adventure Game with Knowledge-Grounded Ability Attribution using CNN and ConceptNet." OBRA is a single-player 2D adventure game in which the player draws freehand on an in-game canvas, a convolutional neural network recognizes the drawing, and the player's own strokes become a playable entity whose ability is derived from a knowledge base.

We would like to request your permission to use the Heuristic Evaluation of Playability (HEP) from Desurvire, Caplan and Toth (2004) as the instrument for an expert review of our game's design by game development practitioners.

We chose HEP specifically because it was validated at the design stage against an evolving game rather than a finished build. Our game is still in development, and our reviewers will be working from a game design plan, a system flowchart, and a recorded demonstration of the core mechanic rather than from an extended play session. HEP is, as far as we can find, the only published playability instrument with evidence supporting use at this stage.

Concretely, we are requesting permission for the following:

1. To administer HEP in its published wording, omitting only those heuristics that cannot be judged without an extended play session. We would not alter the wording of any heuristic we do administer.

2. Should our university's ethics committee require it, to reword a small number of heuristics so that they read naturally for a review conducted from design documentation rather than from gameplay. We would document every change and its reason in our manuscript, and would describe the instrument as adapted rather than adopted.

We would be grateful for permission covering either point 1 alone or both points, whichever you are comfortable granting. The study is non-commercial and purely academic, HEP would be fully cited in our manuscript and reference list, and a copy of your reply would be attached as an appendix, as our university's research ethics committee requires documented permission for every instrument we use.

If you are willing, we would also welcome your confirmation of which published version of the heuristic list we should treat as authoritative, so that we administer the items exactly as you intend.

We would be glad to share our thesis manuscript and results with you once the study is complete, should that be of any interest.

We are working to an institutional deadline, so any reply you are able to give — even a brief one — would help us a great deal. Thank you very much for your time and for the work itself, which has shaped how we are approaching this evaluation.

Respectfully yours,

Lady Kaina V. Gutierrez

Project Leader

BS Computer Science

Department of Computer, Information Sciences and Mathematics

University of San Carlos, Talamban Campus

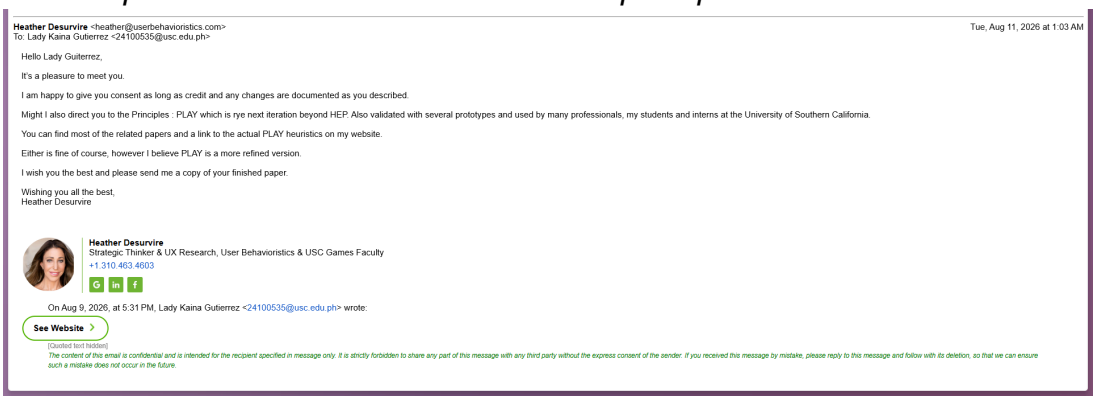
Cebu City, Philippines

24100535@usc.edu.ph

Note. Message sent to Dr. Heather Desurvire on August 10, 2026. Copyright in the source paper is held by the authors rather than by the Association for Computing Machinery, and the request was therefore addressed to the author directly.

Figure B-10

Grant of permission to administer the PLAY principles



Heather Desurvire <heather@userbehavioristics.com> Tue, Aug 11, 2026 at 1:03 AM
To: Lady Kaina Gutierrez <24100535@usc.edu.ph>

Hello Lady Guiterrez,

It's a pleasure to meet you.

I am happy to give you consent as long as credit and any changes are documented as you described.

Might I also direct you to the Principles : PLAY which is rye next iteration beyond HEP. Also validated with several prototypes and used by many professionals, my students and interns at the University of Southern California.

You can find most of the related papers and a link to the actual PLAY heuristics on my website.

Either is fine of course, however I believe PLAY is a more refined version.

I wish you the best and please send me a copy of your finished paper.

Wishing you all the best,
Heather Desurvire

Heather Desurvire
Strategic Thinker & UX Research, User Behavioristics & USC Games Faculty
+1.310.463.4603

On Aug 9, 2026, at 5:31 PM, Lady Kaina Gutierrez <24100535@usc.edu.ph> wrote:

See Website

[Quoted text hidden]

The content of this email is confidential and is intended for the recipient specified in message only. It is strictly forbidden to share any part of this message with any third party without the express consent of the sender. If you received this message by mistake, please reply to this message and follow with its deletion, so that we can ensure such a mistake does not occur in the future.

Note. Received August 11, 2026 from Dr. Heather Desurvire. Permission is conditional on credit and on documentation of any changes to the instrument. Both conditions are satisfied: the fifty principles are administered without alteration, and the instrument is cited in this thesis and will be cited in any associated publication. In the same reply the author recommended PLAY as the refined successor to HEP; the study follows that recommendation.

C. Call for participants

Call for Participants post draft:

Hi everyone. We're 3rd year Computer Science students from the University of San Carlos in Cebu, and we're looking for **3–5 game developers or recent game development graduates** willing to do a design review of our thesis game.

To be upfront: our game is still in development, so this is a **design review, not a playtest**. We'd send you a game design plan, a system flowchart, and a video demo of the core mechanic. A playable prototype may also be available depending on your platform, but it isn't required — the review is designed to be done from the documents.

The game is a 2D adventure game where the player draws freehand sketches, a CNN recognises the drawing, and the drawing itself becomes a playable entity with an ability based on what it is.

The review uses **PLAY (Desurvire & Wiberg, 2009)** — a published, validated set of 50 playability principles. We originally asked to use their earlier HEP heuristics and Dr. Desurvire kindly gave permission and pointed us to PLAY as the refined version, so that's what we're using. It takes about **45–60 minutes**. We'd ask for your role, years of experience, and a link to a portfolio or professional profile so we can report the panel's background in our thesis.

We need honest feedback and a signature on the form confirming it was reviewed by someone with game development experience. You can be credited by name or just by role and years of experience, your choice.

We're students with no budget, so we're hoping someone would be willing to help voluntarily. Critical feedback is far more useful to us than kind feedback, we'd rather find problems now than after we've built on them.

Thank you for your time.

PARTICIPANT SCREENING QUESTIONNAIRE

OBRA Gameplay Evaluation

Thank you for your interest in OBRA, a research project of the Department of Computer, Information Sciences and Mathematics. This short form is used only to check whether you are eligible to take part in a 60-minute play session, and to help us schedule sessions fairly. It takes about 3 minutes.

This is a screening form, not the consent form. If you are eligible, you will be sent a separate informed consent form before your session. Completing this form does not obligate you to take part.

Part A — Contact Information

Full name	
University email address	
Program and year level (or role, if not a student)	
Contact number (optional)	
Days and times you are generally available for a 60-minute session	

Part B — Eligibility

Please answer honestly. There are no right or wrong answers here — we are only checking fit with the study, not evaluating you.

#	Question	Yes	No
1	I am at least 18 years of age.	☐	☐
2	I am currently enrolled at the University of San Carlos, or I am a member of the same campus community (faculty, staff, or affiliated personnel).	☐	☐
3	I am able to operate a computer mouse or trackpad without assistance.	☐	☐
4	I am able to read and respond in English.	☐	☐
5	I am available for a single supervised session of approximately 60 minutes within the study's testing period.	☐	☐

Please answer the following as well. Answering "Yes" to any item below does not disqualify you from participating in other research at USC, it means only that this particular study is not a fit for you right now.

#	Question	Yes	No
6	I am one of the proponents of this study, the faculty adviser, or a person who has already seen the game's level design, its class roster, or its ability-attribution table.	☐	☐
7	I have already taken part in a pilot session for this same game build.	☐	☐
8	I have an uncorrected visual impairment that would prevent me from using a drawing canvas on screen.	☐	☐
9	I have a motor impairment that would prevent me from drawing freehand with a mouse, trackpad, or stylus.	☐	☐

Part C — A Little About You

These two questions help us group volunteers so the study includes people with a range of backgrounds. There are no right or wrong answers.

C1. How confident are you in your ability to draw simple objects or creatures by hand (for example, on paper or with a mouse/stylus)?

1 Not at all confident	2	3	4	5 Very confident
☐	☐	☐	☐	☐

C2. How familiar are you with playing video games?

1 Never play games	2 Rarely play	3 Play sometimes	4 Play regularly	5 Play daily or near-daily
☐	☐	☐	☐	☐

Thank you for completing this form. If you are eligible and selected, we will contact you at the email address above to schedule your session and to send the informed consent form. If you have any questions in the meantime, you may reach the research team at 24100535@usc.edu.ph.

Appendix D: ConceptNet Ability Attribution Table

#	Class	Group	Ability	Mode	ConceptNet derivation
1	bird	Creature	**Fly**	Direct	bird -CapableOf(2.83)-> fly
			Perch	Direct	bird -CapableOf(2.00)-> land_on_branch
			Nest	Direct	bird -CapableOf(2.83)-> build_nest
2	bat	Creature	**Fly**	Direct	bat -CapableOf(1.00)-> fly
			Cling	Direct	bat -CapableOf(1.00)-> hanging_upside_down
3	butterfly	Creature	**Flutter**	Direct	butterfly -CapableOf(2.00)-> fly
			Alight	Direct	butterfly -CapableOf(1.00)-> make_landing
			Draft	Direct	butterfly -CapableOf(1.00)-> cause_wind
4	spider	Creature	**Wall Cling**	Property	spider -HasProperty(1.00)-> sticking_to_wall
			Web	Direct	spider -CapableOf(2.83)-> spin_webs
			Venom	Direct	spider -CapableOf(2.00)-> inject_venom
5	monkey	Creature	**Hang**	Property	monkey -HasProperty(2.00)-> hanging_off_tree
			Swing	Direct	monkey -CapableOf(1.00)-> move_from_one_tree_to_another
			Use Tool	Direct	monkey -CapableOf(1.00)-> use_tool
			Throw	Direct	monkey -CapableOf(1.00)-> throw_banana
6	frog	Creature	**Leap**	Direct	frog -CapableOf(2.00)-> leap
			High Jump	Direct	frog -CapableOf(1.00)-> jump_very_high
			Tongue Grab	Direct	frog -CapableOf(1.00)-> catch_flies_with_tongue
7	fish	Creature	**Swim**	Direct	fish -CapableOf(2.00)-> swim

#	Class	Group	Ability	Mode	ConceptNet derivation
			Breathe Underwater	Part	fish -HasA(4.00)-> gills
			School	Direct	fish -CapableOf(1.00)-> school_together
8	shark	Creature	**Swim**	Direct	shark -CapableOf(1.00)-> swim
			Circle	Direct	shark -CapableOf(1.00)-> circle_swimmer
			Rough Hide	Part	shark -HasA(1.00)-> rough_skin
9	octopus	Creature	**Grapple**	Part	octopus -HasA(1.00)-> eight_tentacles
10	crab	Creature	**Armor**	Part	crab -HasA(1.00)-> external_skeleton
			Grip	Direct	crab -CapableOf(1.00)-> live_on_wet_rocks
11	horse	Creature	**Haul**	Direct	horse -CapableOf(3.46)-> pull_cart
			Gallop	Direct	horse -CapableOf(2.00)-> run_quickly
			Vault	Direct	horse -CapableOf(2.00)-> jump_fence
			Carry Rider	Action	horse -ReceivesAction(1.00)-> ridden
12	elephant	Creature	**Trunk Grab**	Part	elephant -HasA(3.46)-> trunk
			Tusk Ram	Part	elephant -HasA(3.46)-> tusks
			Lift	Direct	elephant -CapableOf(1.00)-> lift_tree
			Heavy	Direct	elephant -CapableOf(1.00)-> weight_1000_kilos
13	snake	Creature	**Slither**	Direct	snake -CapableOf(1.00)-> slither_down_street
			Squeeze	Part	snake -HasA(1.00)-> no_legs
			Startle	Direct	snake -CapableOf(1.00)-> surprise_person
14	sea turtle	Creature	**Shell**	Inherited	sea_turtle -IsA(2.00)-> turtle -CapableOf(1.00)-> hide_in_shell

#	Class	Group	Ability	Mode	ConceptNet derivation
			Amphibious	Inherited	sea_turtle -IsA(2.00)-> turtle -CapableOf(1.00)-> live_in_water_or_on_land
15	snail	Creature	**Move**	Inherited	snail -IsA(1.00)-> animal -CapableOf(4.00)-> move_itself_about
16	ant	Creature	**Carry**	Direct	ant -CapableOf(1.00)-> lift_several_times_own_weight
			Colony	Direct	ant -CapableOf(1.00)-> build_colony
17	bee	Creature	**Sting**	Direct	bee -CapableOf(2.00)-> sting
			Fly	Direct	bee -CapableOf(1.00)-> fly
18	pig	Creature	**Forage**	Direct	pig -CapableOf(2.00)-> find_truffles
			Wallow	Property	pig -HasProperty(1.00)-> rolling_in_mud
19	penguin	Creature	**Slide**	Hand-authored	No usable assertion; authored from the class's established locomotion (belly-sliding on ice)
20	scorpion	Creature	**Sting**	Direct	scorpion -CapableOf(1.00)-> sting
21	circle	Primitive	**Roll**	Property	circle -HasProperty(7.48)-> round
22	square	Primitive	**Platform**	Direct	square -UsedFor(1.00)-> walk_on
			Block	Part	square -HasA(3.46)-> four_sides
23	triangle	Primitive	**Wedge**	Part	triangle -HasA(6.63)-> three_sides
24	axe	Object	**Chop**	Direct	axe -CapableOf(1.00)-> cut_down_tree
			Split	Direct	axe -CapableOf(1.00)-> divide_wood_logs
25	sword	Object	**Cut**	Property	sword -HasProperty(5.29)-> sharp
			Strike	Direct	sword -UsedFor(4.47)-> killing
			Parry	Direct	sword -UsedFor(2.00)-> fencing

#	Class	Group	Ability	Mode	ConceptNet derivation
26	cannon	Object	**Launch**	Direct	cannon -CapableOf(3.46)-> fire_cannonball
			Bombard	Direct	cannon -UsedFor(3.46)-> shooting_cannonballs
			Heavy	Property	cannon -HasProperty(1.00)-> heavy
27	boomerang	Object	**Return**	Direct	boomerang -CapableOf(1.00)-> return_to_thrower
28	flashlight	Object	**Illuminate**	Direct	flashlight -UsedFor(2.00)-> seeing_in_dark
			Beam	Direct	flashlight -CapableOf(1.00)-> light_path
			Ward	Direct	flashlight -CapableOf(1.00)-> keep_wolf_at_bay
29	campfire	Object	**Burn**	Inherited	campfire -IsA(2.00)-> fire -CapableOf(7.21)-> burn_things
			Warm	Inherited	campfire -IsA(2.00)-> fire -CapableOf(4.00)-> warm_hands
			Cook	Inherited	campfire -IsA(2.00)-> fire -UsedFor(2.83)-> cook
30	cloud	Object	**Rain**	Direct	cloud -UsedFor(2.00)-> rain
			Shade	Direct	cloud -CapableOf(2.83)-> hide_sun
			Thunder	Direct	cloud -UsedFor(1.00)-> thunder
31	sun	Object	**Evaporate**	Direct	sun -CapableOf(6.00)-> dry_laundry
			Illuminate	Direct	sun -CapableOf(5.29)-> shine_brightly
			Heat	Direct	sun -CapableOf(4.90)-> heat_earth
			Scorch	Direct	sun -CapableOf(3.46)-> burn_skin
32	fan	Object	**Gust**	Direct	fan -UsedFor(4.47)-> moving_air
			Cool	Direct	fan -UsedFor(6.00)-> cool_off
33	ladder	Object	**Climb**	Action	ladder -ReceivesAction(1.00)-> climbed
			Reach	Direct	ladder -CapableOf(3.46)-> reach_roof

#	Class	Group	Ability	Mode	ConceptNet derivation
			Bridge Gap	Direct	ladder -UsedFor(1.00)-> using_as_bridge
34	stairs	Object	**Ascend**	Direct	stairs -UsedFor(5.29)-> climbing
			Descend	Direct	stairs -UsedFor(2.00)-> walking_down
35	parachute	Object	**Glide**	Direct	parachute -CapableOf(1.00)-> slow_descent_through_air
36	hot air balloon	Object	**Float**	Inherited	hot_air_balloon -IsA(2.00)-> balloon -CapableOf(1.00)-> rise_into_air
			Drift	Inherited	hot_air_balloon -IsA(2.00)-> balloon -CapableOf(1.00)-> float_through_air
37	bridge	Object	**Span**	Direct	bridge -UsedFor(4.00)-> crossing_river
			Cross Ravine	Direct	bridge -CapableOf(2.00)-> bridge_ravine
			Arch	Property	bridge -HasProperty(1.00)-> arched_to_make_stronger
38	sailboat	Object	**Wind Sail**	Action	sailboat -ReceivesAction(3.46)-> propelled_by_wind
			Sail	Inherited	sailboat -IsA(2.00)-> boat -UsedFor(7.21)-> travel_on_water
39	submarine	Object	**Submerge**	Direct	submarine -CapableOf(1.00)-> submerge_under_water
			Deep Dive	Direct	submarine -CapableOf(1.00)-> travel_deep_in_ocean
			Torpedo	Direct	submarine -CapableOf(1.00)-> shoot_torpedo
40	key	Object	**Unlock**	Direct	key -CapableOf(6.33)-> open_lock
			Open Door	Direct	key -CapableOf(6.00)-> open_doors

#	Class	Group	Ability	Mode	ConceptNet derivation
			Free	Direct	key -CapableOf(1.00)-> free_prisoner
41	door	Object	**Passage**	Action	door -ReceivesAction(4.00)-> opened
			Seal	Action	door -ReceivesAction(3.46)-> closed
			Lock	Action	door -ReceivesAction(2.00)-> locked
42	rake	Object	**Gather**	Action	rake -MannerOf-> gather
43	scissors	Object	**Cut**	Direct	scissors -UsedFor(5.66)-> cut
			Sharp Edge	Property	scissors -HasProperty(2.00)-> sharp
			Tear Cloth	Direct	scissors -CapableOf(1.00)-> tear_cloth
44	clock	Object	**Time**	Direct	clock -UsedFor(2.83)-> keeping_time
			Alarm	Direct	clock -UsedFor(2.83)-> set_alarm
			Tick	Direct	clock -CapableOf(2.83)-> continue_to_tick
45	anvil	Object	**Crush**	Direct	anvil -UsedFor(1.00)-> stun_cartoon_character
46	bucket	Object	**Carry Water**	Direct	bucket -UsedFor(4.47)-> carrying_liquids
			Douse	Direct	bucket -UsedFor(1.00)-> taking_water_to_fire
			Helmet	Direct	bucket -UsedFor(1.00)-> protecting_head
47	umbrella	Object	**Shield Rain**	Direct	umbrella -UsedFor(4.47)-> protection_from_rain
			Shade	Direct	umbrella -UsedFor(2.83)-> keeping_sun_off
			Catch Wind	Direct	umbrella -CapableOf(1.00)-> break_in_high_wind
48	tree	Object	**Grow**	Direct	tree -CapableOf(6.00)-> grow
			Shade	Direct	tree -UsedFor(3.46)-> shade
			Climb	Direct	tree -UsedFor(2.83)-> climbing

#	Class	Group	Ability	Mode	ConceptNet derivation
49	mushroom	Object	**Bounce**	Hand-authored	No usable assertion; authored from established platformer convention
50	wheel	Object	**Roll**	Direct	wheel -UsedFor(2.83)-> rolling
			Turn	Direct	wheel -UsedFor(2.00)-> turn

Note. Resolved offline against the ConceptNet 5.7.0 assertions dump; no query is issued at runtime. Primary abilities are set in bold; all remaining abilities for a class are secondary. An arrow denotes a single assertion, read start concept, relation, weight, end concept; two arrows denote an inherited attribution traversed through an *IsA* edge. Concept names are ConceptNet English nodes and retain their underscores. Weight in ConceptNet counts how many independent contributors asserted an edge, which measures how widely a fact is agreed upon rather than how useful the corresponding ability is to the game. It is therefore never the first filter. In the case where an inherited attribution is used, the weight given is that of the parent assertion supplying the ability. Every weight is taken from the dump; none is estimated.

Two classes have no usable assertion at any depth and their abilities are hand-authored and marked as such. *Penguin* is also the worked example for why inheritance requires a guard: penguin -IsA-> bird and bird -CapableOf-> fly, so an unguarded traversal rule would grant a penguin flight. *Mushrooms* also don’t have the properties in ConceptNet that the researchers require in game. In video games, *Mushrooms* are often creatively adapted into interactive environments to serve as trampolines, allowing characters to bounce to high platforms or perform aerial tricks, as seen in games such as Nintendo’s Super Mario (Nintendo EAD, 2013). Similar mechanics will be adapted to OBRA

Appendix E: Software Requirements Specification